

- [Поток \(Thread\) в Java и JVM](#)
 - [Процесс и поток](#)
 - [Потоки в JVM](#)
 - [Виды потоков в Java](#)
 - [Стек и куча — память потоков: общее и приватное](#)
 - [Создание и запуск потоков в коде](#)
 - [Thread vs Runnable — в чём разница?](#)
 - [start\(\) vs run\(\) — в чём разница?](#)
 - [Жизненный цикл — `Thread.State`](#)
 - [Главное отличие: BLOCKED vs WAITING](#)
 - [Цена многопоточности: Переключение контекста \(Context Switch\)](#)
- [Железо и память: откуда берутся гонки данных](#)
 - [Ядра CPU: Конкурентность vs Параллелизм](#)
 - [Кэши CPU: Анатомия задержек](#)
 - [Реальные причины гонок данных в Java](#)
 - [Демонстрация гонки данных \(Race Condition\)](#)
 - [Java Memory Model: happens-before](#)
- [Базовая синхронизация \(языковые примитивы\)](#)
 - [`join\(\)` и `interrupt\(\)` : дождаться потока и попросить его остановиться](#)
 - [`synchronized` — взаимное исключение и устройство монитора](#)
 - [Взаимодействие потоков: `wait\(\)`, `notify\(\)`, `notifyAll\(\)`](#)
 - [`volatile` — барьер памяти, не атомарность](#)
 - [`ThreadLocal` — изоляция вместо синхронизации](#)
- [Явные блокировки и синхронизаторы java.util.concurrent](#)
 - [`ReentrantLock` vs `synchronized`, `ReadWriteLock`, `StampedLock`](#)
 - [`Condition` — `wait` / `notify` для `Lock`](#)
 - [Deadlock / Livelock / Starvation — когда потоки не продвигаются](#)
 - [`CountDownLatch`, `CyclicBarrier`, `Semaphore` — синхронизаторы и общая основа AQS](#)
- [Lock-free: CAS и атомики](#)
 - [CAS и CAS-loop — как работает lock-free, проблема ABA](#)
 - [Atomic-типы: `AtomicInteger`, `AtomicReference`, `LongAdder`](#)
- [Пулы потоков](#)
 - [Пулы потоков и `ThreadPoolExecutor` в Java](#)
 - [`ForkJoinPool` в Java — work-stealing и parallel streams](#)

- [`Callable` / `Future` → `CompletableFuture`: асинхронные результаты](#)
 - [Конкурентные коллекции в Java](#)
 - [Почему обычные коллекции опасны](#)
 - [ConcurrentHashMap](#)
 - [CopyOnWriteArrayList](#)
 - [Блокирующие очереди \(`BlockingQueue`\)](#)
 - [Неблокирующие и упорядоченные коллекции](#)
 - [Как выбрать коллекцию](#)
 - [Virtual Threads](#)
 - [Virtual Threads \(Project Loom\): устройство и применение](#)
 - [Проблемы Virtual Threads: pinning, нативные вызовы, ThreadLocal](#)
 - [Production multithreading](#)
 - [Модели обработки запросов в бэкенде: Thread-per-Request, Event Loop, Virtual Threads](#)
 - [Закон Амдала и узкие места в продакшене](#)
 - [Thread Dump: диагностика многопоточных проблем](#)
 - [Вопросы к собеседованию](#)
 - [Теория и термины](#)
 - [Паттерны и принципы](#)
 - [Код с собеседований](#)
-

Поток (Thread) в Java и JVM

Процесс и поток

- **Процесс** — это выполняющаяся программа, под которую операционная система (ОС) выделяет изолированное адресное пространство. Процессы защищены друг от друга на уровне ядра ОС: один процесс не может напрямую читать или писать в память другого.
- **Поток (Thread)** — это наименьшая единица исполнения внутри процесса. Все потоки одного процесса **разделяют его общую память**, но имеют собственные независимые структуры для вычислений.

Почему поток на порядки дешевле процесса?

При создании нового процесса ОС вынуждена с нуля настраивать таблицы страниц памяти и выделять изолированные ресурсы. Новый поток внутри существующего процесса просто переиспользует уже созданное адресное пространство. От ОС ему

требуются только собственный стек, регистры и запись в таблице планировщика. Именно поэтому масштабировать приложение потоками проще и дешевле.

Потоки в JVM

Запуск команды `java -jar app.jar` создаёт процесс ОС — изолированный экземпляр JVM со своим адресным пространством. Внутри этого процесса сразу начинают работу потоки:

- Поток `main` — главный поток приложения, который автоматически стартует и выполняет метод `public static void main(String[] args)`.
- Служебные потоки JVM — системные потоки (потоки сборщика мусора (GC), JIT-компилятора, обработки сигналов). Они управляют самой виртуальной машиной. Они скрыты от прикладного кода, но активно потребляют ресурсы ОС.

Виды потоков в Java

С технической точки зрения (начиная с Java 21), все потоки в JVM делятся на два вида:

- **Platform Thread** (Платформенный поток) — классический `java.lang.Thread`, который существовал с самой первой версии Java (Java 1.0). Он жестко отображается на поток операционной системы в соотношении 1:1 и полностью планируется штатным планировщиком ОС. И поток `main`, и служебные потоки JVM по умолчанию являются платформенными. Они «тяжелые» для ОС, поэтому их количество ограничено.
- **Virtual Thread** (Виртуальный поток) — новый тип потоков, добавленный в Java 21. Это легкий поток, который управляется самой JVM, а не операционной системой. Множество виртуальных потоков могут выполняться поверх всего нескольких платформенных потоков (соотношение M:N). Они почти не занимают память ОС, создаются мгновенно, и их можно запускать миллионами без блокировки операционной системы.

Стек и куча — память потоков: общее и приватное

Все потоки JVM делят между собой общую память — Heap (Кучу). Любой объект в куче доступен из разных потоков одновременно.

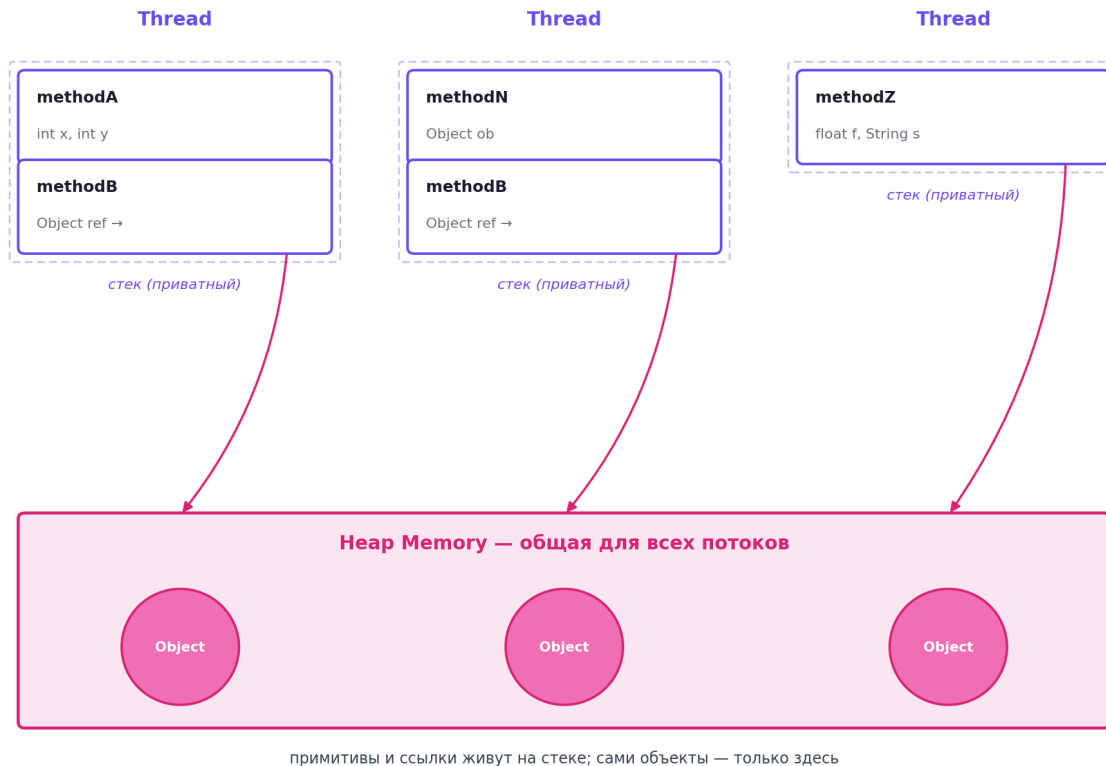
При этом у каждого потока есть своя приватная область:

- Стек вызовов (Stack) — выделяется из *native* памяти ОС под каждый поток. Хранит фреймы вызовов методов, локальные переменные и параметры. Локальные

переменные одного потока физически не видны другим потокам. Размер задается платформозависимым дефолтом (обычно 1 МБ)

- Program Counter (PC-регистр) — указатель на адрес текущей инструкции, которую поток выполняет в данный момент.

Память потоков: стек — приватный, куча — общая



Создание и запуск потоков в коде

В Java существует три основных способа объявить задачу для выполнения:

```
// Способ 1: Наследование от класса Thread (НЕ рекомендуется)
// Минус: Жесткая привязка к механизму потока, Java не поддерживает
// множественное наследование.
// 1 — наследование от Thread
class Worker extends Thread {
    @Override
    public void run() {
        System.out.println("работаю в " + getName());
    }
}
new Worker().start();
```



```
// 2 — Runnable через отдельный класс
class Task implements Runnable {
    @Override
    public void run() {
        System.out.println("работаю в " + Thread.currentThread().getName());
    }
}
new Thread(new Task(), "worker-2").start();

// 3 — Runnable как лямбда, самый лаконичный вариант
Runnable task = () -> System.out.println("работаю в " +
Thread.currentThread().getName());
new Thread(task, "worker-1").start();
```

Thread vs Runnable — в чём разница?

- `Thread` — это управляющая надстройка над физическим потоком ОС (инфраструктура выполнения).
- `Runnable` — это чистая абстракция над задачей (что именно нужно сделать).

Почему композиция через `Runnable` лучше наследования от `Thread` ?

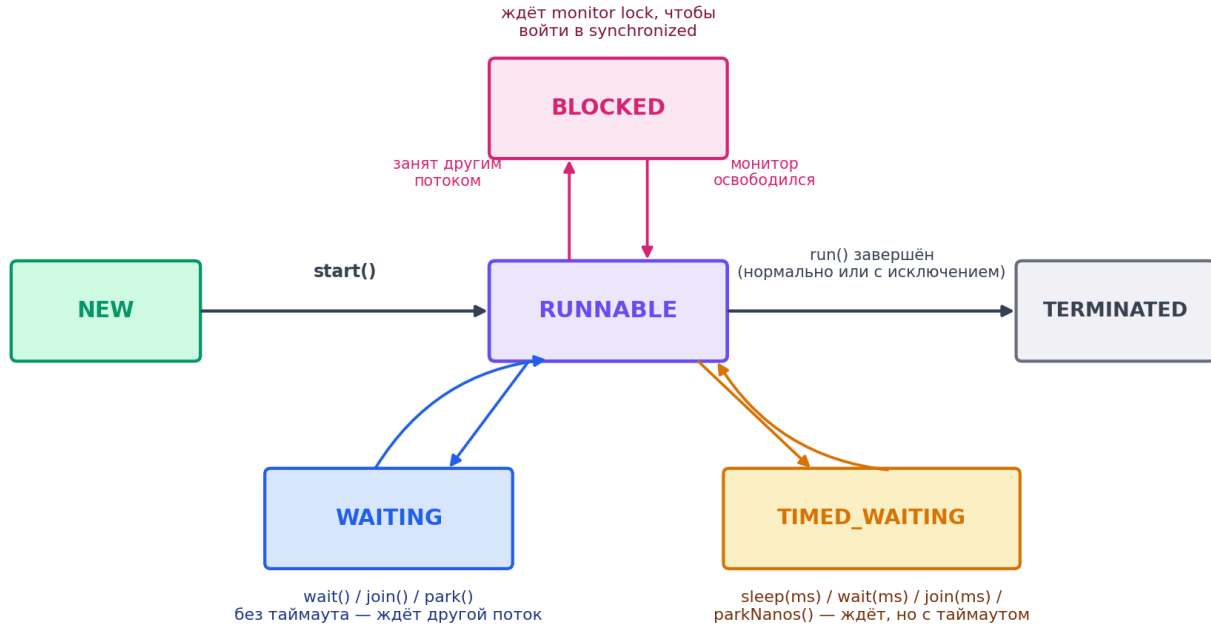
Мы разделяем *задачу* и *механизм выполнения*. Код становится гибким: одну и ту же задачу `Runnable` можно передать в обычный `Thread`, отправить в пул потоков (`ExecutorService`) или запустить внутри `Virtual Thread`. Тестировать и переиспользовать такой код значительно проще. Наследование от `Thread` нужно только если вы создаете свою кастомную инфраструктуру потоков.

`start()` vs `run()` — в чём разница?

- Метод `start()` — запрашивает у ОС выделение нового системного потока, регистрирует его в JVM и планирует выполнение метода `run()` в этом новом потоке.
- Метод `run()` — это обычный вызов метода в Java. Если вызвать его напрямую (`thread.run()`), код выполнится синхронно в текущем потоке, никакой новый поток не создастся.
- Повторный вызов `start()` на одном и том же объекте потока запрещен и всегда бросает `IllegalThreadStateException` (даже если поток уже завершил работу).

Жизненный цикл — `Thread.State`

Жизненный цикл потока: Thread.State



Состояние	Триггер
NEW	Объект создан, <code>start()</code> не вызван
RUNNABLE	На ядре или в очереди на него
BLOCKED	Ждёт <code>monitor lock</code> (<code>synchronized</code>)
WAITING	<code>wait()</code> , <code>join()</code> , <code>park()</code> без таймаута
TIMED_WAITING	<code>sleep(ms)</code> , <code>wait(ms)</code> — проснётся сам
TERMINATED	<code>run()</code> завершился, назад пути нет

Главное отличие: BLOCKED vs WAITING

- BLOCKED** — это *вынужденное* ожидание на входе в критическую секцию. Поток заблокирован на уровне механизма `synchronized` (ждет конкретный монитор объекта). Он не может проснуться сам, пока монитор не освободится.
- WAITING** — это *добровольное* ожидание. Поток сам отдал процессорное время, выполнив команду ожидания. В этот момент он может вообще не претендовать ни на какой `synchronized`-монитор. Чтобы выйти из этого состояния, ему нужен внешний триггер (например, `notify()` от другого потока) или завершение потока, у которого вызвали `join()`.

Цена многопоточности: Переключение контекста (Context Switch)

Реально на процессоре одновременно исполняется ровно столько потоков, сколько ядер (или потоков гипертрединга) доступно в системе. Если потоков больше, **Планировщик ОС** нарезает время на кванты.

- **Состояние RUNNABLE** — поток либо выполняется на ядре, либо ждёт своей очереди. Когда поток блокируется (I/O, сокет, блокировки), планировщик снимает его с ядра и отдаёт процессор следующему.
- **Приоритет потока** ($1 - 10$) — это лишь подсказка (хинт) для ОС. JDK не гарантирует порядок выполнения, финальное решение всегда принимает планировщик конкретной операционной системы. Константы: `MIN_PRIORITY = 1` , `NORM_PRIORITY = 5` , `MAX_PRIORITY = 10` . У виртуальных потоков приоритет всегда `NORM_PRIORITY` , `setPriority` игнорируется.
- **`Thread.yield()`** — подсказка планировщику: «я готов уступить процессор». Планировщик вправе её проигнорировать. В продакшене почти не применяется (по документации: «rarely appropriate»), полезна для отладки: помогает воспроизводить гонки.
- **Переключение контекста (Context Switch)** — процесс снятия одного потока с ядра и постановки другого (на Linux занимает **~1.2–2.2 мкс**). Включает сохранение/загрузку регистров и «остывание» кэша процессора (*cache misses*), пока новый поток заново подтягивает данные из RAM.

Масштаб цен: Одно обращение к L1-кэшу занимает ~1 нс. Время одного переключения контекста (~1.5 мкс) эквивалентно выполнению 1500–2000 операций чтения из сверхбыстрого кэша L1.

Железо и память: откуда берутся гонки данных

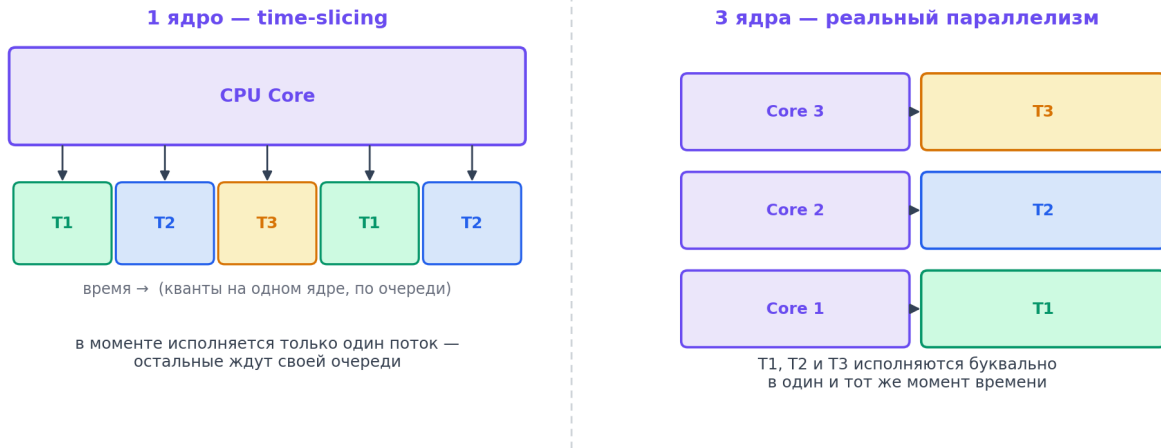
Ядра CPU: Конкурентность vs Параллелизм

Одно ядро процессора способно выполнять ровно одну инструкцию в один момент времени.

- **Конкурентность** (Concurrency) — иллюзия одновременной работы множества потоков на одном ядре. Достигается за счет работы планировщика ОС, который нарезает время на кванты и стремительно переключает потоки между собой (*time-slicing / context switch*).

- **Параллелизм (Parallelism)** — истинно одновременное выполнение задач, которое физически возможно только при наличии нескольких ядер CPU.

Конкурентность vs параллелизм



Закон Амдала

Кажется, что ускорение программы пропорционально росту числа ядер, но это лишь верхняя теоретическая граница. Предел описывает Закон Амдала:

$Speedup = \frac{1}{(1-p) + \frac{p}{N}}$ где p — доля параллелизуемого кода, а N — число ядер. Если 30% кода по логике приложения обязаны выполняться последовательно ($p = 0.7$), то даже бесконечное число ядер не ускорит программу более чем в 3.3 раза. Дополнительно выигрыш уменьшают накладные расходы ОС на синхронизацию потоков.

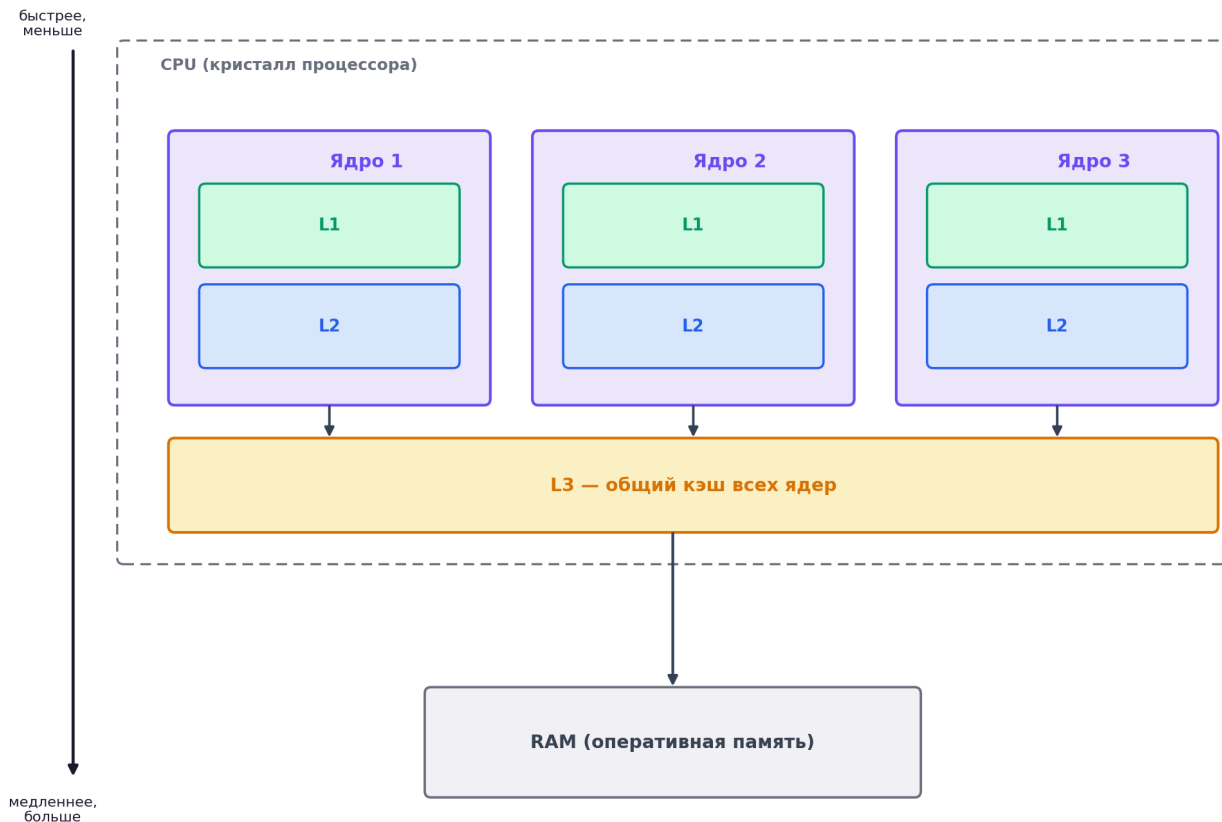
Кэши CPU: Анатомия задержек

Процессор работает значительно быстрее, чем оперативная память (RAM). Чтобы сгладить этот разрыв, ядра CPU используют иерархию кэшей. У каждого ядра есть приватные быстрые кэши L1 и L2, а кэш L3 — общий для всего процессора.

Уровень кэша	Типичная задержка	Относительное замедление
L1	~1 нс (3–5 тактов ядра)	1× (База)
L2	~3–7 нс	~5–10× медленнее
L3	~10–20 нс	~15–30× медленнее
RAM (Основная память)	~50–100 нс	~100–200× медленнее

Иерархия памяти CPU

чем ближе к ядру — тем быстрее и меньше объём



Ключевой вывод: Между L3 и RAM лежит огромный инфраструктурный обрыв — скорость доступа падает на порядок.

Механика работы кэша:

1. При обращении к переменной ядро ищет её последовательно: $L1 \rightarrow L2 \rightarrow L3 \rightarrow RAM$. Ситуация, когда данных нет в кэше, называется Cache Miss. Чем дальше приходится идти, тем дороже обходится операция.
2. Данные загружаются из RAM не по одной переменной, а блоками — Cache Lines (кэш-линиями) обычно по 64 байта. Поэтому последовательный обход массива в памяти происходит значительно быстрее, чем чтение связанных списков со случайным распределением объектов.
3. Первое чтение всегда идет до RAM (Cold Miss), после чего данные оседают в кэшах. Запись происходит аналогично, но в обратную сторону: сначала в L1, а затем постепенно «стекает» вниз.

Реальные причины гонок данных в Java

В основе гонок данных и багов многопоточности лежат три архитектурные причины на уровне компилятора (JIT) и процессора:

1. Кэширование в регистрах (Проблема видимости). JIT-компилятор ради оптимизации имеет право навсегда «запереть» переменную в регистре процессора (например, внутри бесконечного цикла `while(!flag)`), если текущий поток её не меняет. В этом случае поток вообще перестает перечитывать данные из общей памяти и никогда не увидит изменений, сделанных другим потоком.
2. Отсутствие гарантии видимости и перестановка (Reordering). Без ключевых слов `volatile` или `synchronized` компилятор и процессор вправе менять порядок выполнения инструкций местами ради оптимизации конвейера. Запись одного потока может оставаться невидимой для другого произвольно долго.
3. Неатомарность составных операций. Операция `value++` на уровне байт-кода состоит из трех шагов: *прочитать* → *изменить* → *записать*. Без синхронизации потоки перемешивают эти шаги, считывая старое значение до того, как его успеет обновить соседний поток.

Все эти эффекты формализует **Java Memory Model** (JMM). Через правила happens-before она жестко определяет контракт: при каких условиях действия одного потока гарантированно и вовремя станут видны другому.

💡 Разрушение мифа: Дело не в кэшах CPU

Часто считают, что гонки происходят из-за физической рассинхронизации или «поломки» кэшей процессора. Это не так. На уровне железа современные процессоры аппаратно держат строгую согласованность кэшей через протоколы когерентности (например, MESI). Ядра физически обмениваются данными и увидят изменения *когда-нибудь*. Проблема не в железе, а в компиляторе (JIT), который может решить вообще не смотреть в память, или в отсутствии контракта программы, обязывающего показать изменения *вовремя*.

Демонстрация гонки данных (Race Condition)

Ниже представлен классический пример, доказывающий, что общая память без координации потоков приводит к непредсказуемым результатам из-за неатомарности операций:

```
// Классический пример гонки данных (Race Condition)
class Counter {
    private int value = 0;
    public void increment() { value++; } // Неатомарно: read -> add -> write
    public int get() { return value; }
}
```

```

public class RaceDemo {
    public static void main(String[] args) throws InterruptedException {
        Counter counter = new Counter();
        Runnable task = () -> {
            for (int i = 0; i < 100_000; i++) counter.increment();
        };

        Thread t1 = new Thread(task); Thread t2 = new Thread(task);
        t1.start(); t2.start();
        t1.join(); t2.join(); // Ждем завершения

        // Ожидаем 200 000, но выведется МЕНЬШЕ (значение меняется от
        // запуска к запуску)
        System.out.println(counter.get());
    }
}

```

Почему потерялись инкременты?

Поток `t1` и поток `t2` одновременно прочитали `value = 0`. Оба прибавили единицу в своих регистрах (получили `1`). Затем `t1` записал `1` в кучу, а следом `t2` записал туда же `1`. Вместо двух инкрементов произошел только один. Аппаратный кэш при этом работал идеально, но логика приложения была нарушена из-за отсутствия атомарности и синхронизации.

Java Memory Model: happens-before

Модель памяти Java (Java Memory Model, JMM) описывает поведение потоков в среде исполнения Java. Это часть семантики языка Java, набор правил, описывающий выполнение многопоточных программ и правил, по которым потоки могут взаимодействовать друг с другом посредством основной памяти.

Формально модель памяти определяет набор действий межпоточного взаимодействия (эти действия включают в себя, в частности, чтение и запись переменной, захват и освобождений монитора, чтение и запись `volatile` переменной, запуск нового потока), а также модель памяти определяет отношение между этими действиями - *happens-before* - абстракции обозначающей, что если операция `X` связана отношением `happens-before` с операцией `Y`, то весь код следуемый за операцией `Y`, выполняемый в одном потоке, видит все изменения, сделанные другим потоком, до операции `X`.

Зачем нужна JMM

Если бы не было четких правил, многопоточные программы в Java работали бы непредсказуемо из-за оптимизаций. Чтобы программа работала быстрее, система постоянно переставляет инструкции местами. Для одного изолированного потока такие перестановки абсолютно безопасны, но в многопоточной среде они легко ломают логику программы и приводят к рассинхронизации данных.

Этот процесс называется переупорядочиванием инструкций (instruction reordering), и происходит он на трех уровнях:

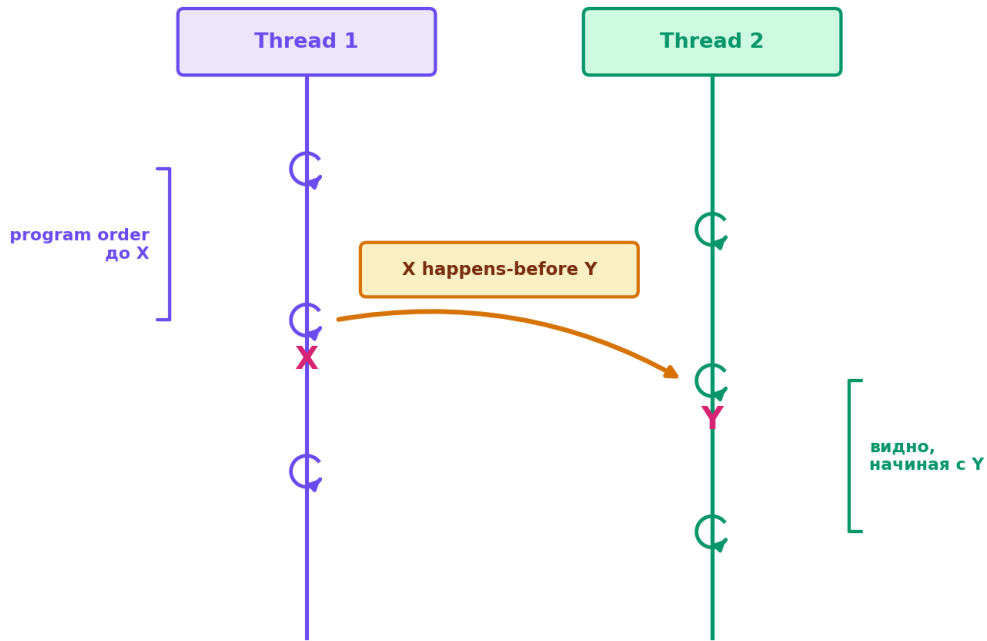
Кто переупорядочивает	Как это устроено (простыми словами)
Компилятор и JIT	Оптимизируют код. Переставляют местами или вообще удаляют строки кода (например, неиспользуемые переменные или пустые циклы), если уверены, что для текущего потока конечный результат вычислений не изменится.
Процессор (CPU)	Выполняет команды вне очереди. Чтобы не простаивать в ожидании данных, процессор берет в работу те задачи, для которых прямо сейчас готовы аргументы, игнорируя их исходный порядок в коде.
Память и кэши	Используют буферы записи. Когда поток что-то записывает, данные сначала попадают в быстрый локальный буфер ядра процессора и не сразу доходят до общей памяти. Из-за этого другие ядра видят изменения с задержкой и в случайном порядке.

Чтобы внутри программы не начался хаос, Модель памяти Java (JMM) берет эти процессы под контроль. Она строго описывает базовые действия потоков (чтение и запись обычных или `volatile` переменных, блокировки, запуск и остановку потоков) и дает им контракт.

Если ваш код выходит за рамки этого контракта, возникает состояние гонки (data race), при котором программа начинает вести себя абсолютно непредсказуемо.

happens-before

Семантика happens-before



Основные рёбра happens-before

Рёбро	Условие
program order	действие A в одном потоке happens-before последующему действию B этого же потока
monitor	unlock() монитора happens-before последующему lock() того же монитора
volatile	запись в volatile-поле happens-before последующему чтению того же поля
thread start	вызов Thread.start() happens-before первым действиям запущенного потока
thread termination	последнее действие потока happens-before успешному возврату из t.join() или возврату false из t.isAlive(), дождавшегося этого потока

Это и есть пять рёбер, которые реально нужны на практике — на них строятся все примеры ниже. Есть ещё два редких правила, их достаточно просто знать, что они существуют: завершение конструктора happens-before вызову `finalize()` объекта (сам `finalize()` deprecated с Java 9, в бою не используется); вызов `t.interrupt()` happens-

before тому моменту, когда поток `t` это обнаруживает — через `InterruptedException` либо `isInterrupted()` / `interrupted()`.

Главная идея: не нужно запоминать отдельные магические свойства `volatile`, `synchronized` и `join()` — нужно уметь построить цепочку happens-before между действиями.

```
// program order – A hb B внутри одного потока
data = 42; // A
System.out.println(data); // B – видит 42

// thread start – start() hb первому действию run()
box[0] = 42;
child.start(); // поток увидит 42 без volatile

// monitor – unlock(LOCK) hb следующему lock(LOCK) того же монитора
synchronized (LOCK) { shared = 9000; } // writer: unlock
synchronized (LOCK) { print(shared); } // reader: lock – видит 9000

// volatile – запись hb следующему чтению того же поля
data = 777; ready = true; // writer
while (!ready) {} // reader
print(data); // видит 777 без synchronized

// thread termination – последнее действие потока hb возврату из
join()/isAlive()==false
worker.start();
worker.join();
print(box[0]); // гарантированно видно
```

«Без happens-before: reordering и broken publication

Пять рёбер выше работают только там, где между потоками реально есть `synchronized` / `volatile` / `start()` / `join()`. Если ни одного из них нет — JVM вправе переставлять независимые по данным операции как угодно: у компилятора и процессора нет обязательства сохранять порядок для стороннего наблюдателя.

```
// Поток 1: x = 0; r1 = y;
// Поток 2: y = 0; r2 = x;
// без hb между потоками результат r1 == 1 && r2 == 1 – возможен,
// хотя недостижим ни при одном последовательном чередовании этих строк
```

Тот же эффект бьёт по публикации объектов: ссылка видна раньше, чем успели записаться его поля (**broken publication**).

```
static Holder holder; // ⚠️ HE volatile — барьера нет

class Holder {
    int a, b;
    Holder() { a = b = 1; }
}

// writer: holder = new Holder();
// reader: Holder h = holder;
//          if (h != null && h.a != 1) anomalies++; // 🔥 возможно даже при h
//          != null
```

Не противоречит главе 5 («volatile защищает ссылку, не объект») — там про мутацию уже опубликованного объекта, здесь — про сам момент публикации.

final-поля — safe publication без volatile

После того как объект **корректно создан** (ссылка не «утекла» наружу до завершения конструктора), любой поток видит его `final`-поля без синхронизации — основа immutable-объектов.

```
final class Point {
    final int x, y;
    Point(int x, int y) { this.x = x; this.y = y; }
}
```

Оговорки: `final` можно изменить через reflection; `final`-константу JIT мог «зашить» в байт-код, так что изменение после создания объекта может не иметь эффекта. Поэтому `final`-поля выставляют только в конструкторе.

Базовая синхронизация (языковые примитивы)

`join()` и `interrupt()` : дождаться потока и попросить его остановиться

`join()` — ожидание завершения потока

`t.join()` блокирует вызывающий поток, пока `t` не завершится. Вызывающий поток при этом в состоянии `WAITING` (`join(ms)` — `TIMED_WAITING`): тот же механизм, что у `wait()` / `sleep()`, только другое условие пробуждения. Ещё `join()` создаёт happens-before ребро: всё, что сделал `t`, гарантированно видно после возврата из `join()`.

Правило: звать там, где результат нужен прямо сейчас. Если ждать нужно несколько потоков, сначала запустить все, потом пройтись `join()` по каждому:

```
// ✅ потоки работают параллельно
for (Thread t : threads) t.start();
for (Thread t : threads) t.join();

// ❌ каждый следующий стартует после полного завершения предыдущего —
параллелизма нет
for (Thread t : threads) { t.start(); t.join(); }
```

`interrupt()` — просьба, а не команда

В Java нет безопасного способа принудительно остановить чужой поток: `Thread.stop()` мог оборвать поток посреди изменения общего состояния, поэтому он убран из рабочего API (в Java 20+ просто бросает `UnsupportedOperationException`). Остаётся кооперативная отмена: `interrupt()` говорит «посмотри на себя и реши сам». Поток вправе проигнорировать запрос — это дизайн, а не баг: только сам поток знает, когда безопасно остановиться.

Как запрос доходит до потока:

- **поток работает** — `interrupt()` ставит флаг прерывания (внутренний `boolean`), поток должен сам его проверять;
- **поток заблокирован в `sleep()` / `wait()` / `join()`** — JVM будит его раньше срока, сбрасывает флаг в `false` и бросает `InterruptedException`;
- **поток ждёт вход в `synchronized (BLOCKED)`** — `interrupt()` его не разбудит: вход в монитор не прерываем. Для прерываемого ожидания лока нужен `Lock.lockInterruptibly()`.

`InterruptedException` — не «что-то сломалось», а доставленный сигнал отмены.

Флаг прерывания

- `thread.isInterrupted()` — читает флаг, ничего не меняет;
- `Thread.interrupted()` (статический) — читает флаг и сбрасывает в `false`.

К моменту `catch (InterruptedException e)` флаг уже `false`: JVM стёрла его, когда бросала исключение. Поэтому внутри `catch` вызывают `Thread.currentThread().interrupt()` — это восстановление информации, а не защитная приписка. Без него факт прерывания теряется: ни этот метод, ни код выше по стеку не узнают, что поток просили остановиться.

```

Thread worker = new Thread(() -> {
    while (!Thread.currentThread().isInterrupted()) {
        try {
            doChunkOfWork();
            Thread.sleep(50);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt(); // вернуть флаг, который
            стёрла JVM
            break; // и реально завершить
            работу
        }
    }
    System.out.println("worker: graceful stop");
});

worker.start();
// ...
worker.interrupt();
worker.join(); // дождаться, пока он аккуратно завершится

```

Антипаттерн: поймал и проигнорировал

```

catch (InterruptedException e) {
    // ничего: ни break, ни восстановления флага
}

```

Компилируется и «работает», но поток получил явную просьбу остановиться и молча её проигнорировал, а остальной код об этом не узнает. Минимум: восстановить флаг и выйти из цикла/метода.

Альтернатива блокирующему ожиданию — опрос в цикле (busy loop). Почему это плохо — в разделе про планировщик.

synchronized — взаимное исключение и устройство монитора

Что даёт synchronized

Главное — **взаимное исключение**: в критической секции на одном мониторе одновременно находится не больше одного потока. Видимость (ребро monitor из блока про JMM) — следствие, а не цель.

У каждого объекта есть встроенный монитор (intrinsic lock). Три формы записи:

```

class Counter {
    private int value;
    private final Object lock = new Object();

    synchronized void inc1() { value++; }           // монитор this
    void inc2() { synchronized (this) { value++; } } // то же самое, явно
    void inc3() { synchronized (lock) { value++; } } // приватный объект-
замок
    static synchronized void sInc() { }             // монитор
Counter.class, HE this
}

```

Форма	Какой монитор
synchronized -метод	this
synchronized (obj) {}	obj
static synchronized -метод	Counter.class

static synchronized и instance- synchronized друг друга **не блокируют** — это разные мониторы. Взаимное исключение работает только между потоками, берущими **один и тот же** монитор.

Что происходит с потоком

```

synchronized (lock) {    // монитор занят → поток BLOCKED
    // ...                // монитор освободелся → один из ждущих получает его
→ RUNNABLE
}

```

- ждёт вход в synchronized → BLOCKED (не WAITING : тот — про wait() / join() / park());
- ожидание монитора **не прерывается** через interrupt() — для этого Lock.lockInterruptibly() ;
- выход из блока (штатный или по исключению) всегда освобождает монитор.

Реентерабельность

Поток, уже владеющий монитором, входит в synchronized на нём повторно без самоблокировки:

```
synchronized void outer() { inner(); }    // тот же this → не блокируется
synchronized void inner() { }
```

Цена `synchronized`

Счётчик под `synchronized` корректен (2 потока × 100 000 инкрементов → всегда 200 000), но потоки в секции идут строго по очереди.

- при конкуренции ждущие потоки паркуются в ОС, пробуждение `BLOCKED` → `RUNNABLE` — переключение контекста;
- чем длиннее критическая секция, тем хуже масштабирование.

Для простых счётчиков дешевле CAS-атомики — отдельный блок.

Монитор объекта

Монитор — внутренний механизм синхронизации в Java. Логически у него две очереди:

```
// модель, а не реальный код JVM
class ObjectMonitor {
    Thread owner;           // кто держит монитор
    Queue<Thread> entryList; // ждут входа в synchronized(obj)
    Queue<Thread> waitList;  // ждут notify/notifyAll
}
```

- `wait()` → освобождает монитор → поток уходит в `waitList` (`WAITING`)
- `notify()` → один поток из `waitList` → в `entryList`
- `notifyAll()` → все потоки из `waitList` → в `entryList`

⚠ Пробуждение ≠ немедленный вход. Поток из `waitList` снова становится `BLOCKED` и должен повторно захватить монитор, прежде чем продолжить выполнение.

Где этот объект живёт. Состояние лока хранится в **Mark Word** — части заголовка объекта (два младших бита: `01` не заблокирован, `00` thin, `10` inflated). Пока конкуренции нет, обходятся CAS в заголовке — дёшево. Конкуренция или `wait()` → JVM создаёт настоящий `ObjectMonitor` с очередями выше, а заголовок указывает на него; ждущие потоки паркуются через ОС — это дорогой путь. Biased locking в современной JVM нет (отключён в JDK 15, удалён в JDK 18).

Взаимодействие потоков: `wait()`, `notify()`, `notifyAll()`

`wait()`, `notify()` и `notifyAll()` — механизм взаимодействия потоков через монитор объекта. Он позволяет потоку не занимать CPU, пока нужное состояние не изменится.

Если условие ещё не выполнено, поток вызывает `wait()` . При этом он **освобождает монитор и переходит в состояние ожидания**:

```
synchronized (lock) {  
    while (!condition) {  
        lock.wait();  
    }  
}
```

Другой поток изменяет состояние и отправляет уведомление:

```
synchronized (lock) {  
    condition = true;  
    lock.notifyAll();  
}
```

`notify()` будит один ожидающий поток, а `notifyAll()` — все ожидающие. Однако после уведомления поток не продолжает работу сразу: сначала он должен снова получить монитор.

Условие всегда проверяется через `while` , а не `if` :

```
while (!condition) {  
    lock.wait();  
}
```

Это необходимо, потому что после пробуждения условие могло снова стать ложным или поток мог проснуться без уведомления (**spurious wakeup**).

`wait()` , `notify()` и `notifyAll()` можно вызывать только при владении соответствующим монитором, то есть внутри `synchronized` . Иначе будет `IllegalMonitorStateException` .

Важно

- `wait()` выбрасывает `InterruptedException` , поэтому метод должен либо объявить `throws InterruptedException` , либо обработать исключение через `try/catch` .
- `notify()` **не освобождает монитор**. Поток, вызвавший `notify()` , продолжает держать монитор до выхода из `synchronized` , и только после этого разбуженный поток сможет его получить.
- В большинстве случаев предпочтительнее `notifyAll()` . `notify()` имеет смысл использовать, когда все ожидающие потоки равнозначны и гарантированно можно

разбудить любой из них.

- Состояние, которое проверяется в `while`, и его изменение должны выполняться под **одним и тем же монитором**. Это предотвращает проблемы с потерянными уведомлениями (`lost wakeup`).

`volatile` — барьер памяти, не атомарность

Что даёт `volatile`

Две гарантии для **одной переменной**, и ни одной сверх этого:

- **видимость** — запись в `volatile`-поле становится видна последующим чтениям этого поля из любого потока;
- **порядок** — операции с `volatile` нельзя переставлять с другими операциями потока (hb-ребро `volatile` из блока про JMM: что записано до `volatile`-записи, видно после чтения того же поля).

Взаимного исключения нет: два потока по-прежнему могут одновременно менять поле.

Классика: флаг остановки

```
private volatile boolean running = true;

void work() {
    while (running) { /* ... */ }    // без volatile — может не завершиться
    никогда
}

void stop() { running = false; }
```

Без `volatile` JIT вправе прочитать `running` один раз в регистр и не перечитывать: внутри цикла поток сам поле не меняет. Поэтому `running = false` из другого потока такой цикл может не заметить вообще.

Чего `volatile` не делает: составные операции

```
private volatile int counter;

void inc() { counter++; }    // read → add → write: три шага, гонка остаётся
```

Два потока по 100 000 вызовов `inc()` обычно дают результат меньше 200 000. Нужен `synchronized` или `AtomicInteger`.

Критерий	<code>synchronized</code>	<code>volatile</code>
Защищает от гонок в составных операциях	✓	✗
Гарантирует видимость записи	✓	✓
Цена	высокая (lock, возможен BLOCKED)	низкая (барьер, без блокировки потока)

Отдельный случай: запись и чтение `volatile long / double` атомарны всегда. Обычные `long / double` JMM разрешает читать и писать «половинками» (word tearing).

Как это работает: барьеры, а не «сброс кэша»

Кэши ядер уже согласованы аппаратно (MESI), так что фраза «`volatile` перечитывает кэш» неточна. Проблема без `volatile` в другом: нет гарантии, **когда** значение станет видно, и JIT вправе держать его в регистре.

`volatile` решает это барьерами памяти:

- запись + **store barrier** — значение не застревает в регистре или store buffer ядра;
- чтение + **load barrier** — нельзя использовать значение из регистра, читаем заново;
- оба барьера запрещают переставлять инструкции вокруг (reordering).

Барьеры — деталь реализации; формально гарантии описываются через happens-before.

`volatile` защищает ссылку, не объект

```
class Config { int retries; String host; }

private volatile Config config = new Config();

config = new Config();    // ✓ новую ссылку увидят все потоки
config.retries = 3;       // ✗ запись в поле объекта – не volatile-запись,
                          // видимость не гарантирована
```

Паттерн **safe publication**: собрать объект целиком (лучше неизменяемый), потом присвоить `volatile`-ссылке. Дальше объект не менять, а при изменении подменять

ссылку на новый. Если объект приходится менять после публикации — защищать его поля отдельно (`synchronized` , атомики).

Когда `volatile` подходит

- флаг состояния (`running` , `ready`), который пишет один поток, а читают другие;
- публикация ссылки на неизменяемый снимок (конфиг, кэш);
- ленивая инициализация в double-checked locking: поле-синглтон обязано быть `volatile` .

Как только над значением нужно сделать больше одной операции атомарно — `volatile` недостаточен.

`ThreadLocal` — изоляция вместо синхронизации

`ThreadLocal` позволяет хранить данные, отдельные для каждого потока. Если несколько потоков используют один объект `ThreadLocal` , каждый получает своё собственное значение.

`ThreadLocal` — это как «карман» внутри каждого потока: можно положить данные, и другие потоки их не увидят.

```
private static final ThreadLocal<String> CONTEXT = new ThreadLocal<>();

CONTEXT.set("data");
String value = CONTEXT.get();

CONTEXT.remove();
```

Изменение значения в одном потоке не влияет на значения в других потоках. Поэтому для таких данных не требуется дополнительная синхронизация.

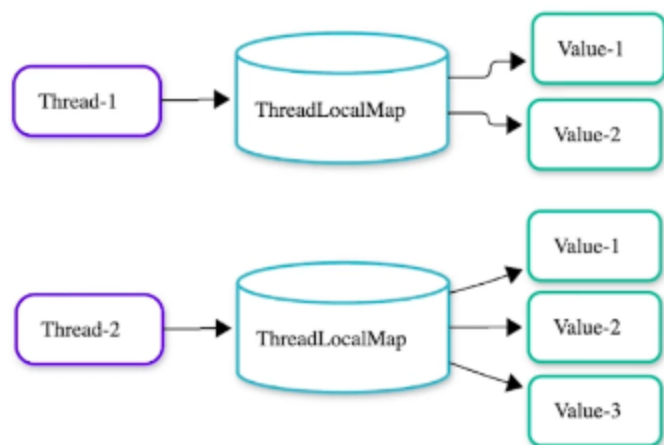
Где используется в реальной жизни

В Spring (например, `SecurityContextHolder`) хранит контекст аутентификации через `ThreadLocal` : каждый HTTP-запрос обрабатывается **своим потоком** (потоки переиспользуются пулом), поэтому каждому запросу можно привязать «свои» данные — текущего юзера, трейс-id, транзакцию — без передачи их через все методы в параметрах

Как работает внутри

Значения фактически хранятся не в самом `ThreadLocal` , а в специальной структуре `ThreadLocalMap` , которая принадлежит каждому потоку.

Упрощённо:



При вызове `set()` или `get()` `ThreadLocal` обращается к текущему потоку и работает с его `ThreadLocalMap`.

Ключи в этой структуре хранятся через `WeakReference`, а значения — по сильной ссылке. Поэтому при использовании `ThreadLocal` особенно важно правильно очищать данные.

Главная проблема — Thread Pool

Потоки в `ThreadPoolExecutor`, `Tomcat` и других пулах обычно не уничтожаются после выполнения задачи. Они используются повторно.

Если записать данные:

```
CONTEXT.set(data);
```

и не удалить их, следующий запрос может получить старое значение через тот же поток. Кроме того, значение может продолжать занимать память, пока живёт поток.

Поэтому после завершения работы `ThreadLocal` необходимо очищать.

Правильный шаблон

Обычно используется `try-finally`:

```
try {  
    CONTEXT.set(data);  
  
    // работа с контекстом  
} finally {
```

```
CONTEXT.remove();  
}
```

Явные блокировки и синхронизаторы java.util.concurrent

ReentrantLock vs synchronized, ReadWriteLock, StampedLock

Пакет: `java.util.concurrent.*locks*`. Явные блокировки: захват и освобождение вручную, взамен — гибкость, которой нет у `synchronized`.

ReentrantLock — ЯВНЫЙ ЭКСКЛЮЗИВНЫЙ ЛОК

```
private final ReentrantLock lock = new ReentrantLock();  
  
void update() {  
    lock.lock();           // сразу перед try  
    try {  
        // критическая секция  
    } finally {  
        lock.unlock();     // ТОЛЬКО в finally, иначе при исключении лок  
                           // останется занятым навсегда  
    }  
}
```

- **реентерабелен**, как `synchronized`: владелец может захватить лок повторно, лок ведёт счётчик захватов (`getHoldCount()`), и `unlock()` нужен на каждый `lock()`;
- `new ReentrantLock(true)` — **fair-режим**: при конкуренции лок достаётся дольше всех ждущему потоку. Это не гарантия честности планировщика, и пропускная способность обычно ниже, чем у нечестного;
- `tryLock()` без параметров **игнорирует fair-режим**: заберёт свободный лок, даже если кто-то уже ждёт.

Три способа захвата

Метод	Поведение
<code>lock()</code>	обычный захват; поток блокируется, пока лок занят; не прерывается <code>interrupt()</code>
<code>tryLock(timeout, unit)</code>	ждёт не дольше таймута → <code>true</code> / <code>false</code> ; без аргументов — одна попытка без ожидания

Метод	Поведение
<code>lockInterruptibly()</code>	захват, который можно прервать через <code>interrupt()</code> → <code>InterruptedException</code>

```

if (lock.tryLock(50, TimeUnit.MILLISECONDS)) {
    try {
        // работаем
    } finally {
        lock.unlock();
    }
} else {
    // лок не взяли за 50 мс — запасной сценарий
}

```

- `lockInterruptibly()` решает проблему из блока про `synchronized`: ожидание монитора прервать нельзя, а ожидание лока можно;
- `tryLock(timeout)` избавляет от вечного ожидания и помогает разорвать deadlock: не взяли второй лок за таймаут → **отпускаем уже взятые** и пробуем заново (без этого deadlock просто переезжает).

ReentrantLock vs synchronized

Критерий	<code>synchronized</code>	<code>ReentrantLock</code>
Освобождение	автоматически, при выходе из блока	вручную, <code>unlock()</code> в <code>finally</code>
Прерываемое ожидание	✗	<code>lockInterruptibly()</code>
Таймаут	✗	<code>tryLock(timeout)</code>
Справедливость	✗	<code>new ReentrantLock(true)</code>
Условия	<code>wait/notify</code> (одна очередь на монитор)	<code>Condition</code> (несколько)
Простота	очень просто	сложнее, легко забыть <code>unlock()</code>

По умолчанию — `synchronized`. `ReentrantLock` берут, когда нужны таймаут, прерываемое ожидание, `fair`-режим или несколько условий.

Под любым локом не делают I/O, сетевые вызовы и долгие операции: все остальные потоки встанут в очередь. Критическая секция должна быть короткой.

ReadWriteLock — читатели и писатель

Два режима одного лока:

- **read lock** — разделяемый: сколько угодно читателей одновременно;
- **write lock** — эксклюзивный: один писатель, читателей в этот момент нет.
- В любой момент лок либо у **сколько угодно читателей**, либо у **одного писателя**; читатели и писатель одновременно не бывают. (исключение: поток, держащий write lock, может сам взять ещё и read lock — downgrade, см. ниже).

```
private final ReadWriteLock rw = new ReentrantReadWriteLock();
private final Map<String, String> cache = new HashMap<>();

String get(String k) {
    rw.readLock().lock();
    try { return cache.get(k); }
    finally { rw.readLock().unlock(); }
}

void put(String k, String v) {
    rw.writeLock().lock();
    try { cache.put(k, v); }
    finally { rw.writeLock().unlock(); }
}
```

Когда стоит брать: данных много, **читателей заметно больше, чем писателей**, и операция под локом дороже самой синхронизации. Иначе обычный `ReentrantLock` или `synchronized` проще и не медленнее.

Очередь: писатель ждёт, пока текущие читатели закончат. Пока он ждёт, новые читатели в fair-режиме встанут **за** ним, а не обгоняют (в нечестном режиме порядок спецификацией не определён).

Ограничения `ReentrantReadWriteLock`:

- **upgrade невозможен**: держа read lock, захватить write lock нельзя — поток будет ждать сам себя вечно (deadlock);
- **downgrade можно**: взять write lock, затем read lock, затем отпустить write lock;
- **starvation**: в нечестном режиме (по умолчанию) порядок захвата не определён, и при постоянной конкуренции читатели или писатели могут ждать неограниченно долго; в fair-режиме порядок примерно по очереди прихода;
- у read lock нет `Condition` (`newCondition()` бросает `UnsupportedOperationException`);

- два лока — два `unlock()` , оба строго в `finally` .

StampedLock — ОПТИМИСТИЧНОЕ ЧТЕНИЕ

Идея: читатель **не берёт лок вообще**, а читает и потом проверяет, не вмешался ли писатель. При частых чтениях это снимает конкуренцию за read lock.

Режим	Вызов	Освобождение
запись	<code>writeLock()</code> → stamp	<code>unlockWrite(stamp)</code>
чтение	<code>readLock()</code> → stamp	<code>unlockRead(stamp)</code>
оптимистичное чтение	<code>tryOptimisticRead()</code> → stamp	не требуется; проверка <code>validate(stamp)</code>

```
private double x, y;
private final StampedLock sl = new StampedLock();

void move(double dx, double dy) {
    long stamp = sl.writeLock();
    try { x += dx; y += dy; }
    finally { sl.unlockWrite(stamp); }
}

double distance() {
    long stamp = sl.tryOptimisticRead(); // лок не берём
    double cx = x, cy = y;             // читаем в ЛОКАЛЬНЫЕ ПЕРЕМЕННЫЕ
    if (!sl.validate(stamp)) {          // писатель вмешался → читаем под
        // настоящим read lock
        stamp = sl.readLock();
        try { cx = x; cy = y; }
        finally { sl.unlockRead(stamp); }
    }
    return Math.hypot(cx, cy);
}
```

Правила:

- между `tryOptimisticRead()` и `validate()` значения могут быть **несогласованными**: читать только в локальные переменные и использовать их **после** успешной проверки;
- **не реентерабелен**: повторный захват в том же потоке приводит к блокировке;

- нет `Condition`; сам класс не реализует `Lock` и `ReadWriteLock` (есть обёртки `asReadLock()`, `asWriteLock()`, но и у них нет `Condition`);
- штамп нужно передавать при освобождении, а `0` из `try`-методов означает «не получилось».

Что выбрать

Инструмент	Когда
<code>synchronized</code>	по умолчанию: простая взаимная блокировка
<code>ReentrantLock</code>	нужны <code>tryLock</code> , прерываемое ожидание, <code>fair</code> , несколько <code>Condition</code>
<code>ReentrantReadWriteLock</code>	много чтений, мало записей, дорогие операции под локом
<code>StampedLock</code>	очень много коротких чтений, готовы жить без реентерабельности

`Condition` — `wait / notify` для `Lock`

Пакет: `java.util.concurrent.locks`. `Condition` — то же ожидание условия, что `wait() / notify()`, но для явного `Lock`. Создаётся из лока:

```
Lock lock = new ReentrantLock();
Condition cond = lock.newCondition();
```

Реальная польза — когда ждут разные условия и нужно будить только нужную группу потоков.

Соответствие `wait / notify`

Монитор (<code>synchronized</code>)	<code>Lock + Condition</code>
<code>lock.wait()</code>	<code>cond.await()</code>
<code>lock.notify()</code>	<code>cond.signal()</code>
<code>lock.notifyAll()</code>	<code>cond.signalAll()</code>
<code>wait(ms)</code>	<code>await(time, unit) → boolean</code> , <code>awaitNanos(n)</code>
одна очередь ожидания на монитор	несколько <code>Condition</code> на одном локе

`await()` атомарно отпускает лок и усыпляет поток; перед возвратом поток **снова захватывает** лок, как и у `wait()`.

Правила те же, что у wait / notify

```
lock.lock(); // 1. вызывать только при удерживаемом локе
try {
    while (!condition) { // 2. условие в while, не if
        cond.await();
    }
    // работаем с состоянием
    cond.signalAll(); // 3. состояние поменяли под тем же локом, потом сигналим
} finally {
    lock.unlock(); // 4. unlock() только в finally
}
```

- **Только при владении локом.** Иначе у `ReentrantLock` будет `IllegalMonitorStateException`;
- **while, не if.** Spurious wakeup допускается и для `Condition`: документация прямо требует ждать в цикле с проверкой условия;
- **signal() лок не отпускает:** сигналирующий поток держит лок до `unlock()`, и только потом разбуженный сможет продолжить;
- `await()` бросает `InterruptedException` — обработка как везде (восстановить флаг).

Зачем несколько условий

С `wait / notify` у монитора одна очередь: все ждущие стоят вместе, и приходится вызывать `notifyAll()`. С `Condition` очередей несколько, и сигнал идёт только тем, кто ждёт именно это событие.

Минимальный пример: в офисе одни потоки ждут документы, другие деньги.

```
class Office {
    private final Lock lock = new ReentrantLock();
    private final Condition docsReady = lock.newCondition(); // очередь ждущих документы
    private final Condition moneyReady = lock.newCondition(); // очередь ждущих деньги
    private boolean hasDocs, hasMoney;

    void bringDocs() {
        lock.lock();
        try {
            hasDocs = true;
        }
    }
}
```

```

docsReady.signalAll(); // будим только ждущих
документы
    } finally { lock.unlock(); }
}

void takeDocs() throws InterruptedException {
    lock.lock();
    try {
        while (!hasDocs) docsReady.await();
        hasDocs = false;
    } finally { lock.unlock(); }
}
// bringMoney() / takeMoney() – то же самое с moneyReady и hasMoney
}

```

Практический пример: ограниченный буфер. Производители ждут места (notFull), потребители ждут данных (notEmpty):

```

class BoundedBuffer<E> {
    private final Lock lock = new ReentrantLock();
    private final Condition notFull = lock.newCondition(); // ждут
производители
    private final Condition notEmpty = lock.newCondition(); // ждут
потребители

    private final Object[] items = new Object[100];
    private int putPtr, takePtr, count;

    public void put(E x) throws InterruptedException {
        lock.lock();
        try {
            while (count == items.length) notFull.await(); // нет места
            items[putPtr] = x;
            if (++putPtr == items.length) putPtr = 0;
            count++;
            notEmpty.signal(); // появился
элемент → будим потребителя
        } finally {
            lock.unlock();
        }
    }

    @SuppressWarnings("unchecked")
    public E take() throws InterruptedException {
        lock.lock();

```

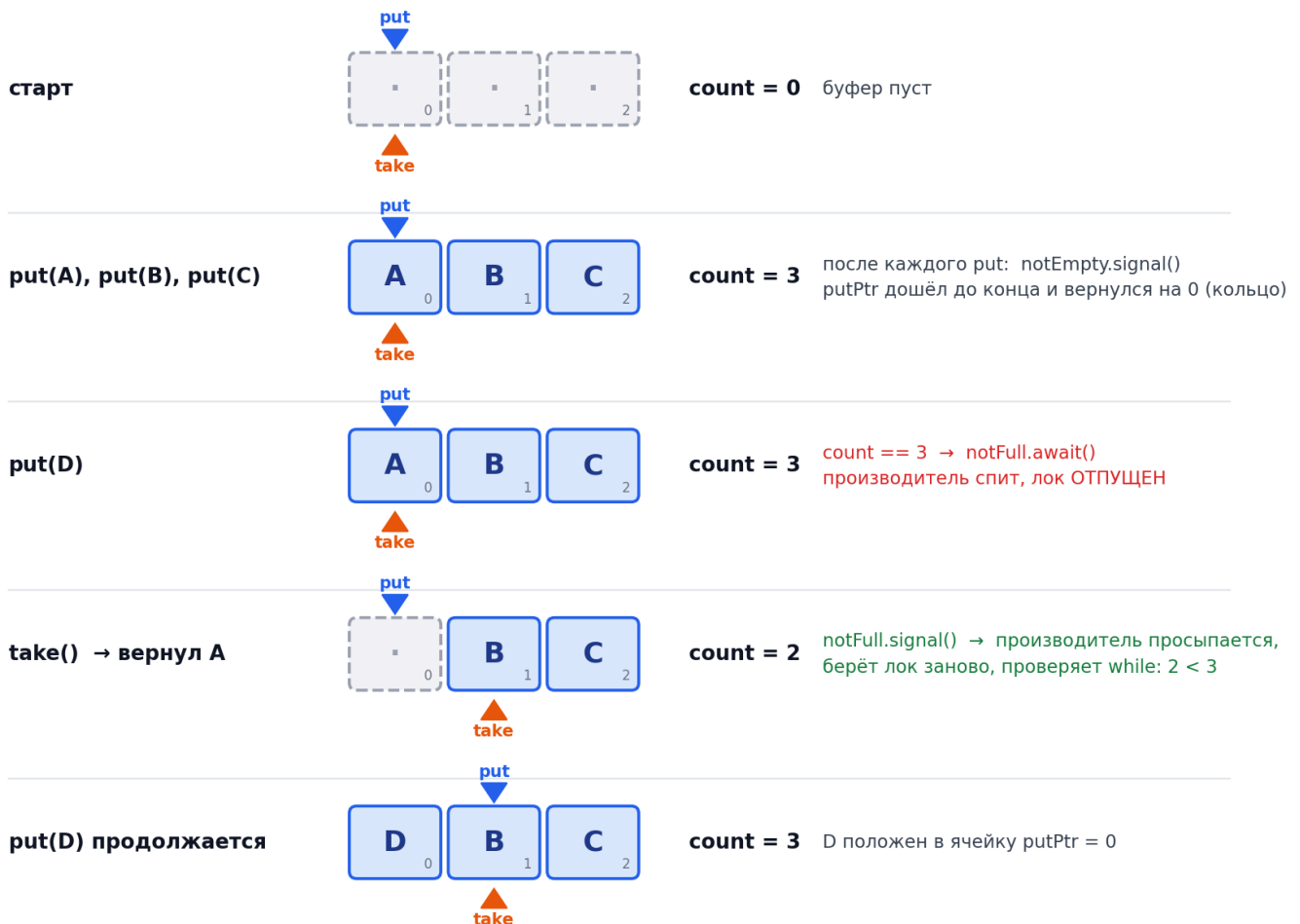
```

try {
    while (count == 0) notEmpty.await();           // нет данных
    E x = (E) items[takePtr];
    if (++takePtr == items.length) takePtr = 0;
    count--;
    notFull.signal();                             //
освободилось место → будим производителя
    return x;
} finally {
    lock.unlock();
}
}

```

Кольцевой буфер BoundedBuffer (3 места)

▼ put — putPtr (куда класть) ▲ take — takePtr (откуда брать)



Здесь `signal()` безопасен: все ждущие на одном `Condition` равнозначны, и сигнал подойдёт любому из них. С одной очередью `notify()` мог бы разбудить «не того» (производителя, когда нужен потребитель).

Ограничения

- `Condition` привязан к конкретному локу и работает только с ним;
- у `read lock` в `ReentrantReadWriteLock` `Condition` нет: `readLock().newCondition()` бросает `UnsupportedOperationException`. Работает только `writeLock()`;
- у `StampedLock` и его обёрток `asReadLock()` / `asWriteLock()` `Condition` тоже не поддерживается;
- на практике вместо ручной реализации буфера берут готовый `BlockingQueue` (см. конкурентные коллекции): например, `ArrayBlockingQueue` устроен именно так, на одном локе и двух `Condition`.

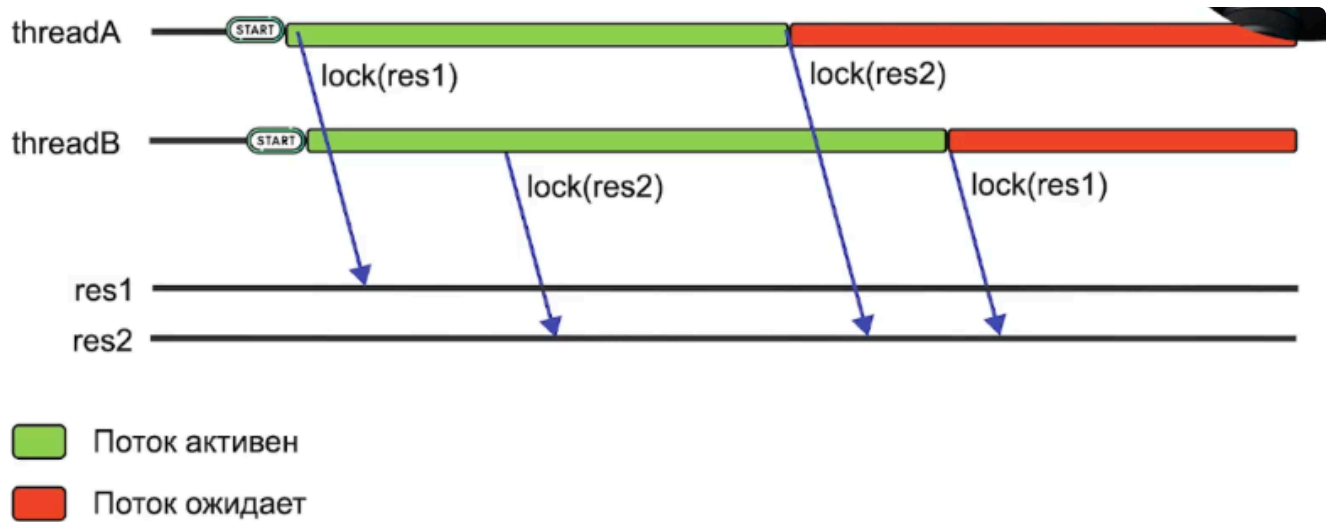
Deadlock / Livelock / Starvation — когда потоки не продвигаются

Три способа «застрять»

	Deadlock	Livelock	Starvation
Потоки	ждут, ничего не делают	активно работают, но уступают друг другу	готовы работать, но ресурс им не достаётся
Прогресс	нет, навсегда	нет	есть у других потоков, у этого нет
Причина	кольцо ожидания ресурсов	симметричное «вежливое» отступление	нечестная выдача лока, долгие захваты
Решение	единый порядок захвата, <code>tryLock</code>	случайная пауза (<code>backoff</code>)	<code>fair</code> -режим, короткие секции

Deadlock — взаимная блокировка

Определение: два или более потока **навсегда заблокированы**: каждый удерживает ресурс (лок) и ждёт ресурс, который держит другой из них. Ни один не может продолжить, потому что освободить ресурс может только тот, кто сам застрял. Потоки при этом в состоянии `BLOCKED` или `WAITING`, CPU они не тратят.



Пример: два потока переводят деньги между двумя счетами в противоположных направлениях.

```
static void transfer(Account from, Account to, int amount) {
    synchronized (from.lock) {           // t1: lock(1), t2: lock(2)
        synchronized (to.lock) {         // t1 ждет lock(2), t2 ждет lock(1)
            → вечное ожидание
            from.balance -= amount;
            to.balance += amount;
        }
    }
}

// t1: transfer(acc1, acc2, 100)
// t2: transfer(acc2, acc1, 100) ← обратный порядок
```

⚠ Deadlock проявляется не сразу: может понадобиться много итераций. Отсутствие зависания в тесте не значит, что его нет в проде.

Причина здесь: **разный порядок захвата локов в разных потоках.**

Условия Коффмана

Deadlock возможен, только если выполнены **все четыре** условия.

💡 Ключевая стратегия: **нарушить хотя бы одно условие** → дедлок невозможен.

#	Условие	Как нарушить
1	Взаимное исключение: ресурс занят одним потоком	неизменяемые объекты, atomics, lock-free (редко возможно)

#	Условие	Как нарушить
2	Удержание и ожидание: держит ресурс и просит ещё	брать всё сразу или <code>tryLock</code> с откатом
3	Нет принудительного освобождения: ресурс нельзя отобрать	<code>tryLock(timeout)</code> : не взяли за срок → отдаём то, что есть
4	Циклическое ожидание: цепочка потоков по кругу	единый порядок захвата

Решение 1: единый порядок захвата

Ломаем циклическое ожидание: все потоки берут локи в одном и том же порядке, например по `id`.

```
static void transfer(Account from, Account to, int amount) {
    Account first = from.id < to.id ? from : to;      // порядок один для
    BSEX потоков
    Account second = (first == from) ? to : from;

    synchronized (first.lock) {
        synchronized (second.lock) {
            from.balance -= amount;
            to.balance += amount;
        }
    }
}
```

Порядок задавай по уникальному ключу (`id`). Сравнение по `hashCode()` ненадёжно: у разных объектов он может совпасть, и порядок снова окажется неопределённым.

Решение 2: `tryLock(timeout)` с откатом

Нужен `ReentrantLock`. Не взяли второй лок за срок → **отпускаем первый** и пробуем заново.

```
static void transfer(Account from, Account to, int amount) throws
InterruptedException {
    while (true) {
        if (from.lock.tryLock(10, TimeUnit.MILLISECONDS)) {
            try {
                if (to.lock.tryLock(10, TimeUnit.MILLISECONDS)) {
                    try {
                        from.balance -= amount;
                    }
                }
            }
        }
    }
}
```

```

        to.balance += amount;
        return;
    } finally { to.lock.unlock(); }
    }
} finally { from.lock.unlock(); } // второй не взяли →
отдаём первый
    }
    Thread.sleep(ThreadLocalRandom.current().nextLong(5, 15)); //
случайная пауза против livelock
    }
}

```

Случайная пауза обязательна: без неё оба потока одновременно отпустят, одновременно повторят и снова столкнутся (это уже livelock).

Как найти deadlock в проде

Сначала нужен PID (номер процесса) нужной JVM. Его показывает `jps` — утилита из JDK, которая выводит все запущенные Java-процессы:

```

$ jps
4821 MyApp
5102 Jps

```

Дальше `jstack` — утилита из JDK, которая по PID печатает **thread dump**: все потоки JVM, их состояния (`RUNNABLE` , `BLOCKED` , `WAITING` ...) и стеки вызовов, то есть где в коде каждый поток сейчас стоит. Приложение при этом не останавливается:

```
jstack 4821
```

Если есть deadlock на `synchronized` или локах `java.util.concurrent` , в конце дампа будет раздел:

```

Found one Java-level deadlock:
"t1": waiting to lock monitor ... which is held by "t2"
"t2": waiting to lock monitor ... which is held by "t1"

```

Livelock — «вежливый» deadlock

Определение: потоки **не заблокированы** и активно работают (`RUNNABLE`): реагируют друг на друга и меняют своё состояние, но из-за симметричной реакции не продвигаются к цели. Состояние меняется, прогресса нет, CPU при этом занят впустую. Типичная

причина — симметричный протокол уступки: оба одновременно отступают, одновременно повторяют и снова сталкиваются.



Пример: у супругов одна ложка, оба голодны, и каждый уступает её другому.

```
// у супругов одна ложка, оба голодны
void eatWith(Spoon spoon, Diner spouse) {
    while (isHungry) {
        if (spoon.getOwner() != this) { sleep(10); continue; } // ложка не
у меня — жду
        if (spouse.isHungry) {                               // супруг голоден → уступаю
            spoon.setOwner(spouse);
            continue;                                         // уступают оба, бесконечно, CPU
        }
        spoon.use(); isHungry = false;
    }
}
```

Лечение — нарушить симметрию: **случайный backoff** (иногда уступаю, иногда ем; пауза случайной длины перед повтором).

Поле `isHungry` здесь читает другой поток (`spouse.isHungry`), поэтому оно должно быть `volatile` , иначе видимость не гарантирована.

Starvation — «голодание» потока

Определение: поток готов работать, но **постоянно не получает нужный ресурс** (лок, процессорное время), потому что другие потоки раз за разом получают его раньше. Система в целом работает и прогресс у других потоков есть, а у этого его нет. От deadlock отличается тем, что ресурс не заблокирован навсегда: поток теоретически может его получить, но фактически не получает.

Типичные причины:

- нечестный лок: порядок выдачи не определён, один поток может постоянно «обгоняться»;
- `ReentrantReadWriteLock` в нечестном режиме: читатели или писатели могут ждать неограниченно;
- длинные операции под локом: остальные накапливаются в очереди.

Лечение: fair-режим (`new ReentrantLock(true)` , `new Semaphore(n, true)` , `fair ReentrantReadWriteLock`), короткие критические секции. Цена fair-режима: пропускная способность ниже.

CountDownLatch , CyclicBarrier , Semaphore — синхронизаторы и общая основа AQS

Пакет: `java.util.concurrent`

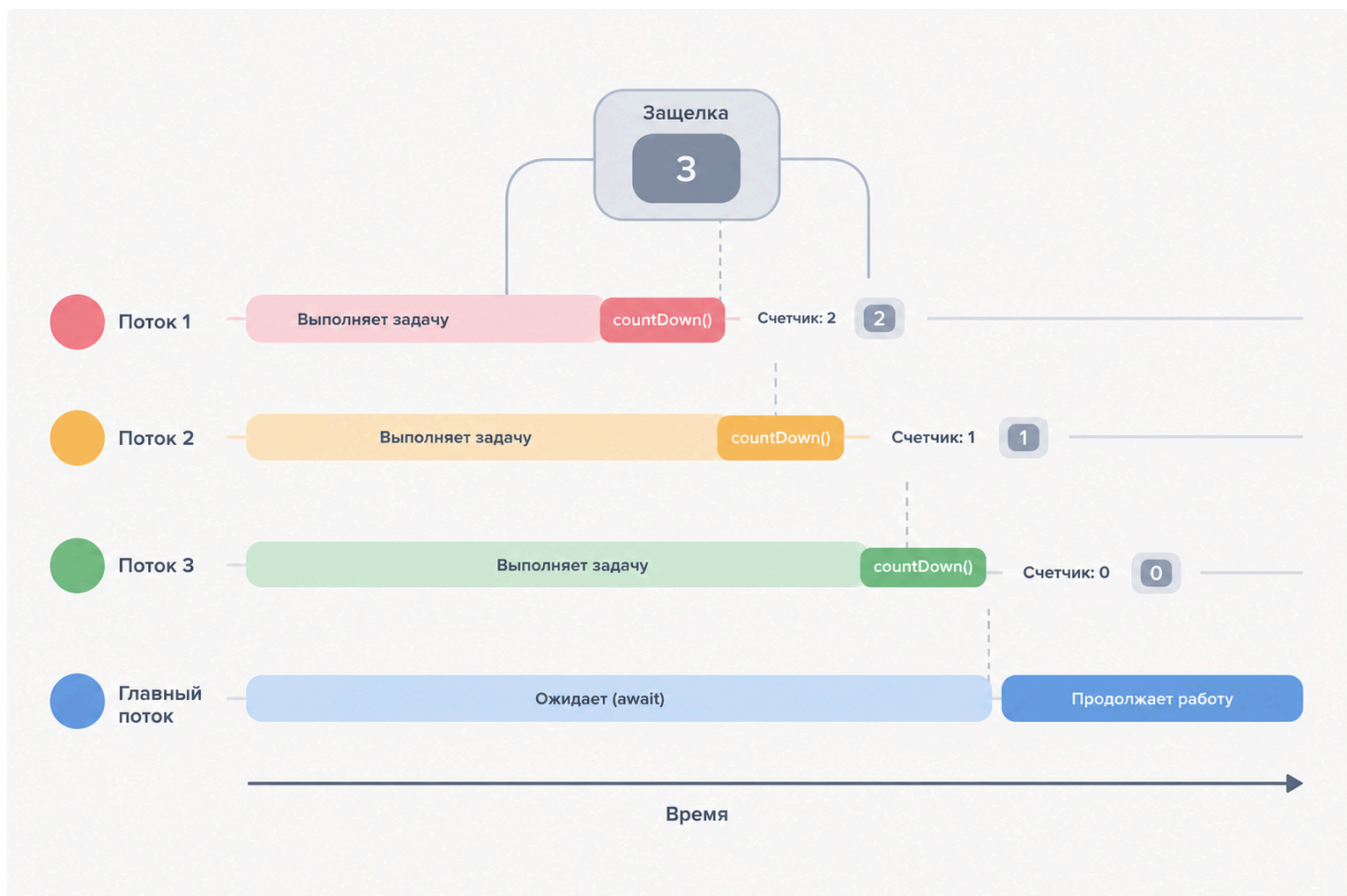
Лок защищает критическую секцию: внутри один поток. **Синхронизатор** координирует потоки между собой: ждать события, ждать друг друга, ограничивать число одновременных. Их отличие от лока — **нет владельца**: «открыть» или «вернуть» может не тот поток, который ждал.

	Что считает	Кто ждет	Сбрасывается
<code>CountDownLatch</code>	событий (счётчик к нулю)	один или несколько потоков, пока другие отчитаются	нет, одноразовый
<code>CyclicBarrier</code>	пришедших потоков	все участники друг друга	да, многоразовый

	Что считает	Кто ждет	Сбрасывается
Semaphore	свободных разрешений (permits)	потоки сверх лимита	не нужно: acquire / release

CountDownLatch — защёлка-счётчик

Определение: счётчик, который уменьшается до нуля. Потоки в `await()` блокируются, пока он не станет 0; после этого все ждущие отпускаются, а любой новый `await()` возвращается сразу. **Одноразовый:** счётчик сбросить нельзя.



Два типичных использования:

- **ворота (count = 1):** все ждут в `await()`, один поток открывает через `countDown()`;
- **дождаться N задач (count = N):** главный поток ждёт, пока N воркеров отчитаются.

```
CountDownLatch latch = new CountDownLatch(3);           // ждём 3 события
int[] results = new int[3];

for (int i = 0; i < 3; i++) {
```

```

int id = i;
new Thread(() -> {
    try {
        results[id] = calc(id);           // 1) пишем результат
    } finally {
        latch.countDown();               // 2) отчитываемся
        ВСЕГДА, даже при исключении
    }
}).start();
}

latch.await();                           // 3) ждём, пока
счётчик станет 0
// results видны без volatile

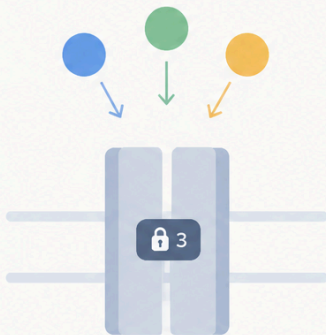
```

- **happens-before бесплатно:** всё, что поток сделал до `countDown()`, видно после возврата из `await()`. Поэтому `results` не обязан быть `volatile`;
- `countDown()` держи в `finally`: если воркер упал и не отчитался, `await()` без таймаута повиснет навсегда;
- `await(timeout, unit)` возвращает `false`, если время вышло раньше нуля.

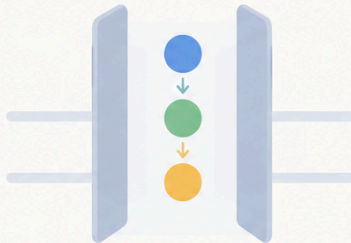
CyclicBarrier — барьер для группы потоков

Определение: точка встречи фиксированной группы из N потоков: каждый, дойдя до `await()`, ждёт, пока соберутся все N. Когда пришёл последний, все отпускаются одновременно. **Многоразовый** (cyclic): после прохода барьер готов к следующему кругу.

1 Потоки приходят к барьеру



2 Все потоки проходят через барьер



3 Барьер снова закрывается и ждет следующую фазу



```
CyclicBarrier barrier = new CyclicBarrier(3, () -> System.out.println("фаза завершена"));
```

// второй аргумент – barrier action: выполняется один раз за круг, последним пришедшим потоком, до отпуски остальных

```
Runnable worker = () -> {  
    try {  
        for (int phase = 0; phase < 2; phase++) {  
            doPhase(phase);  
            barrier.await(); // ждём остальных; после прохода барьер  
            МОЖНО ИСПОЛЬЗОВАТЬ СНОВА  
        }  
    } catch (InterruptedException | BrokenBarrierException e) {  
        Thread.currentThread().interrupt();  
    }  
};
```

- **всё или ничего:** если один поток ушёл раньше (прерван, истёк таймаут, упал), остальные получают `BrokenBarrierException`;
- если потоков меньше `N`, все застрянут в ожидании навсегда;
- `await()` возвращает порядок прихода: `0` — пришёл последним;


```
}  
}
```

`acquire()` ставят **перед** `try`: если ожидание прервут, разрешение не взято, и `release()` в `finally` вернул бы лишнее: счётчик вырос бы выше исходного.

- **нет владельца**: разрешение может вернуть любой поток, не обязательно взявший его. Поэтому семафор с одним разрешением (binary) не равен мьютексу: нет реентерабельности и проверки владельца;
- **fair-режим**: `new Semaphore(n, true)` — FIFO. В нечестном допускается обгон (barging). Цена fair: пропускная способность ниже;
- `tryAcquire()` без ожидания **игнорирует fair-режим**: заберёт разрешение, если оно свободно, даже когда кто-то ждёт;
- `acquire(n) / release(n)` — несколько разрешений сразу;
- семафор не защищает инварианты данных: он лишь ограничивает число потоков;
- happens-before: действия до `release()` видны после успешного `acquire()`.

AQS — общая основа

AQS (AbstractQueuedSynchronizer) — базовый механизм пакета

`java.util.concurrent.locks`, предназначенный для реализации синхронизаторов на основе состояния (state) и очереди ожидающих потоков.

AQS реализует общую механику синхронизации, а конкретный синхронизатор определяет правила доступа к ресурсу через state

AQS предоставляет готовую инфраструктуру:

- очередь ожидающих потоков;
- блокировку и пробуждение потоков через `LockSupport.park/unpark`;
- обработку прерываний и таймаутов;
- атомарное изменение состояния через CAS.

Основное состояние AQS хранится в поле:

```
volatile int state;
```

Смысл state определяется конкретным синхронизатором.

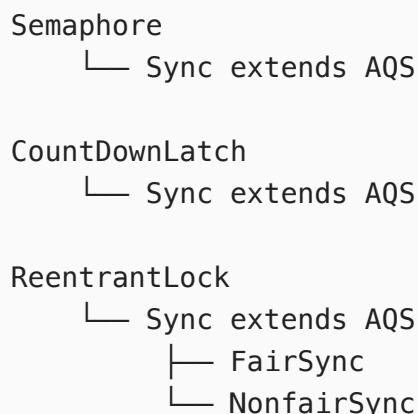
AQS не определяет самостоятельно, когда поток может получить доступ. Конкретный синхронизатор задаёт правила работы со `state` через методы `tryAcquire`, `tryRelease`, `tryAcquireShared`, `tryReleaseShared`.

Значение `state` в конкретных синхронизаторах:

- `Semaphore` — количество доступных разрешений (`permits`).
- `CountDownLatch` — количество оставшихся операций до достижения `0` .
- `ReentrantLock` — состояние блокировки и количество повторных захватов текущим владельцем.

Связь с AQS:

`Semaphore`, `CountDownLatch` и `ReentrantLock` не наследуются от AQS напрямую. Они содержат внутренний класс `Sync`, который наследуется от `AbstractQueuedSynchronizer`.



Lock-free: CAS и атомики

CAS и CAS-loop — как работает lock-free, проблема ABA

Что такое lock-free

Lock-free — подход, где потоки не берут лок и не блокируются друг на друге: общее состояние обновляется атомарной инструкцией процессора. Гарантия: какой-то из потоков всегда продвигается вперёд, поэтому deadlock невозможен, а нет блокировки — нет и переключения контекста на ожидании.

Основа — **CAS (compare-and-swap)**.

CAS — аппаратная атомарная операция

Одна неделимая инструкция процессора: «если по адресу лежит ожидаемое значение, записать новое».


```
compareAndSwap(адрес, expected, new):
    если *адрес == expected:
        *адрес = new           // всё одной неделимой инструкцией
        return true
    иначе:
        return false           // другой поток успел раньше, ничего не меняем
```

В Java это `compareAndSet(expected, new)` у атомарных классов. Он ещё и даёт гарантии видимости как `volatile`: такие же happens-before, что и у `volatile`-записи/чтения.

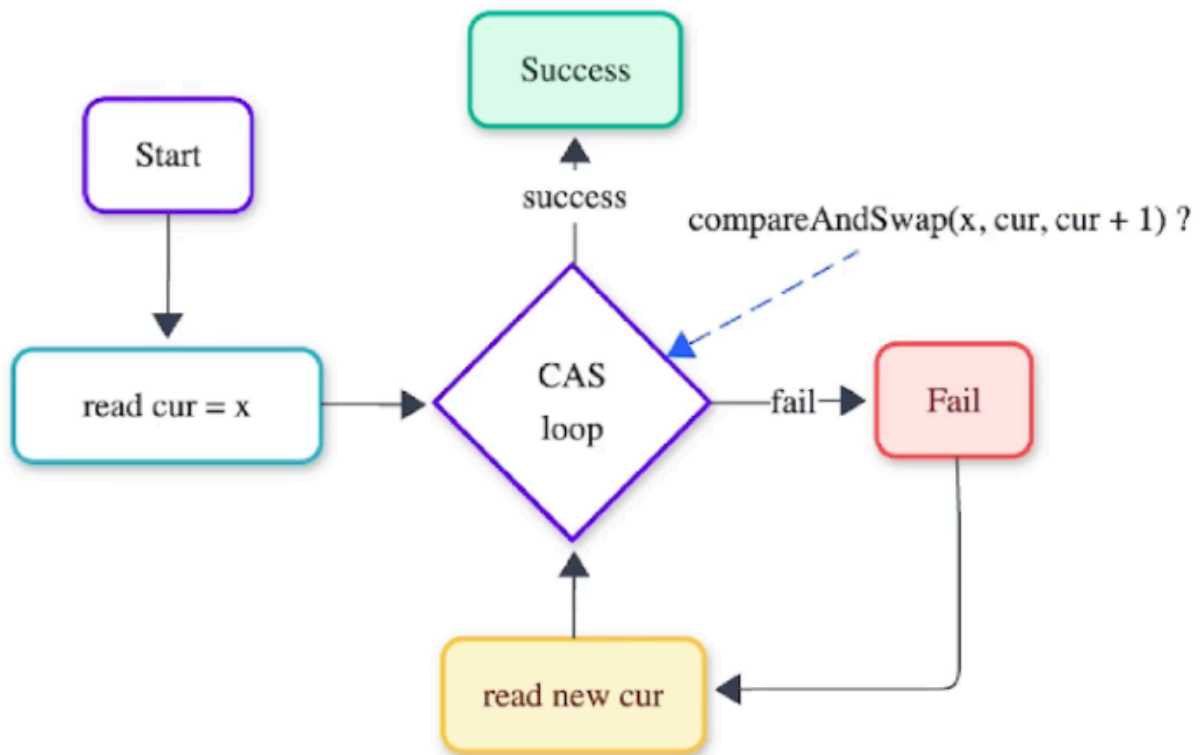
CAS-loop — повторять, пока не получится

Составная операция (`counter++`) строится как цикл: прочитал, вычислил, попытался записать; если кто-то успел раньше — повторить.

```
AtomicInteger value = new AtomicInteger();

int increment() {
    int old, next;
    do {
        old = value.get();           // 1. читаем текущее
        next = old + 1;              // 2. вычисляем новое
    } while (!value.compareAndSet(old, next)); // 3. пишем, только если
    никто не вмешался; иначе повтор
    return next;
}
```

Именно так внутри устроены `incrementAndGet()`, `updateAndGet()` и остальные операции атомиков.



Оптимистичная блокировка

Принцип CAS: не запираем ресурс заранее, а **пробуем обновить и проверяем, не изменилось ли оно за это время**. При конфликте — перечитываем и повторяем. Противоположность — пессимистичная блокировка (`synchronized` , `Lock`): сначала запираем, потом работаем.

Тот же принцип в базе данных:

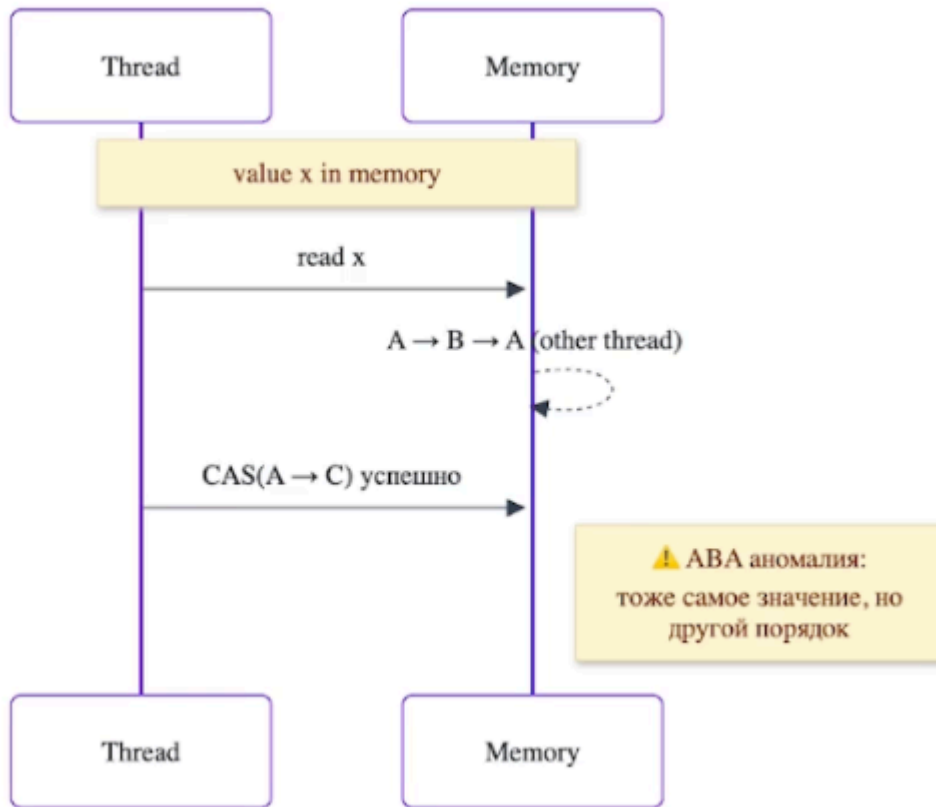
```
UPDATE accounts SET balance = 80, version = 2
WHERE id = 777 AND version = 1;    -- сработает, только если версия не
изменилась
-- Rows affected = 0 → кто-то успел раньше → перечитать и повторить
```

Цена CAS

- **Низкая и умеренная конкуренция:** CAS дешевле лока. Нет блокировки потока, очереди и переключения контекста;
- **Высокая конкуренция:** много неудачных попыток, потоки жгут CPU на повторях, пропускная способность падает. Deadlock при этом невозможен: в каждом раунде хотя бы один поток succeeds, но отдельному потоку может «не везти» долго;
- лекарство от конкуренции на одном счётчике — `LongAdder` (см. ниже).

Проблема АВА

Суть: поток читает значение **A** и собирается сделать $\text{CAS}(A \rightarrow C)$. Пока он ждёт, другой поток успевает сделать $A \rightarrow B \rightarrow A$. CAS видит «A», считает, что ничего не менялось, и успешно записывает C. Значение то же, но промежуточное состояние потеряно.



```
AtomicReference<String> ref = new AtomicReference<>("A");

// Поток 1: oldValue = ref.get() -> "A"; пауза; ref.compareAndSet(oldValue,
"C") -> true ❌
// Поток 2 за время паузы: compareAndSet("A", "B"); compareAndSet("B",
"A"); // A -> B -> A
```

В Java с GC это обычно проявляется, когда объект или узел **возвращают обратно в состояние** (пул, повторное использование, значения вроде кэшированных `Integer`): тот же объект по ссылке может скрыть, что структура вокруг него изменилась.

Решение: `AtomicStampedReference`. Вместе со ссылкой хранится **штамп** (номер версии). CAS проверяет пару «ссылка + штамп»:

```
AtomicStampedReference<String> ref = new AtomicStampedReference<>("A", 0);
// значение + штамп
```

```
int[] holder = new int[1];
String old = ref.get(holder);           // значение возвращается,
штамп кладётся в holder[0]

boolean ok = ref.compareAndSet(old, "C", holder[0], holder[0] + 1);
// успех, только если ссылка == old И штамп совпал; штамп при записи
увеличиваем
```

Даже если значение вернулось к «А», штамп уже другой, и CAS вернёт `false`: поток перечитает актуальное состояние.

Для простого флага «менялось или нет» есть облегчённый `AtomicMarkableReference` (вместо номера версии — булев признак).

Atomic-типы: `AtomicInteger`, `AtomicReference`, `LongAdder`

Пакет: `java.util.concurrent.atomic`. Классы для атомарных операций над **одной переменной** без блокировок, через CAS.

Какой класс когда

Класс	Когда брать
<code>AtomicInteger</code> , <code>AtomicLong</code>	счётчики, индексы, генераторы id
<code>AtomicBoolean</code>	флаги, «выполнить один раз»
<code>AtomicReference<T></code>	атомарная смена ссылки на объект
<code>AtomicStampedReference<T></code>	ссылка + версия (защита от ABA)
<code>LongAdder</code>	счётчик с очень высокой конкуренцией записи (метрики, статистика)

`AtomicInteger` : счётчик

```
static final AtomicInteger counter = new AtomicInteger(0);

Runnable r = () -> {
    for (int k = 0; k < 250_000; k++) counter.getAndIncrement();    //
    атомарно, без synchronized
};
// 4 потока → всегда ровно 1_000_000
```

Почему не обычный `int: counter++` — три шага (прочитать → увеличить → записать), и два потока затирают обновления друг друга. Почему не `volatile int: volatile` даёт видимость, но не атомарность составной операции. Атомик даёт и то и другое.

Основные методы:

Метод	Что делает
<code>get()</code> / <code>set(v)</code>	чтение / запись (с гарантиями <code>volatile</code>)
<code>incrementAndGet()</code>	+1, возвращает новое значение (<code>++i</code>)
<code>getAndIncrement()</code>	+1, возвращает старое значение (<code>i++</code>)
<code>addAndGet(d)</code> / <code>getAndAdd(d)</code>	то же с произвольной дельтой
<code>compareAndSet(expect, update)</code>	ядро CAS: меняет, только если текущее равно <code>expect</code>
<code>updateAndGet(f)</code> / <code>accumulateAndGet(x, f)</code>	применить функцию через CAS-loop

AtomicReference : атомарная смена ссылки

```
AtomicReference<String> name = new AtomicReference<>("Alice");

name.updateAndGet(n -> n.equals("Alice") ? "Bob" : "Alice"); // читает,
вычисляет, пишет через CAS-loop
```

⚠ Две ловушки:

- **функция в `updateAndGet` должна быть без побочных эффектов:** при конкуренции CAS не проходит, и она вызывается повторно. Никаких `println`, добавления в список и т. п. внутри;
- **`compareAndSet` сравнивает ссылки через `==`, а не `equals()`:** для новых объектов (`new`, `Integer` вне кэша) CAS с «таким же» значением вернёт `false`.

AtomicBoolean : флаг и «один раз»

```
AtomicBoolean started = new AtomicBoolean(false);

if (started.compareAndSet(false, true)) {
    start(); // выполнится ровно в одном потоке; остальные получают
false и пропустят
}
```

LongAdder : счётчик под высокой конкуренцией

Идея: AtomicLong — одна ячейка, за которую при большом числе потоков идёт борьба (много неудачных CAS). LongAdder при конкуренции **разбивает счётчик на несколько ячеек**, потоки пишут в разные, а при чтении sum() складывает их.

```
LongAdder hits = new LongAdder();

hits.increment();           // из многих потоков
long total = hits.sum();    // читаем итог
```

- при высокой конкуренции пропускная способность **заметно выше**, цена — больше памяти (по документации);
- sum() — **не атомарный снимок**: обновления, идущие во время подсчёта, могут не попасть в результат. Подходит для метрик и статистики, не для точной синхронизации;
- нет getAndIncrement() : значение, которое вернуло бы «старое», здесь не определено. Если нужен id или индекс, берут AtomicLong ;
- при низкой конкуренции разницы с AtomicLong почти нет.

Ограничения атомиков

- **одна переменная.** Инвариант между несколькими полями (например, min и max) двумя атомиками не сохранить: между двумя операциями другой поток увидит несогласованное состояние. Выход: один AtomicReference на **неизменяемый** объект, содержащий оба значения, либо лок;
- атомики не имеют equals / hashCode / compareTo и плохо подходят как ключи коллекций;
- в CAS-loop при сильной конкуренции много повторов, а вместе с ними бесполезная работа CPU.

Что выбрать для счётчика

Ситуация	Инструмент
простой счётчик, умеренная конкуренция	AtomicInteger / AtomicLong
счётчик метрик, много потоков, чтение редко	LongAdder
нужно «старое значение» (id, индекс)	AtomicInteger / AtomicLong

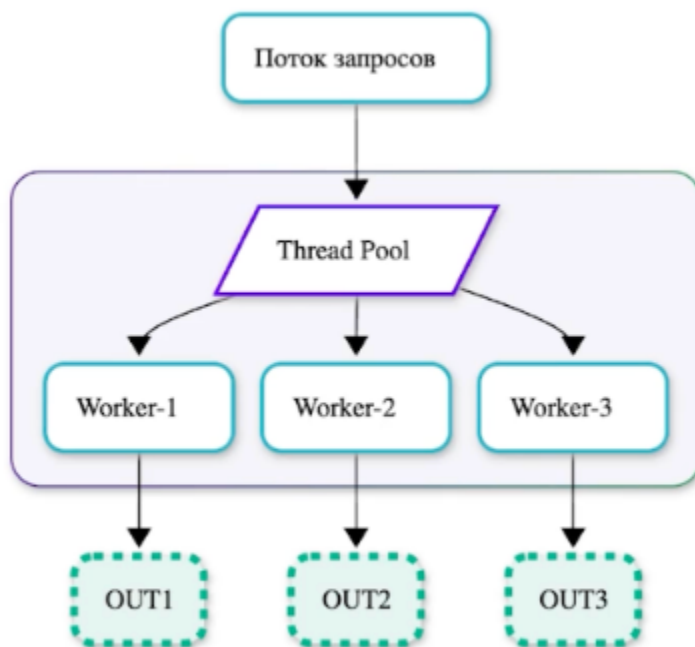
Ситуация	Инструмент
несколько связанных полей	<code>synchronized</code> / <code>Lock</code> или <code>AtomicReference</code> на неизменяемый объект

Пулы потоков

Пулы потоков и `ThreadPoolExecutor` в Java

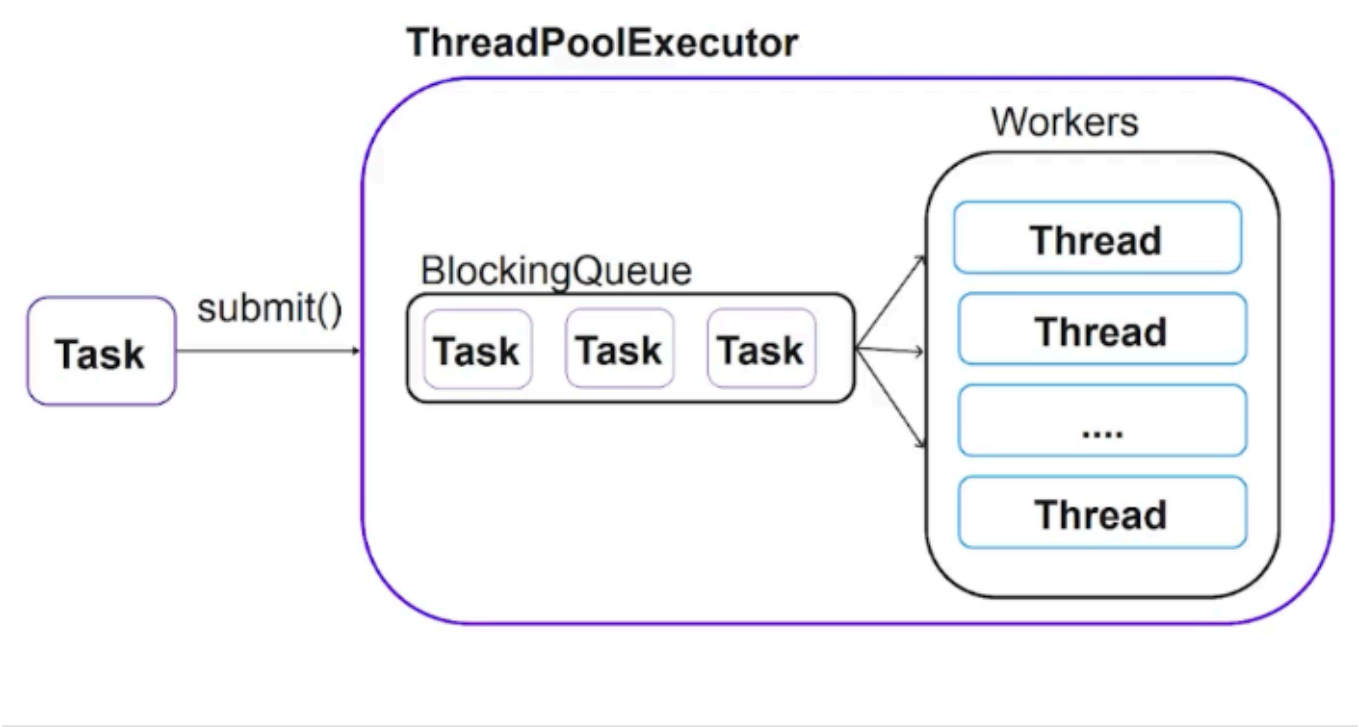
Пул потоков — это механизм управления группой заранее созданных рабочих потоков и очередью задач. Вместо создания нового потока на каждую операцию приложение отправляет задачи в очередь, откуда их по очереди забирают и исполняют свободные потоки. Пакет: `java.util.concurrent`.

Преимущества использования пула



- Снижение расходов на ресурсы: создание потока требует памяти под стек, системных вызовов и переключений контекста. Пул повторно использует уже существующие потоки.
- Ограничение параллелизма: лишние задачи ждут своей очереди, а не порождают тысячи неконтролируемых потоков и не исчерпывают память.
- Разделение ответственности: отделяет описание задачи (`Runnable` / `Callable`) от логики её выполнения (сколько потоков, какая очередь, что делать при перегрузке).

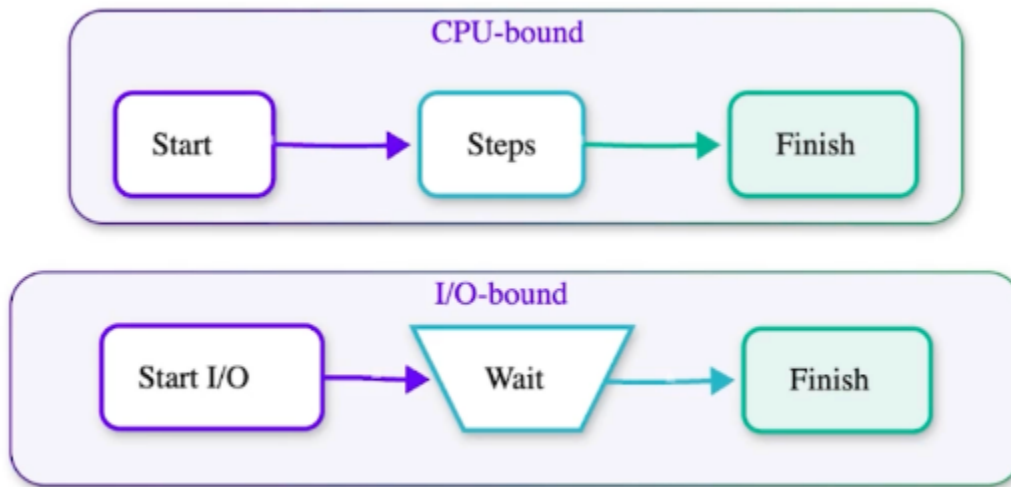
Пул реализует классический паттерн **Producer–Consumer**: клиенты ставят задачи в `BlockingQueue` , а воркеры (`Thread`) в бесконечном цикле берут их методом `take()` . Если задач нет, воркер блокируется и не расходует CPU.



Расчёт размера пула (CPU-bound vs IO-bound)

Количество потоков зависит от характера задач:

Тип задач	На что уходит время	Оптимальный размер пула
CPU-bound	вычисления, обработка данных	≈ число ядер (избыток потоков лишь добавит переключений контекста)
IO-bound	ожидание сети, СУБД, диска	заметно больше числа ядер: потоки большую часть времени заблокированы на вводе-выводе



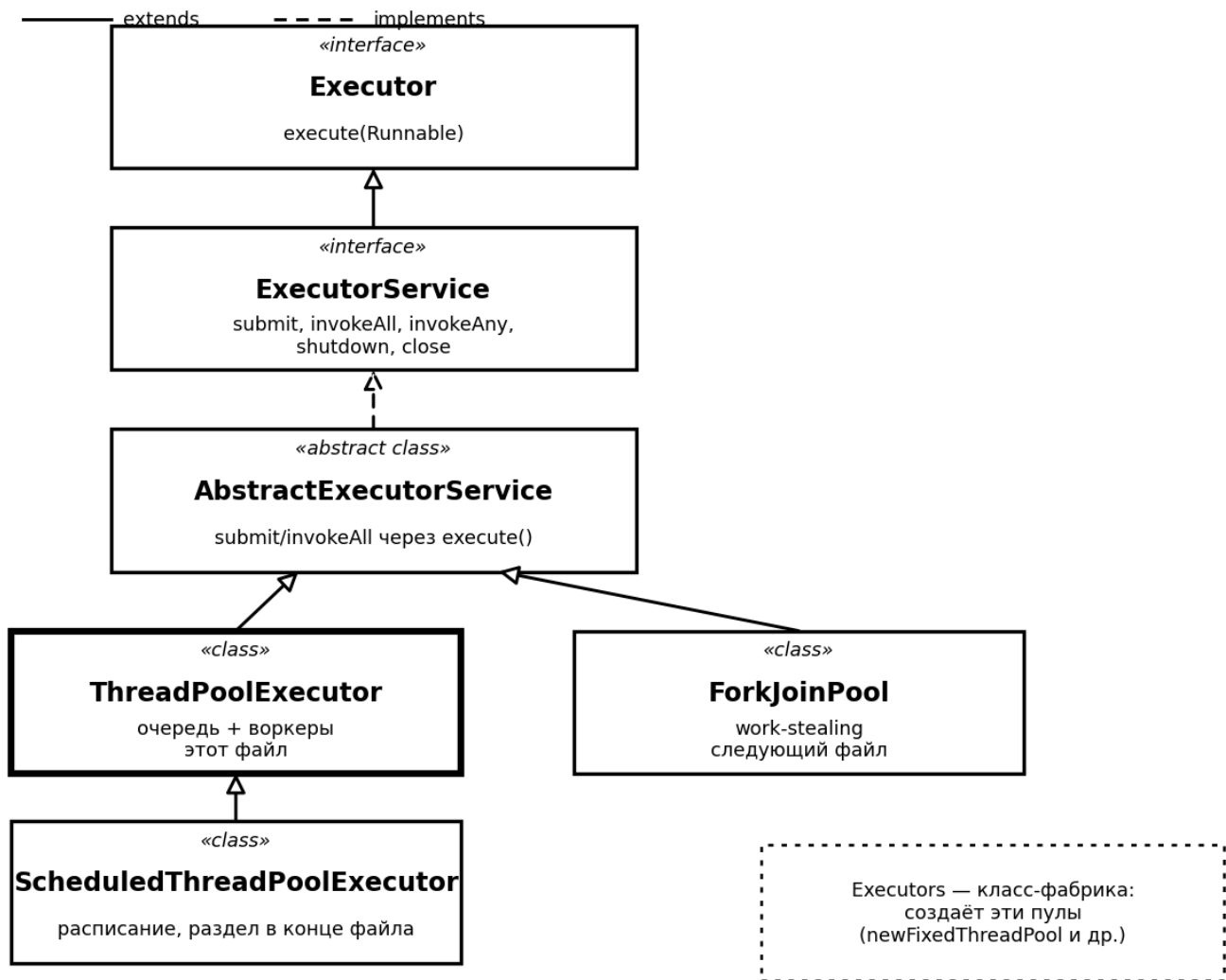
Формула для смешанной нагрузки:

Размер пула= $N_{cpu} \times (1 + \frac{\text{время ожидания}}{\text{время вычислений}})$

Примечание: формула служит отправной точкой. Итоговый размер подбирается профилированием и нагрузочным тестированием.

Иерархия классов и интерфейсов

`ThreadPoolExecutor` — обычный класс (не интерфейс), основная реализация пула потоков в JDK. Остальные имена вокруг него — интерфейсы, абстрактный класс и фабрика. На схеме жирной рамкой выделен класс, который разбирается в этом файле; `ScheduledThreadPoolExecutor` (он наследует `ThreadPoolExecutor` и дополнительно реализует `ScheduledExecutorService`) разобран в конце файла, `ForkJoinPool` — в следующем:



Для быстрого создания пулов используется утилитный класс-фабрика `Executors` : он просто создаёт `new ThreadPoolExecutor(...)` с готовыми параметрами.

Соответствие фабричных методов и реальных классов

Фабричный метод	Реальный класс	Особенности
<code>newFixedThreadPool(n)</code>	<code>ThreadPoolExecutor</code>	фиксированное количество потоков, неограниченная очередь (<code>LinkedBlockingQueue</code>)
<code>newCachedThreadPool()</code>	<code>ThreadPoolExecutor</code>	<code>core = 0</code> , <code>max = Integer.MAX_VALUE</code> простаивающие потоки умирают через <code>keepAliveTime</code> очередь-передаточная <code>SynchronousQueue</code>


```

потоков в пуле
    60, TimeUnit.SECONDS,           // keepAliveTime: время жизни
лишних (сверх core) простаивающих потоков
    new ArrayBlockingQueue<>(2),     // workQueue: ограниченная
очередь задач
    Executors.defaultThreadFactory(), // threadFactory: имена,
daemon-статус потоков
    new ThreadPoolExecutor.AbortPolicy() // handler: что делать при
переполнении
);

```

Алгоритм обработки входящей задачи

1. Если потоков меньше `corePoolSize`, создаётся новый поток, даже если в пуле есть простаивающие воркеры.
2. Если `corePoolSize` достигнут, задача помещается в `workQueue`.
3. Если очередь полностью заполнена, создаётся новый поток вплоть до `maximumPoolSize`.
4. Если потоков уже `maximumPoolSize` и очередь полна, срабатывает `RejectedExecutionHandler`.

Важное следствие: потоки сверх `corePoolSize` появляются только после полного заполнения очереди. Если очередь не ограничена (как в `FixedThreadPool`), пул никогда не создаст больше `corePoolSize` потоков, а `maximumPoolSize` игнорируется.

Пример распределения нагрузки

Конфигурация: `core = 2`, `max = 4`, очередь `ArrayBlockingQueue(2)`. Последовательно приходят 7 долгих задач (результат проверен запуском):

- Задачи 1–2: запускаются в двух потоках (`core`). Потоков: 2, в очереди: 0.
- Задачи 3–4: помещаются в очередь. Потоков: 2, в очереди: 2.
- Задачи 5–6: очередь полна, создаются дополнительные потоки до лимита `max`. Потоков: 4, в очереди: 2.
- Задача 7: очередь полна, лимит `max` исчерпан, бросается `RejectedExecutionException`.

Типы очередей задач

- `SynchronousQueue` (прямая передача): не хранит задачи. Либо её сразу забирает свободный поток, либо создаётся новый. Требуется большого или неограниченного

`maximumPoolSize` .

- `LinkedBlockingQueue` (без указания размера): неограниченная ёмкость, `maximumPoolSize` теряет смысл, высокий риск OOM при перегрузках.
- `ArrayBlockingQueue(n)` (ограниченная): жёстко лимитирует потребление ресурсов, но требует точной балансировки `core` , `max` и ёмкости `n` .

Стратегии отказа (`RejectedExecutionHandler`)

Срабатывает при достижении лимитов пула, а также при попытке отправить задачу в уже остановленный (`shutdown`) пул.

- `AbortPolicy` (по умолчанию): бросает `RejectedExecutionException` . Подходит, когда сбой нельзя игнорировать.
- `CallerRunsPolicy` : задача выполняется непосредственно в вызывающем потоке (том, что вызвал `execute`). Это замедляет отправителя и создаёт естественный back-pressure.
- `DiscardPolicy` : задача молча отбрасывается. Допустимо только для не критичных данных (например, метрик).
- `DiscardOldestPolicy` : удаляет самую старую задачу из головы очереди и пытается добавить новую заново.

Особенности эксплуатации пула

- Жизненный цикл потоков: потоки из группы `core` по умолчанию не умирают при простое. Чтобы они завершались по истечении `keepAliveTime` , вызывают `allowCoreThreadTimeOut(true)` . Сами `core`-потоки создаются лениво, при первых задачах (`prestartAllCoreThreads()` создаёт их заранее).
- Именованые потоки: стандартная фабрика даёт безликие имена вида `pool-1-thread-3` . Для читаемых логов и дампов потоков передают собственную `ThreadFactory` :

```
AtomicInteger count = new AtomicInteger();
ThreadFactory customFactory = r -> new Thread(r, "orders-worker-" +
count.incrementAndGet());
```

- Обработка исключений (`execute` против `submit`):

```
pool.execute(() -> { throw new RuntimeException("boom"); }); // стек
печатается в stderr, воркер умирает, пул создаёт замену
pool.submit (() -> { throw new RuntimeException("boom"); }); // тишина:
исключение лежит в Future до вызова get()
```

- `execute(Runnable)` : необработанное исключение уходит в `stderr`, рабочий поток завершается, пул создаёт ему замену.
- `submit(...)` : исключение сохраняется внутри `Future` и выбрасывается (обёрнутым в `ExecutionException`) только при `future.get()`. Если `get()` не вызвать, ошибка исчезнет из логов.

Остановка пула

Рабочие потоки пула по умолчанию пользовательские (не `daemon`), поэтому пока пул не остановлен методами `shutdown()` или `shutdownNow()`, JVM не завершится.

```
pool.shutdown(); // мягко: новые не
принимает, текущие и очередь дорабатывают
try {
    if (!pool.awaitTermination(5, TimeUnit.SECONDS)) {
        pool.shutdownNow(); // жёстко: interrupt
воркерам, очередь возвращается списком
    }
} catch (InterruptedException e) {
    pool.shutdownNow();
    Thread.currentThread().interrupt(); // восстанавливаем
флаг
}
```

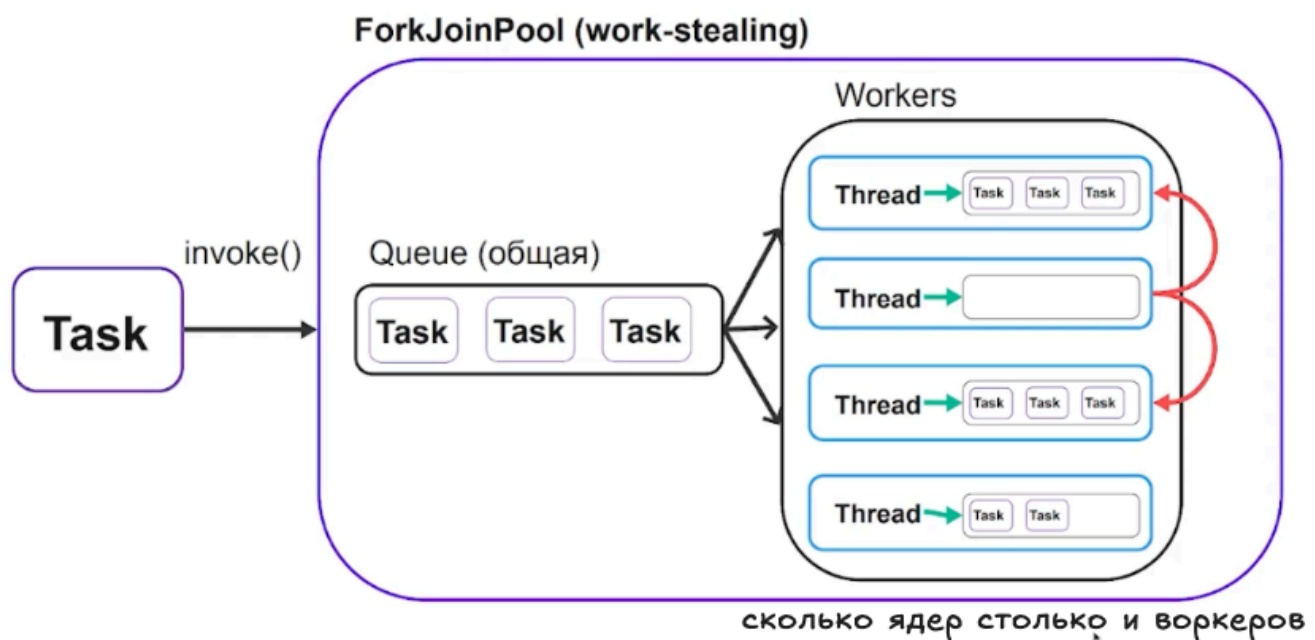
- `shutdown()` : перестаёт принимать задачи и не ждёт завершения.
- `awaitTermination(t, u)` : ждёт завершения, вернёт `true`, если пул успел остановиться.
- `shutdownNow()` : шлёт `interrupt()` воркерам и возвращает невыполненные задачи. Это лишь просьба остановиться: задача, игнорирующая прерывание, может не завершиться, поэтому код задач должен уважать флаг прерывания.
- `close()` (JDK 19+, `try-with-resources`): вызывает `shutdown()` и ждёт завершения всех задач без ограничения по времени; при прерывании вызывает `shutdownNow()`.

ForkJoinPool в Java — work-stealing и parallel streams

`ForkJoinPool` — отдельный вид пула для задач, которые делятся на подзадачи и ждут их результаты. Потоки в нём сами забирают работу друг у друга, чтобы ядра не простаивали. Через него по умолчанию работают `parallel streams` и `async-методы CompletableFuture`.
Пакет: `java.util.concurrent`.

`ForkJoinPool` — пул для «разделяй и властвуй»

`ForkJoinPool` — пул, рассчитанный на задачи, которые делятся на подзадачи и ждут их результаты (`fork` — отправить подзадачу, `join` — дождаться). В нём работает `work-stealing`: потоки сами забирают работу друг у друга, чтобы ядра не простаивали.



- Своя очередь у каждого воркера: у каждого потока двусторонняя очередь (deque); свои подзадачи он берёт с одного конца (по умолчанию LIFO, стек).
- Кража задач: простаивающий воркер забирает задачу с противоположного конца чужой очереди (самые крупные, «старые» подзадачи).
- Параллелизм: по умолчанию равен числу ядер.

Зачем он, если есть обычный пул с потоками = ядрам

Рекурсивная задача ждёт подзадачи (`join()`). В обычном пуле такой поток просто блокируется, и ядро простаивает; добавлять потоки нельзя: переключения контекста съедят выигрыш CPU-bound. Воркер `ForkJoinPool` вместо ожидания берёт другую задачу (свою или чужую), и все ядра заняты без лишних потоков. Мелкие подзадачи при этом лежат в локальной очереди без борьбы за общую.

RecursiveTask — сумма массива

Схема: пока задача большая, делим; когда стала маленькой (порог), считаем в лоб.

```
class SumTask extends RecursiveTask<Long> {
    private static final int THRESHOLD = 10_000; // ниже порога делить
    // дороже, чем считать
    private final int[] arr; private final int from, to;

    SumTask(int[] arr, int from, int to) { this.arr = arr; this.from = from;
    this.to = to; }

    @Override
    protected Long compute() {
        int len = to - from;
        if (len <= THRESHOLD) { // база: считаем
            // сами
            long sum = 0;
            for (int i = from; i < to; i++) sum += arr[i];
            return sum;
        }
        int mid = from + len / 2;
        SumTask left = new SumTask(arr, from, mid);
        SumTask right = new SumTask(arr, mid, to);
        left.fork(); // левую половину
        // отправляем в пул
        long r = right.compute(); // правую считаем в
        // этом же потоке
        long l = left.join(); // ждём левую
        return l + r;
    }
}

long sum = ForkJoinPool.commonPool().invoke(new SumTask(arr, 0,
arr.length));
```

Метод	Смысл
<code>fork()</code>	асинхронно отправить подзадачу в пул, вернуться сразу
<code>join()</code>	дождаться результата подзадачи
<code>compute()</code>	выполнить подзадачу прямо в текущем потоке
<code>pool.invoke(task)</code>	отправить корневую задачу и дождаться результата

- Стандартный приём: одну половину `fork()` , вторую `compute()` самому, чтобы не плодить лишнюю задачу.
 - `RecursiveTask<V>` возвращает результат, `RecursiveAction` — нет (сортировка, обход).
 - Экземпляр `ForkJoinTask` одноразовый: на каждый запуск нужен новый объект.
-

Common pool — общий пул JVM

`ForkJoinPool.commonPool()` — один статический пул на всю JVM.

- Параллелизм: по умолчанию число ядер – 1 (на машине с 2 ядрами `getCommonPoolParallelism()` вернул 1). Меняется свойством `java.util.concurrent.ForkJoinPool.common.parallelism`.
- Потоки `daemon`, а `shutdown()` на этом пуле ничего не делает.
- Через него по умолчанию работают `parallel streams` и асинх-методы `CompletableFuture` без явного `executor`.

⚠ Пул общий, поэтому тяжёлая или блокирующая задача занимает его воркеры, и чужие задачи встанут в очередь: задержка всплывает в совсем другом месте приложения. В серверном коде для тяжёлой и IO-работы берут свой пул, а `common` оставляют для коротких CPU-вычислений.

- Задачи `ForkJoinPool` по документации «не должны блокироваться» (I/O, `sleep`, ожидание лока). Для неизбежной блокировки есть `ManagedBlocker`, но в конспекте достаточно правила «блокирующее не сюда».
 - Потоки `common pool daemon` и не держат JVM: программа может завершиться раньше асинхронной задачи. Перед выходом ждут через `commonPool().awaitQuiescence(...)`.
-

Parallel streams

```
long s = IntStream.rangeClosed(1, 1_000_000)
    .parallel() // разбиение на части, выполнение в
common pool, склейка результата
    .mapToLong(i -> i)
    .sum();
```

- Порядок обработки элементов не определён (`peek` / `forEach` печатают вразнобой), детерминирован только результат.
- Годаются для чистых CPU-вычислений над большими данными, без побочных эффектов и блокировок.
- Не для IO-bound: займут общий пул.
- Лямбды не должны менять общее состояние без синхронизации.

ThreadPoolExecutor против ForkJoinPool

	ThreadPoolExecutor	ForkJoinPool
Очередь	одна общая	своя deque у каждого воркера + воровство
Задачи	независимые	делимые, ждущие подзадачи (fork/join)
Нагрузка	CPU-bound и IO-bound	CPU-bound
Типично	сервер, задачи извне	parallel streams, divide & conquer

Callable / Future → CompletableFuture : асинхронные результаты

`Callable` и `Future` описывают задачу, которая возвращает результат из пула потоков, но результат получают только блокирующим `get()`.

`CompletableFuture` развивает эту идею: вместо ожидания описывают, что сделать с результатом, когда он появится. Пакет: `java.util.concurrent`.

Callable и Future — задача с результатом

`Callable<V>` — как `Runnable`, но возвращает результат и может бросать checked-исключения. `Future<V>` — «ручка» на задачу, которая выполняется в пуле: через неё получают результат, отменяют задачу и проверяют статус.

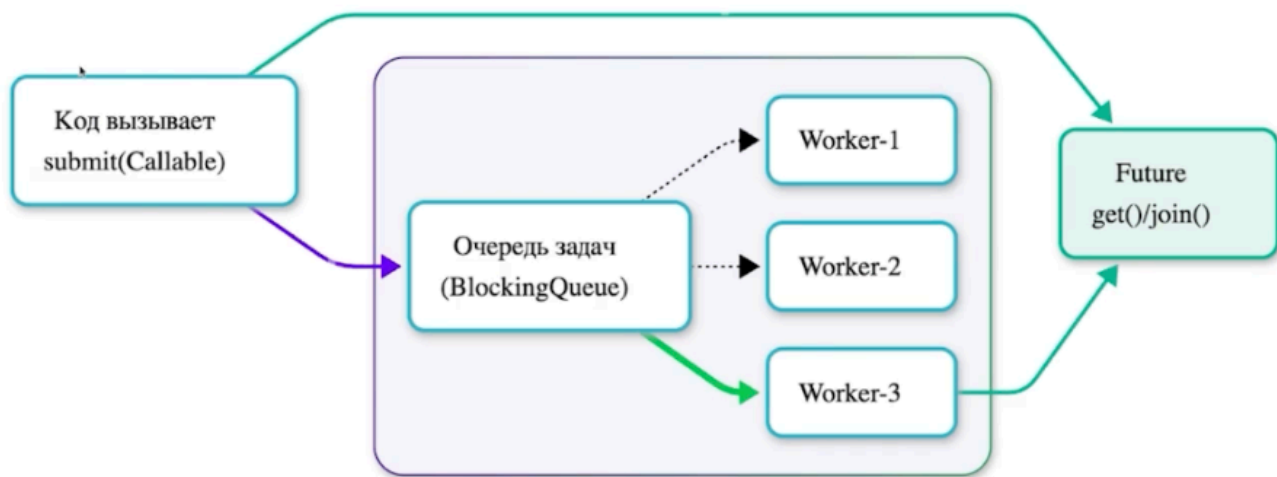
Свойство	Runnable	Callable<V>
Метод	<code>void run()</code>	<code>V call() throws Exception</code>
Результат	нет	да, через <code>Future</code>
Checked-исключения	нельзя бросить	можно

```

ExecutorService pool = Executors.newFixedThreadPool(4);

Future<Integer> f = pool.submit(() -> 42);    // вернулся сразу: задача уже в
очереди или выполняется
// ... своя работа ...
Integer result = f.get();                    // блокируемся, пока результат
не готов

```



- `submit` оборачивает задачу в `FutureTask`: он реализует и `Runnable`, и `Future`, поэтому пул выполняет его как обычную задачу.
- `Future` возвращается сразу, когда задача уже запланирована или запущена. Ждать на этом месте не нужно.

Методы Future

- `get()` / `get(t, unit)`: ждёт результат (блокирует). С таймаутом может бросить `TimeoutException`.
- `isDone()`: `true`, если задача завершилась любым способом: результат, исключение или отмена.
- `cancel(mayInterrupt)`: отмена. Если задача ещё не стартовала, она не запустится. Если выполняется, при `mayInterrupt = true` потоку шлётся `interrupt()`.
- `isCancelled()`: задачу отменили.
- `resultNow()`, `exceptionNow()`, `state()` (JDK 19+): результат или исключение без ожидания и состояние `RUNNING` / `SUCCESS` / `FAILED` / `CANCELLED`.

Что бросает get()

Исход задачи	<code>get()</code>
завершилась нормально	возвращает результат
бросила исключение	<code>ExecutionException</code> , настоящая причина в <code>getCause()</code>
отменена	<code>CancellationException</code>
ждущий поток прервали	<code>InterruptedException</code>

⚠ Исключение в задаче не убивает пул: оно прячется в `Future` и всплывает только при `get()`. Если `Future` выброшен без `get()`, ошибка пропадёт молча.

`invokeAll`, `invokeAny` и порядок результатов

```
List<Future<String>> fs = pool.invokeAll(tasks); // ждёт ВСЕ; список в
// порядке задач; у каждого isDone() == true
String first = pool.invokeAny(tasks);           // результат первой
// УСПЕШНОЙ, остальные отменяются
```

- Если вызывать `get()` по списку `Future` по очереди, ожидание идёт в порядке списка: первая долгая задача задержит чтение уже готовых.
- Для «брать по мере готовности» есть `ExecutorCompletionService`.

Ограничения `Future`

- Результат можно получить только блокирующим `get()`: нельзя сказать «когда будет готово, сделай то-то».
- Нельзя объединять задачи (дождаться двух и склеить), нет цепочек.
- Обработка ошибок только через `try/catch` вокруг `get()`.

Всё это решает `CompletableFuture` (ниже).

`CompletableFuture` — асинхронные цепочки

`CompletableFuture<T>` реализует `Future` и `CompletionStage`. Идея: вместо блокирующего `get()` описываем, что сделать с результатом, когда он появится: цепочка стадий, объединение нескольких задач, обработка ошибок. Каждая стадия — тоже `CompletableFuture`. Пакет: `java.util.concurrent`.

```
CompletableFuture<Void> cf = CompletableFuture
    .supplyAsync(() -> loadUser())           // запуск в пуле, вернёт
```

значение

```
.thenApply(user -> user.name()) // преобразовать результат
```

```
.thenAccept(name -> log(name)) // потребить, ничего не
```

возвращая

```
.exceptionally(ex -> { log(ex); return null; }); // поймать ошибку
```

любой стадии выше

```
cf.join();
```

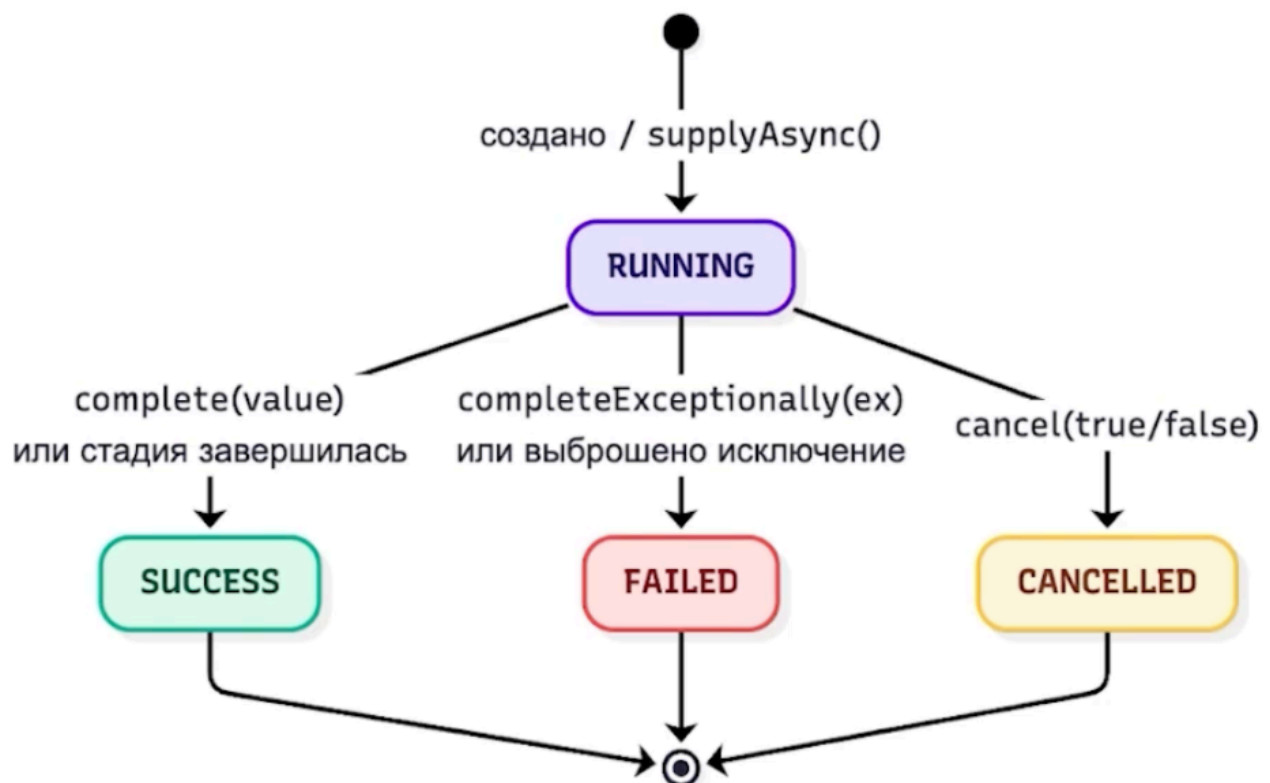
```
// дождаться (в main, иначе
```

```
JVM может выйти раньше)
```

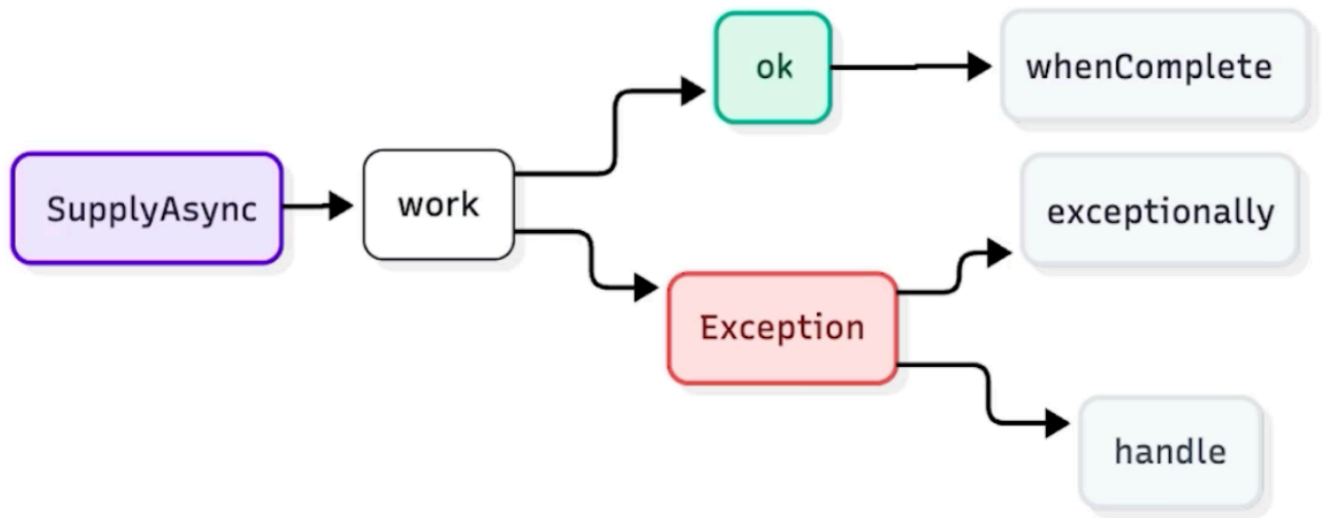
Создание

- `supplyAsync(Supplier)` : асинхронно выполнить и вернуть значение.
- `runAsync(Runnable)` : асинхронно выполнить, значения нет (`CF<Void>`).
- `completedFuture(v)` : уже готовый результат.
- `new CompletableFuture<>() + complete(v) / completeExceptionally(ex)` : завершить вручную из любого потока.
- У `supplyAsync / runAsync` есть вариант со вторым аргументом: своим `Executor` .

Жизненный цикл



Стадия завершается нормально (значение) или исключением. Исключение «протекает» вниз по цепочке: все `then*` пропускаются, пока не встретится `exceptionally` / `handle` / `whenComplete`.



Основные операторы

Оператор	Что делает	Когда
<code>thenApply(f)</code>	$T \rightarrow U$, <code>f</code> возвращает значение	преобразование результата
<code>thenAccept(c)</code> / <code>thenRun(r)</code>	потребить результат / просто выполнить	конец цепочки
<code>thenCompose(f)</code>	<code>f</code> возвращает <code>CompletableFuture</code> , цепочка сшивается	следующий шаг тоже асинхронный
<code>thenCombine(other, f)</code>	ждёт два независимых CF и склеивает результаты	параллельные вызовы
<code>thenAcceptBoth</code> / <code>runAfterBoth</code>	то же, но ничего не возвращают	оба результата нужны / нужен только факт «оба готовы»
<code>allOf(...)</code> / <code>anyOf(...)</code>	ждать все / первую	fan-out / fan-in

`thenApply` против `thenCompose`

```
cf.thenApply(x -> asyncCall(x));    // CF<CF<R>>: вложенный CF, неудобно
cf.thenCompose(x -> asyncCall(x));  // CF<R>:      «flatMap» для асинхронных
цепочек
```

thenCompose против thenCombine

```
// ПОСЛЕДОВАТЕЛЬНО: второй вызов зависит от первого
loadUser(id).thenCompose(user -> loadOrders(user));

// ПАРАЛЛЕЛЬНО: два независимых вызова, запущенные заранее, склеиваем
результат
var user    = loadUser(id);
var orders  = loadOrders(id);
user.thenCombine(orders, (u, o) -> new Page(u, o)).join();
```

Правило: независимые задачи запускаем сразу и склеиваем `thenCombine`; вторая зависит от первой: `thenCompose`. Если последовательно соединить независимые вызовы через `thenCompose`, параллелизм теряется.

allOf и anyOf

```
CompletableFuture.allOf(cf1, cf2, cf3).join();    // CF<Void>:
ждем все
var a = cf1.join(); var b = cf2.join();           // значения
забираем у каждого отдельно

Object first = CompletableFuture.anyOf(cf1, cf2, cf3).join(); // первая
завершившаяся, тип Object
```

- `allOf` результатов не несёт (`CF<Void>`).
- `anyOf` возвращает первую завершившуюся стадию, в том числе завершившуюся ошибкой: если самая быстрая упала, упадёт и `anyOf` (проверено: быстрая ошибка победила медленный успех). Это не «первая успешная».

Обработка ошибок

Оператор	Видит результат	Видит ошибку	Может заменить значение
<code>exceptionally(ex -> ...)</code>	нет	да	да (ветка ошибки)
<code>handle((res, ex) -> ...)</code>	да	да	да
<code>whenComplete((res, ex) -> ...)</code>	да	да	нет: побочный эффект, результат и ошибка идут дальше как есть

```
future.handle((res, ex) -> ex != null ? 0 : res + 10);    // обе ветки в
одной стадии
future.whenComplete((res, ex) -> log(res, ex));          // только
логирование, ничего не меняет
```

- `exceptionallyCompose` (JDK 12+): как `exceptionally`, но вместо значения возвращает новый CF (резервный асинхронный вызов).
- Обёртка исключений: исключение, дошедшее до зависимой стадии или `join()`, приходит как `CompletionException` (причина в `getCause()`). `join()` бросает `CompletionException`, `get()` — `ExecutionException` с той же причиной. Checked-исключений у `join()` нет, поэтому его чаще и берут. Проверено:

```
join -> CompletionException  cause=IllegalStateException
get  -> ExecutionException   cause=IllegalStateException
```

- Поэтому в `exceptionally` / `handle` на нижних стадиях `ex` часто оказывается `CompletionException`, а настоящая ошибка лежит в `ex.getCause()`.

Таймауты (JDK 9+)

```
cf.orTimeout(2, TimeUnit.SECONDS);                      // не успел: завершится
TimeoutException
cf.completeOnTimeout(DEFAULT, 2, TimeUnit.SECONDS)     // не успел: завершится
значением по умолчанию
```

Оба метода меняют сам `cf`, а не создают новый.

Какой поток выполняет стадию

Каждый оператор имеет три формы: `thenApply`, `thenApplyAsync(f)`, `thenApplyAsync(f, executor)`.

Форма	Где выполнится
<code>thenApply</code> (без <code>Async</code>)	в потоке, который завершил предыдущую стадию (или в потоке, вызвавшем метод, если стадия уже готова)
<code>thenApplyAsync(f)</code>	в <code>ForkJoinPool.commonPool()</code>
<code>thenApplyAsync(f, executor)</code>	в переданном пуле

Лёгкие стадии (преобразование, лог) оставляют без `Async`, тяжёлые уводят в пул, чтобы не занимать чужой поток.

Свой пул обязателен для IO

Без явного `executor` все `supplyAsync` / `*Async` идут в `common pool` (общий с `parallel streams`, размер — ядра - 1). Блокирующие IO-вызовы там быстро забивают воркеры.

```
ExecutorService io = Executors.newFixedThreadPool(50);
CompletableFuture.supplyAsync(() -> callRemoteService(), io);
```

Если `common pool` не поддерживает параллелизм хотя бы 2 (например, на 1–2 ядрах), по документации каждая задача запускается в новом потоке (в проверке на машине с 2 ядрами это был обычный не-daemon `Thread-N`).

Ловушки

- `cancel()` не прерывает выполнение: для `CompletableFuture` это то же, что `completeExceptionally(new CancellationException())`. Стадия завершена, но уже запущенная задача продолжает работать до конца (проверено: после `cancel(true)` задача доспала и закончилась, `interrupt` не пришёл). Это отличие от `FutureTask`.
- Ошибки теряются без обработчика: цепочка упала, логов нет, пока не вызвали `join()`. Цепочку всегда заканчивают `exceptionally` / `handle` / `whenComplete`.
- `join()` / `get()` внутри стадии блокирует воркер пула и губит асинхронность: вместо этого `thenCompose` или `thenCombine`.

- `main` завершился раньше: потоки `common pool daemon`, поэтому без `join()` в `main` асинхронная задача может не успеть.
 - `allOf` не возвращает значений и не отменяет остальные стадии при ошибке одной.
-

Конкурентные коллекции в Java

Конкурентные коллекции из `java.util.concurrent` — потокобезопасные коллекции, которые вместо одной блокировки на всю структуру используют более тонкие механизмы: CAS, блокировку отдельной корзины, снимки массива. Цель — чтобы потоки как можно меньше ждали друг друга. Пакет: `java.util.concurrent`.

Почему обычные коллекции опасны

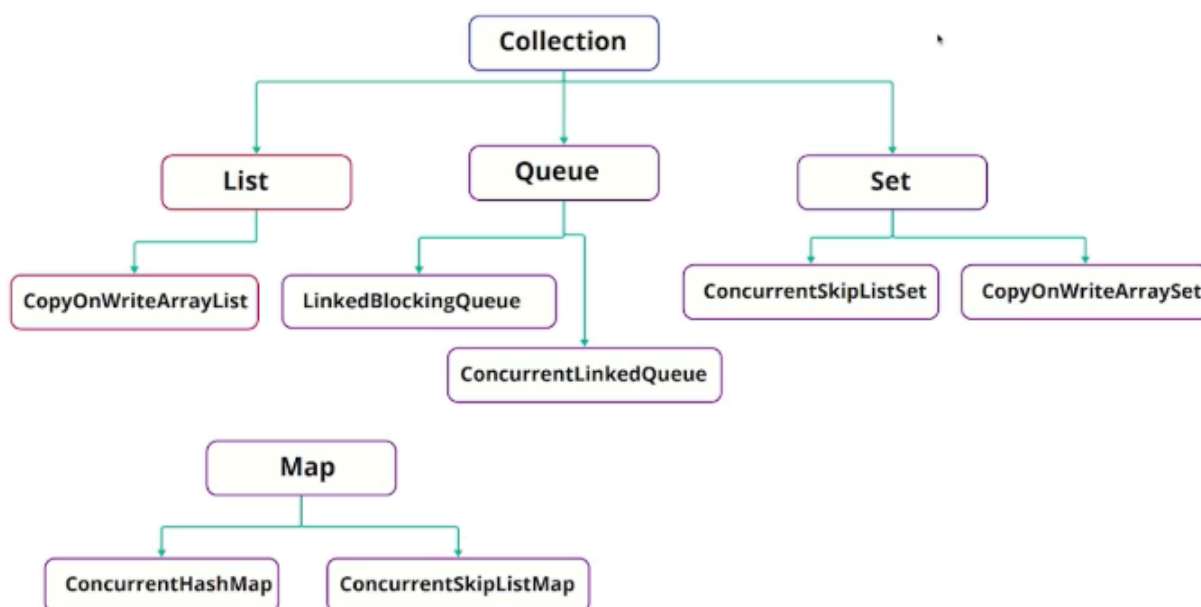
`ArrayList` и `HashMap` не потокобезопасны: при конкурентной записи теряются элементы, ломается внутреннее состояние, возможны исключения.

```
List<Integer> list = new ArrayList<>();           // 2 потока × 100_000
add()
Map<Integer, Integer> map = new HashMap<>();      // 2 потока × 50_000
put()
// ArrayList: size = 173477 вместо 200000   (запуск; число меняется от раза
к разу)
// HashMap:   size = 95036   вместо 100000
```

Обёртки `Collections.synchronizedList/Map` работают, но это один монитор на всю коллекцию, поэтому параллелизм пропадает. Есть и другие недостатки:

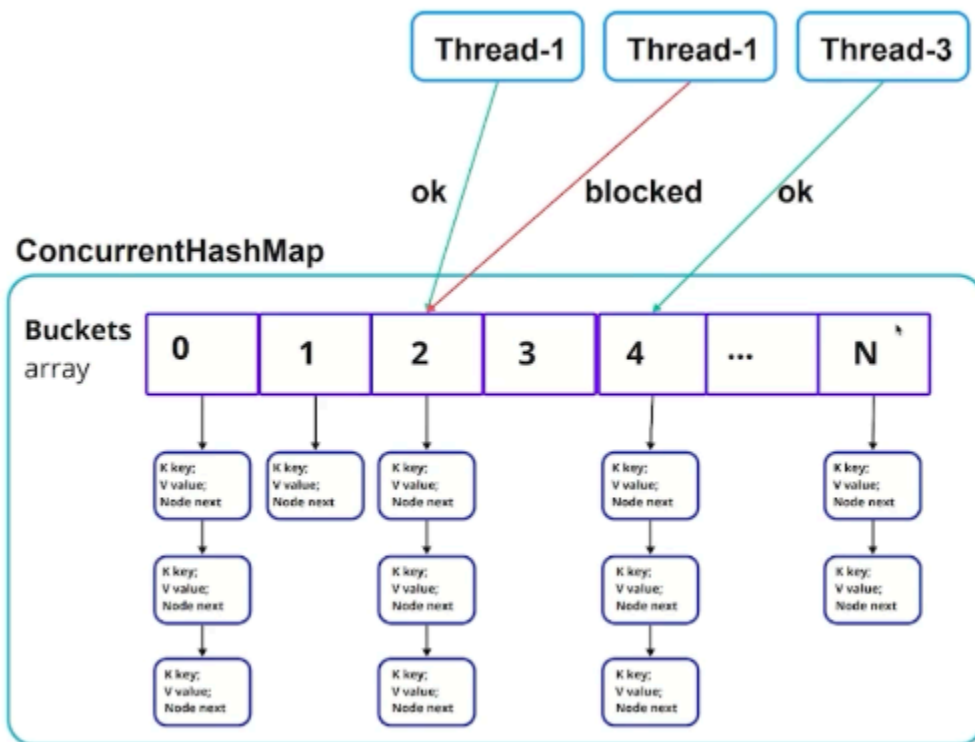
- итерироваться по такой коллекции нужно вручную под `synchronized (coll)`, иначе `ConcurrentModificationException` (проверено: добавление во время обхода бросает его);
- составные операции вроде «проверил, потом положил» остаются неатомарными.

Concurrent collections



ConcurrentHashMap

`ConcurrentHashMap` (CHM) — хеш-таблица для сценария «много потоков одновременно читают и пишут». Документация: «full concurrency of retrievals and high expected concurrency for updates».



Как устроена

- Чтение (`get`) без блокировок: читатели видят актуальные значения через `volatile` - ссылки и могут идти параллельно с записью.
- Запись в пустую корзину идёт через `CAS` без блокировки, в занятую корзину — через `synchronized` на первый узел. Блокируется только эта корзина, а не вся таблица.
- `concurrencyLevel` в конструкторе — лишь подсказка для внутреннего размера, а не число блокировок.
- Из старого конспекта (условные цифры, 50 писателей + 5000 читателей):
`synchronizedMap` ~3 с, `CHM` ~0,6 с. Точные числа зависят от машины, но разница на порядок при росте читателей постоянна.

Правила использования

- Ключ и значение не могут быть `null`: иначе `NullPointerException` (у `HashMap` можно).
- Итератор `weakly consistent`: `ConcurrentModificationException` не бросает, отражает состояние на момент создания или позже, изменения после этого могут быть видны, а могут и нет. Использовать его должен один поток.
- `size()` и `isEmpty()` при параллельных изменениях дают приблизительное значение «для мониторинга, но не для управления логикой»; для точного количества как `long` есть `mappingCount()`.
- Одиночная операция атомарна, а последовательность из нескольких вызовов — нет:

```
// ❌ check-then-act: два вызова, между ними вклинивается другой поток
Integer v = map.get("k"); map.put("k", v == null ? 1 : v + 1); // 4
потока × 100_000: итог 139194 из 400000
```

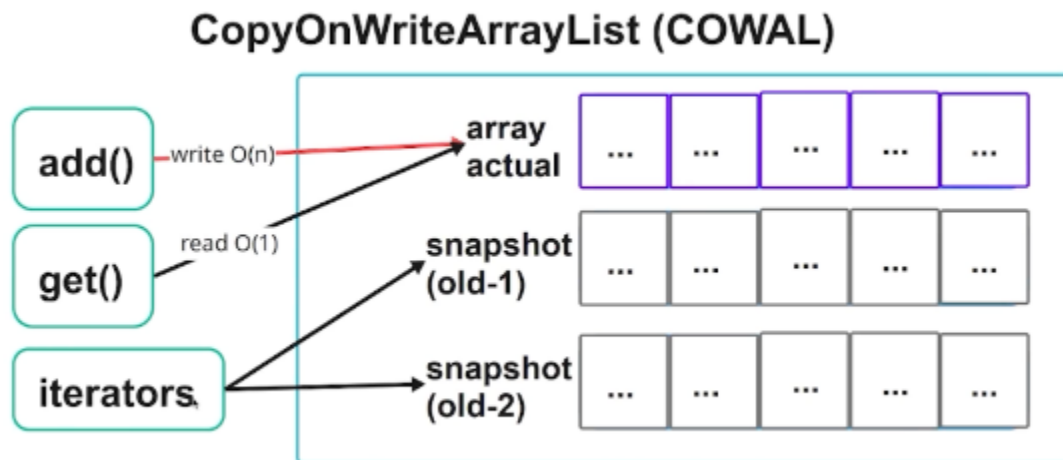
```
// ✅ одна атомарная операция
map.merge("k", 1, Integer::sum); // итог
400000
```

- Атомарные операции: `putIfAbsent`, `computeIfAbsent`, `compute`, `merge`. Функция внутри должна быть короткой, без побочных эффектов и не должна менять эту же карту.
- Счётчик на ключ, шаблон из документации (связь с `LongAdder` из блока про атомики):

```
ConcurrentHashMap<String, LongAdder> freqs = new ConcurrentHashMap<>();
freqs.computeIfAbsent(key, k -> new LongAdder()).increment();
```

CopyOnWriteArrayList

Идея: каждая запись копирует весь массив и подменяет ссылку. Читатели работают со снимком, который больше не меняется.



```
// из JDK: запись под synchronized, массив копируется целиком, поле array – volatile
public boolean add(E e) {
    synchronized (lock) {
        Object[] es = getArray();
        es = Arrays.copyOf(es, es.length + 1);
        es[es.length - 1] = e;
    }
}
```

```

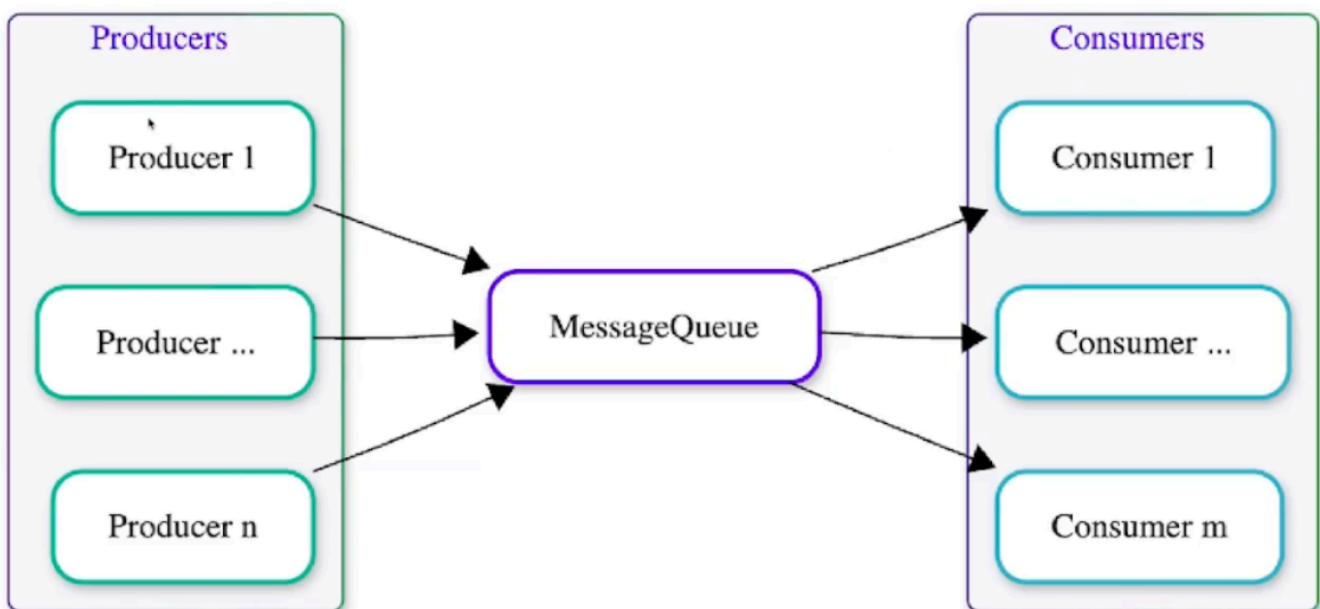
        setArray(es);
        return true;
    }
}

```

- Чтение без блокировок: `get` и обход идут по текущему массиву.
- Итератор — снимок: во время обхода можно менять список, исключения не будет, но изменения итератор не увидит (проверено: итерировали 3 элемента, добавляя по одному, итог 6 элементов, итератор прошёл 3).
- Запись $O(n)$ и новый массив при каждом изменении; писатели между собой сериализуются через `synchronized`. Старые снимки держат итераторы, пока GC их не уберёт.
- Подходит, когда читают намного чаще, чем пишут: слушатели событий, конфиги, справочники. Документация: может быть эффективнее альтернатив, «когда обходов намного больше, чем изменений».
- `CopyOnWriteArraySet` построен на нём же, и те же издержки на запись.

Блокирующие очереди (`BlockingQueue`)

`BlockingQueue` — очередь, которая сама блокирует потоки: `put` ждёт места, `take` ждёт элемента. Основа паттерна `Producer–Consumer`: производители кладут, потребители берут, очередь развязывает их скорости и безопасна для нескольких производителей и потребителей.



Четыре формы каждой операции

Операция	Исключение	Специальное значение	Блокируется	Таймаут
Вставка	<code>add(e)</code>	<code>offer(e)</code>	<code>put(e)</code>	<code>offer(e, time, unit)</code>
Извлечение	<code>remove()</code>	<code>poll()</code>	<code>take()</code>	<code>poll(time, unit)</code>
Просмотр	<code>element()</code>	<code>peek()</code>	—	—

- На полной очереди `add` бросает `IllegalStateException("Queue full")`, `offer` возвращает `false` (проверено), `put` ждёт, `offer` с таймаутом ждёт и потом возвращает `false`.
- На пустой `remove` бросает `NoSuchElementException`, `poll` возвращает `null`.
- `null` в очередь класть нельзя (`NullPointerException`): `null` — признак «пусто» для `poll`.
- Happens-before: действия потока до помещения элемента в очередь видны потоку, получившему этот элемент.
- `drainTo(collection)` забирает все доступные элементы за один вызов; `remainingCapacity()` — сколько ещё влезет (`Integer.MAX_VALUE`, если лимита нет).

Реализации

Очередь	Внутри	Особенности
<code>ArrayBlockingQueue</code>	кольцевой массив фиксированной ёмкости	один <code>ReentrantLock</code> и два <code>Condition</code> (<code>notEmpty</code> , <code>notFull</code>), опциональный <code>fair</code> -режим; ёмкость задаётся при создании
<code>LinkedBlockingQueue</code>	связный список	две блокировки: <code>putLock</code> для производителей, <code>takeLock</code> для потребителей, поэтому выше пропускная способность; по умолчанию ёмкость <code>Integer.MAX_VALUE</code>, то есть неограниченная ; нода на каждый элемент
<code>SynchronousQueue</code>	не хранит элементов	каждая вставка ждёт соответствующего извлечения (прямая передача); так работает <code>newCachedThreadPool</code>
<code>PriorityBlockingQueue</code>	куча	неограниченная, элементы по приоритету, а не по порядку вставки

Очередь	Внутри	Особенности
LinkedBlockingDeque	связный список, двусторонний	блокирующая двусторонняя очередь; не lock-free

- `ArrayBlockingQueue` устроена ровно как `BoundedBuffer` из файла про `Condition`: один лок, два условия.
- Ограниченная очередь даёт back-pressure: производитель замедляется, когда потребитель не успевает. Неограниченная (`LinkedBlockingQueue` без размера) этого не даёт и при перегрузе ведёт к `OutOfMemoryError`.
- Корректная остановка потребителя: Poison Pill, то есть специальный элемент-сигнал «завершайся»:

```

BlockingQueue<Integer> queue = new ArrayBlockingQueue<>(50);
int POISON = Integer.MIN_VALUE;

// производитель: queue.put(x);    ...    в конце: queue.put(POISON);
// потребитель:
while (true) {
    int x = queue.take();              // блокируется, пока нет элемента
    if (x == POISON) break;
    handle(x);
}

```

Неблокирующие и упорядоченные коллекции

Коллекция	Особенности	Где уместна
<code>ConcurrentLinkedQueue</code> / <code>ConcurrentLinkedDeque</code>	lock-free на CAS, неограниченные, нет блокирующего <code>take()</code> , <code>size()</code> за $O(n)$	очередь событий, когда ждать не нужно (потребитель опрашивает <code>poll()</code>)
<code>ConcurrentSkipListMap</code> / <code>ConcurrentSkipListSet</code>	отсортированные, операции за $O(\log n)$	нужен порядок ключей и конкурентный доступ

Если потребителю нужно ждать, пока появятся данные, берут `BlockingQueue`. `ConcurrentLinkedQueue` подходит там, где ждать не нужно, а нужна просто быстрая очередь без блокировок.

Как выбрать коллекцию

Задача	Что брать
словарь, много параллельных чтений и записей	<code>ConcurrentHashMap</code>
список, читают почти всегда, пишут редко	<code>CopyOnWriteArrayList</code>
очередь между потоками с ожиданием и back-pressure	<code>ArrayBlockingQueue</code> / <code>LinkedBlockingQueue</code> (с ограничением ёмкости)
очередь без блокировок, ждать не нужно	<code>ConcurrentLinkedQueue</code>
нужен порядок ключей	<code>ConcurrentSkipListMap</code> / <code>Set</code>
заплата для старого кода	<code>Collections.synchronizedXxx</code>

Virtual Threads

Virtual Threads (Project Loom): устройство и применение

Виртуальные потоки — лёгкие потоки, которые планирует сама JVM поверх небольшого набора обычных OS-потоков. Они позволяют писать обычный блокирующий код и при этом держать десятки тысяч одновременных задач. В JDK 21 стали стабильной функцией (не preview). Класс тот же, `java.lang.Thread`.

⚠ Версии JDK

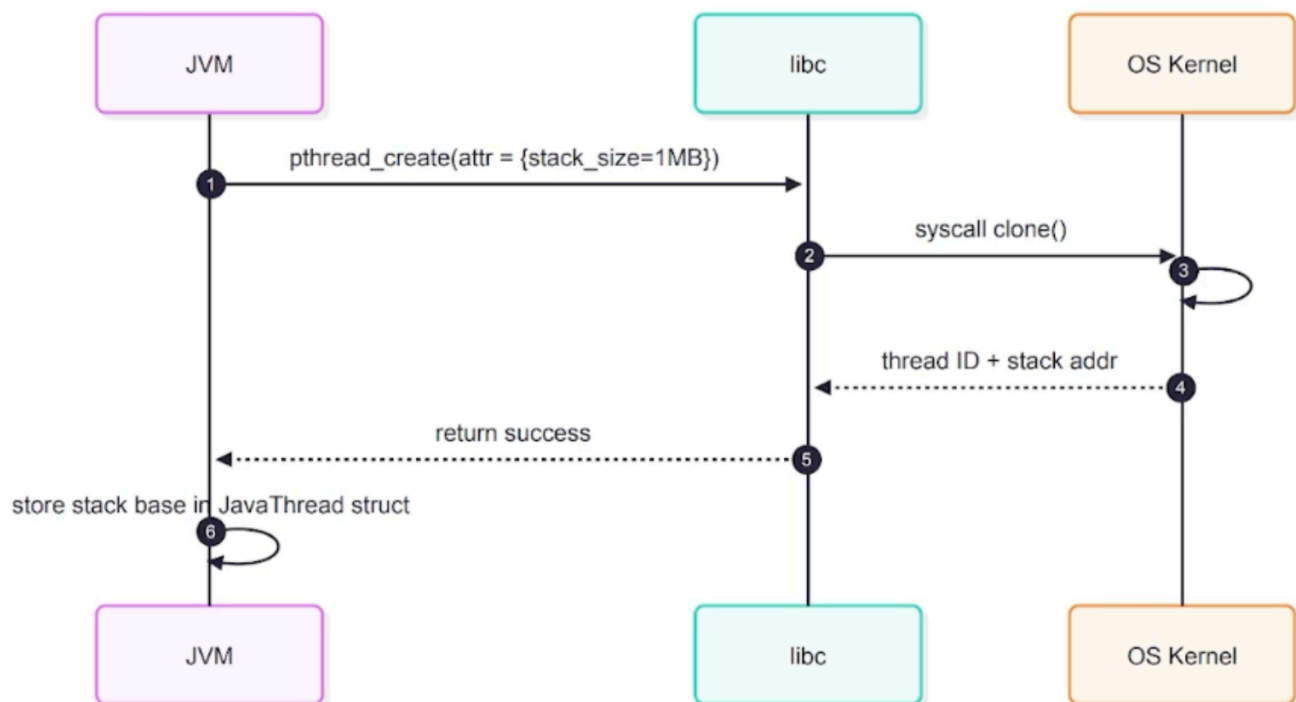
Виртуальные потоки стабильны с JDK 21, но на версиях до 25 их нужно использовать аккуратно:

- **JDK 21–23:** блокировка внутри `synchronized` закрепляет (pin) носитель, масштабируемость падает. Внутри критической секции с IO берите `ReentrantLock`.
- **JDK 24+ (LTS-линейка: JDK 25):** JEP 491 убирает pinning на `synchronized`. Остаются нативные вызовы.
- **JDK 25+:** `ScopedValue` финализирован (JEP 506) и заменяет `ThreadLocal` для передачи контекста.

- Трассировка pinning: `-Djdk.tracePinnedThreads` работает на JDK 21–23, в JDK 24+ свойство удалено, остаётся JFR-событие `jdk.VirtualThreadPinned`.

Проблема: OS-потоки дорогие

Платформенный поток — это 1:1 обёртка над потоком ОС, и его создание требует системного вызова.



- Мало: на процесс реально создают десятки или сотни потоков, тысячи уже тяжело.
- Дорого: у каждого свой native stack (по умолчанию порядка 1 МБ, настраивается – `Xss`), syscall на создание и переключение контекста.
- Предел: при тысячах потоков `OutOfMemoryError: unable to create native thread`.

Потоки в бэкенде в основном ждут:

Тип операции	CPU	Ожидание IO
HTTP-запрос	меньше 1%	больше 99%
JDBC-запрос	меньше 1%	больше 99%
Чтение и запись файла	меньше 1%	больше 99%

Классическое решение — пул из N потоков, N много больше числа ядер. Но держать 10 000 OS-потоков, которые просто ждут сокет, расточительно. Альтернатива — асинхронный код (колбэки, `CompletableFuture`, реактивные библиотеки), но он сложнее писать и отлаживать.

Что такое виртуальный поток

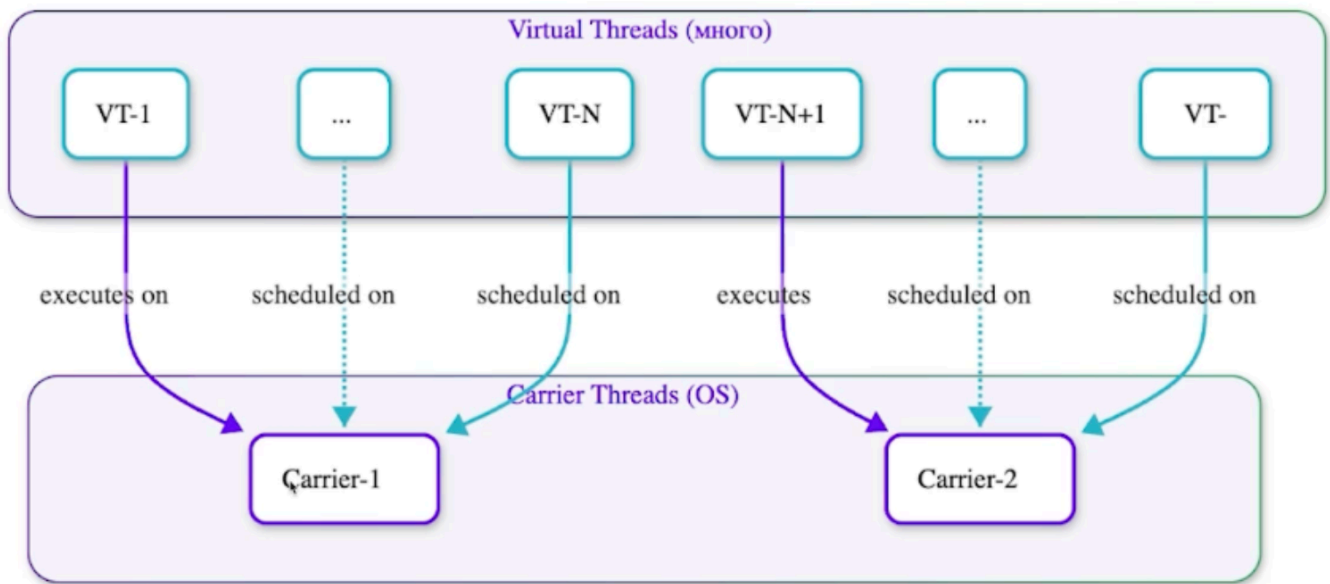
Виртуальный поток — экземпляр `Thread`, не привязанный к конкретному OS-потоку. JVM исполняет его на носителе (`carrier thread`) — обычном платформенном потоке из своего пула.

- Модель N:M: много виртуальных потоков отображаются на малое число `carrier`-потоков.
- Блокирующая операция (IO, `sleep`, ожидание `ReentrantLock`, `LockSupport.park`) снимает виртуальный поток с носителя (`unmount`), и носитель берёт другой.
- Планировщик — `ForkJoinPool` в FIFO-режиме; параллелизм по умолчанию равен числу доступных процессоров (меняется свойством `jdk.virtualThreadScheduler.parallelism`).
- Виртуальные потоки всегда `daemon`: сделать их не-`daemon` нельзя. Если `main` завершился, JVM не будет ждать их.

```
Thread t = Thread.ofVirtual().name("vt-1").start(() ->
System.out.println(Thread.currentThread()));
// VirtualThread[#19,vt-1]/runnable@ForkJoinPool-1-worker-1    ← запущен на
carrier'e из ForkJoinPool
t.join();
```

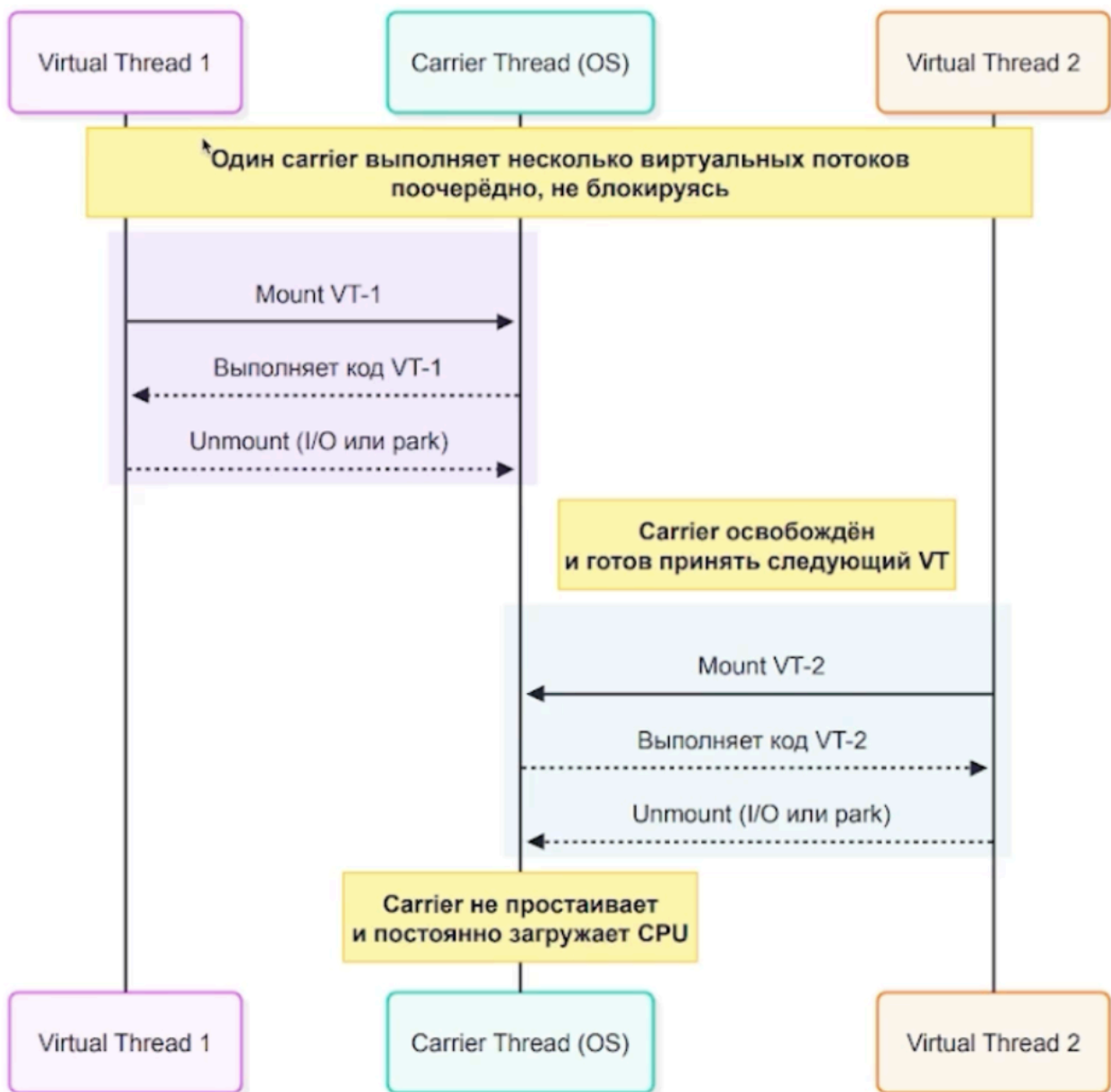
Связь с пулами

- `ForkJoinPool` — планировщик для CPU-bound задач: воркеры сами считают подзадачи, `work-stealing` балансирует ядра.
- Виртуальные потоки — для IO-bound задач: на блокирующем вызове поток паркуется и освобождает носитель.



Mount / Unmount стека

- При блокировке виртуальный поток снимает стек с носителя (unmount), и носитель свободен для других.
- При возобновлении стек монтируется обратно (mount), возможно, уже на другом носителе.
- Между монтированиями стек живёт в куче (continuation), поэтому виртуальный поток занимает ровно столько, сколько нужно его стеку (обычно килобайты), а не мегабайт под резерв.



⚠ Не любая блокировка снимает поток с носителя: если поток «прибит» (pinning), носитель остаётся занят. Об этом отдельный файл.

Виртуальные vs платформенные потоки

Критерий	Виртуальные	Платформенные
Соответствие ОС	N:M через carrier-потоки	1:1 с OS-потоком
Где хранится стек	в куче (continuation)	в native stack
Память	килобайты, по реально используемому	порядка 1 МБ резерва
Количество	десятки и сотни тысяч	десятки, сотни

Критерий	Виртуальные	Платформенные
Планирование	JVM	ОС
Блокировка	паркуются, носитель свободен	блокируют ОС-поток
Переключение	дешёвое, без системного вызова	дорогое, syscall
Daemon	всегда	настраивается

Создание

```
Thread.ofVirtual().name("vt-" + id).start(task);           // билдер, как
Thread, но виртуальный
Thread.startVirtualThread(task);                           // короткая форма

try (ExecutorService ex = Executors.newVirtualThreadPerTaskExecutor()) {
    // новый виртуальный поток на каждую задачу
    ex.submit(task);
}                                                         // close(): ждёт
завершения всех задач
```

API зеркалит обычный Thread: те же start, join, isAlive.

newVirtualThreadPerTaskExecutor() — обёртка для совместимости с кодом на ExecutorService: submit, Future, try-with-resources.

Демо: 100 000 задач

```
try (var ex = Executors.newVirtualThreadPerTaskExecutor()) {
    for (int i = 0; i < 100_000; i++) {
        ex.submit(() -> { Thread.sleep(1000); return null; });    //
имитация IO: ждём 1 с
    }
}
// 100_000 виртуальных потоков, каждый спит по 1 с: ~2,2 с суммарно (JDK 21,
2 ядра)
```

С Executors.newFixedThreadPool(10_000) на таком же объёме нагрузка упирается в 10 000 платформенных потоков (порядка 10 ГБ под стеки, по оценке старого конспекта) вплоть до OutOfMemoryError: unable to create native thread. Стоимость виртуальной задачи — объект в куче.

Главные правила

- Не пулить: один виртуальный поток на одну задачу. Создавать их дёшево, поэтому пул из виртуальных потоков не нужен. Число виртуальных потоков равно числу одновременных задач. Для ограничения нагрузки на ресурс (например, соединения с БД) берут `Semaphore`.
- Блокирующий код — обычный: `Thread.sleep`, `JDBC`, `HttpClient.send` пишут как раньше.
- Не для CPU-bound: виртуальные потоки не быстрее, они дают масштаб (throughput), а не скорость (latency). Для вычислений — `ForkJoinPool`.
- Ориентир из документации: если в приложении никогда не бывает порядка 10 000 одновременных задач, выигрыша от виртуальных потоков, скорее всего, не будет.

Задача	Что брать
IO-bound сервер, много одновременных запросов	виртуальные потоки
CPU-bound вычисления	<code>ForkJoinPool</code>
независимые фоновые задачи с ограничением параллелизма	<code>ThreadPoolExecutor</code>

В Spring Boot 3.2+ виртуальные потоки для обработки запросов включаются настройкой (подробности в блоке «На проде»).

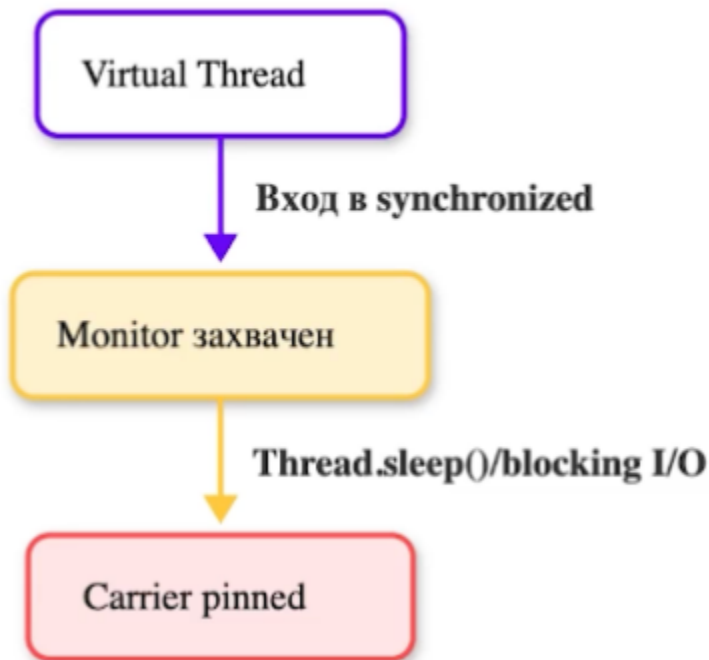
Проблемы Virtual Threads: pinning, нативные вызовы, ThreadLocal

Виртуальные потоки дёшевы, пока блокирующая операция снимает их с носителя. Ситуации, когда это не происходит, называются pinning. Второй источник проблем — `ThreadLocal`, который на миллионах потоков превращается в утечку памяти.

Что такое pinning

Pinning (закрепление) — состояние, когда виртуальный поток держит carrier-поток и не может быть снят с него при блокирующей операции.

В нормальной ситуации при IO происходит unmount, и носитель свободен для других виртуальных потоков. При pinning:



Носитель простаивает вместе с виртуальным потоком, и масштабируемость падает до уровня обычных потоков, а то и хуже.

Когда происходит

Причина	JDK 21	JDK 24+ (JEP 491)
блокировка внутри <code>synchronized</code> (блок или метод), в том числе <code>Object.wait()</code>	pinning	pinning нет
нативный метод или вызов через Foreign Function API, который блокируется	pinning	pinning остаётся

В JDK 24 вышел JEP 491 «Synchronize Virtual Threads without Pinning»: виртуальные потоки теперь могут захватывать, держать и отпускать монитор независимо от носителя. В LTS-линейке это JDK 25. Остаются редкие случаи, не связанные с `synchronized`.

Вывод по версиям:

- JDK 21 (и до JDK 24): долгая блокировка внутри `synchronized` ломает масштабируемость, внутри критической секции с IO берут `ReentrantLock`;
- JDK 25+: `synchronized` снова безопасен, но правило «не делать долгий IO внутри лока» остаётся в силе независимо от виртуальных потоков.

Эксперимент: `synchronized` против `ReentrantLock`

10 виртуальных потоков, каждый спит 1 с внутри критической секции. Для каждого свой лок, то есть конкуренции за лок нет, мешает только pinning. Носителей два (на машине 2 ядра):

```
// A: sleep внутри synchronized – носитель закреплён на всё время сна
synchronized (monitor) { Thread.sleep(1000); }

// B: sleep внутри ReentrantLock – виртуальный поток снимается с носителя
lock.lock();
try { Thread.sleep(1000); } finally { lock.unlock(); }
```

Вариант	Время (JDK 21, 2 носителя)
<code>synchronized</code>	~5 с (10 задач, по 2 одновременно: пять «волн»)
<code>ReentrantLock</code>	~1 с (все 10 спят одновременно)

Объяснение: ожидание `ReentrantLock` и `sleep` под ним реализованы через `LockSupport.park`, который понимает виртуальные потоки и освобождает носитель. Монитор в JDK 21 привязан к носителю, поэтому сон под ним блокирует и носитель.

Почему монитор закрепляет, а `ReentrantLock` нет

- Монитор (JDK 21): JVM запоминает владельца монитора как платформенный поток (носитель), а не виртуальный. Если виртуальный поток отцепится от носителя внутри `synchronized`, на этом носителе смонтируется другой виртуальный поток, и JVM сочтёт его владельцем того же монитора: взаимное исключение сломается. Поэтому размонтирование запрещено, пока монитор удерживается.
- `ReentrantLock`: обычный Java-объект на AQS, владелец хранится ссылкой на сам `Thread` (в том числе виртуальный), а ожидание идёт через `LockSupport.park`, который размонтирует виртуальный поток. Привязки к носителю нет.
- JEP 491 (JDK 24, в LTS это JDK 25) исправляет именно это: владельцем монитора становится виртуальный поток. Для этого переделали внутреннюю реализацию мониторов в JVM: в `ObjectMonitor` вместо указателя на платформенный поток записывается идентификатор потока (`Thread.tid`), а при размонтировании захваченные мониторы (lock stack носителя) копируются в stack chunk виртуального потока и восстанавливаются на следующем носителе. Работает только в режиме lightweight locking (он по умолчанию); устаревший legacy-режим не поддерживается.

Как обнаружить

```
-Djdk.tracePinnedThreads=full      # или short: печатает стек при pinning  
(JDK 21–23)
```

Вывод при блокировке под `synchronized` (проверено, режим `short`):

```
VirtualThread[#100025]/runnable@ForkJoinPool-1-worker-2 reason:MONITOR  
  V.lambda$main$3(V.java:23) <== monitors:1
```

- `reason:MONITOR` — причина закрепления, `<== monitors:1` — на каком кадре захвачен монитор;
- JFR-событие `jdk.VirtualThreadPinned` срабатывает, когда блокирующая операция заняла дольше порога (по умолчанию 20 мс);
- в JDK 24+ свойство `jdk.tracePinnedThreads` удалено, его установка ничего не делает; JFR-событие остаётся и теперь срабатывает только для нативных вызовов.

Нативные вызовы

Второй источник pinning — JNI и Foreign Function API.

- Нативный код может заснуть внутри `native`-метода: старые JNI-обёртки над сокетами и файлами, некоторые драйверы БД, устаревшие версии Netty.
- JVM не может размонтировать виртуальный поток посреди нативного кода: `native`-стек принадлежит конкретному носителю.
- Итог: виртуальный поток с нативным вызовом держит носитель всё время вызова. JEP 491 этого не меняет.

Что делать:

- обновить драйверы и библиотеки до версий, совместимых с виртуальными потоками (Netty, gRPC, драйверы БД выпускали такие обновления);
- не держать нативные вызовы внутри критических секций;
- искать pinning в нагрузочных тестах (JFR или `jdk.tracePinnedThreads` на JDK 21–23).

ThreadLocal на виртуальных потоках

`ThreadLocal` работает и у виртуальных потоков, но смысл меняется.

- Платформенный поток живёт долго и обслуживает много задач, поэтому кеширование в `ThreadLocal` окупается (классический пример — `SimpleDateFormat` в пуле).
- Виртуальный поток не переиспользуется: на каждую задачу свой. Копия `ThreadLocal` создаётся заново для каждого потока, а при миллионах потоков тяжёлые значения (буферы, кеши) складываются в большую память.

```
// ❌ кеш «на поток» для пула: для виртуальных потоков бессмыслен, каждый
поток создаёт свой экземпляр
static final ThreadLocal<SimpleDateFormat> FMT =
    ThreadLocal.withInitial(SimpleDateFormat::new);

// ✅ неизменяемый объект, один на всех
static final DateTimeFormatter FMT = DateTimeFormatter.ISO_DATE_TIME;
```

Первый вариант лечат заменой на потокобезопасный и неизменяемый объект. Для передачи контекста (пользователь, trace id) есть замена.

ScopedValue — замена для контекста

`ScopedValue` (JEP 506, финализирован в JDK 25) — неизменяемое значение, доступное внутри ограниченной области кода.

```
// было: ThreadLocal — изменяемый, живёт, пока не вызовешь remove()
static final ThreadLocal<Context> CTX_OLD = new ThreadLocal<>();

// стало: ScopedValue — привязано к области выполнения, освобождается само
static final ScopedValue<Context> CTX = ScopedValue.newInstance();

ScopedValue.where(CTX, context).run(() -> {
    // внутри этой области и во всём, что отсюда вызывается, CTX.get()
    // вернёт context
    handle();
}); // вышли из области: привязки
    // нет, remove() не нужен
```

Сравнение с `ThreadLocal`:

Свойство	<code>ThreadLocal</code>	<code>ScopedValue</code>
Изменяемость	<code>set</code>	неизменяемое, привязывается один раз
Время жизни	пока не <code>remove()</code>	только внутри области <code>run / call</code>

Свойство	ThreadLocal	ScopedValue
Передача дочерним потокам	копирование, дорого	без копирования (в потоки StructuredTaskScope)
Утечки	возможны	нет: привязка снимается при выходе

Для нового кода на JDK 25+ это рекомендуемая замена ThreadLocal для передачи контекста.

☰ Кейс Netflix: pinning в проде на JDK 21

Netflix описал в техблоге («Java 21 Virtual Threads — Dude, Where's My Lock?», 2024) зависания сервисов на Spring Boot 3 + Tomcat с виртуальными потоками.

- **Симптомы:** случайные таймауты, инстансы «живы», но не отвечают; растёт число сокетов в состоянии `closeWait`.
- **Причина:** виртуальный поток внутри `synchronized` ждал `ReentrantLock` (в Zipkin/Brave, очередь спанов) и закреплялся на носителе. Носителей было 4 (по числу vCPU), и все 4 оказались закреплены потоками, ждущими этот лок.
- **Взаимная блокировка:** поток, который должен был получить лок следующим, уже был «разбужен», но ему не хватало свободного носителя, чтобы продолжить. Владелец ждал носителя, носители ждали владельца.
- **Как нашли:** `jcmd <pid> Thread.dump_to_file` (обычный `jstack` виртуальные потоки не показывает полностью): тысячи «пустых» виртуальных потоков, по числу сокетов в `closeWait`; затем `heap dump` в Eclipse MAT.
- **Вывод:** pinning опасен не только падением производительности, но и настоящим дедлоком, когда все носители закреплены. JEP 491 (JDK 24) убирает закрепление на `synchronized` и этот класс проблем.

Production multithreading

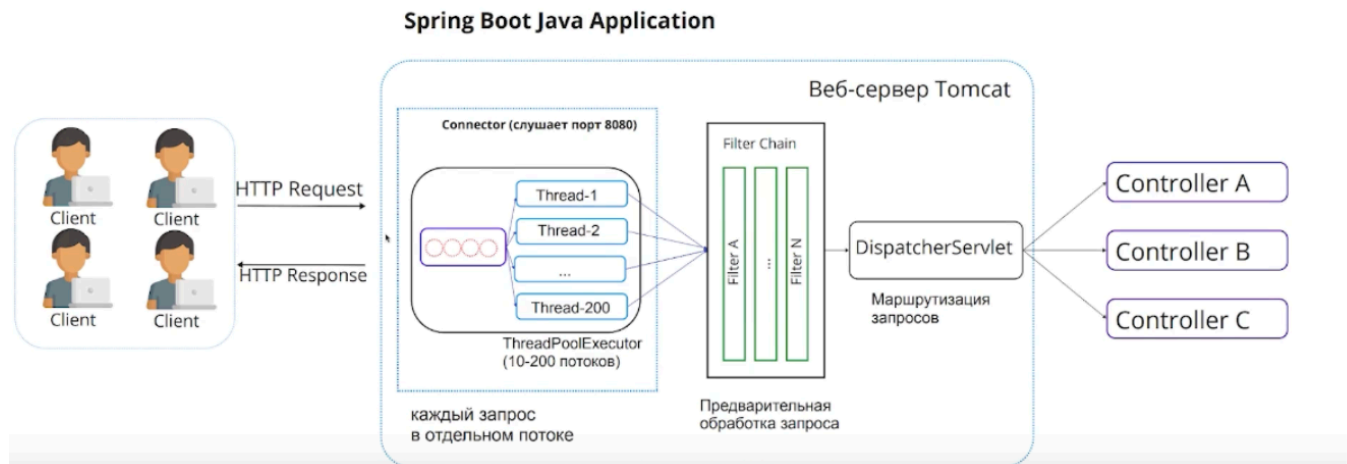
Модели обработки запросов в бэкенде: Thread-per-Request, Event Loop, Virtual Threads

Бэкенд — Java-приложение на сервере, основная работа которого — I/O: ходить в БД и другие сервисы, читать и писать по сети. Современные сервисы держат тысячи и десятки тысяч запросов в секунду, поэтому ключевой вопрос — как организовать потоки, пока те ждут ответов. Есть три модели: поток на запрос, event loop и виртуальные потоки.

Thread-per-Request (Tomcat)

Каждый входящий запрос обрабатывается отдельным потоком из пула. Это модель по умолчанию в Spring Boot (`spring-boot-starter-web` + встроенный Tomcat), код пишется обычный, блокирующий.

Thread-per-request: Tomcat



Лимиты Tomcat по умолчанию

Параметр	Значение	Что это
<code>maxThreads</code> (<code>server.tomcat.threads.max</code>)	200	потоков обработки запросов
<code>maxConnections</code> (<code>server.tomcat.max-connections</code>)	8192	сколько соединений Tomcat принимает одновременно (NIO)
<code>acceptCount</code> (<code>server.tomcat.accept-count</code>)	100	очередь входящих соединений в ОС, когда <code>maxConnections</code> исчерпан; при переполнении соединения отклоняются
<code>maxQueueSize</code> внутреннего executor	<code>Integer.MAX_VALUE</code>	очередь задач пула потоков: по умолчанию неограниченная

- Tomcat использует собственный `ThreadPoolExecutor` с модифицированной очередью: если свободных потоков нет и потоков меньше `maxThreads`, он создаёт новый поток, а не складывает задачу в очередь (в отличие от обычного `ThreadPoolExecutor` из блока про пулы)

- Потоки платформенные: стек порядка 1 МБ на поток, поэтому поднять тысячи потоков дорого.

Где возникает узкое место

- Поток почти всё время блокирован: ждёт БД или сеть (CPU меньше 1%, ожидание больше 99%).
- Заблокированный поток не может взять следующий запрос.
- Если запросов приходит больше, чем система обрабатывает, очередь растёт.

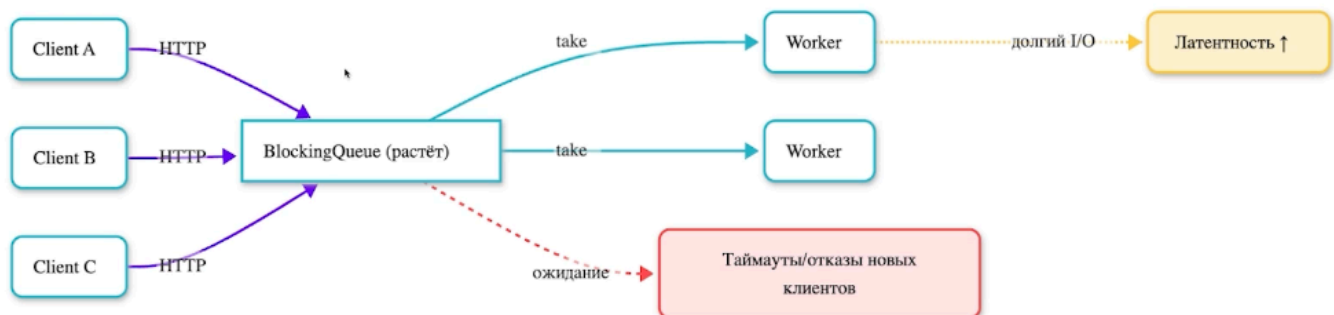
Оценка пропускной способности по закону Литтла: число одновременных запросов равно произведению пропускной способности на время обработки одного запроса, $L = \lambda \cdot W$. Отсюда потолок пула:

200 потоков, запрос занимает 100 мс → максимум $200 / 0.1 = 2000$ запросов/с
 200 потоков, запрос занимает 1 с → максимум 200 запросов/с

Если приходит больше, избыток накапливается:

Обработка: 50 запросов/с, приход: 60 запросов/с
 → +10 запросов в очереди каждую секунду
 → очередь неограниченно растёт → память забита → сервис падает

Последствия переполненной очереди



- Самые старые запросы отваливаются по таймауту: клиенты ждут 10–30 с и отключаются, а сервис продолжает обрабатывать уже ненужные запросы.
- Клиентам кажется, что сервис упал, хотя он жив.
- Защита: ограничить очередь (например, `acceptCount`, ограниченная очередь `executor`), чтобы лишние запросы отклонялись сразу.

Rate Limiter

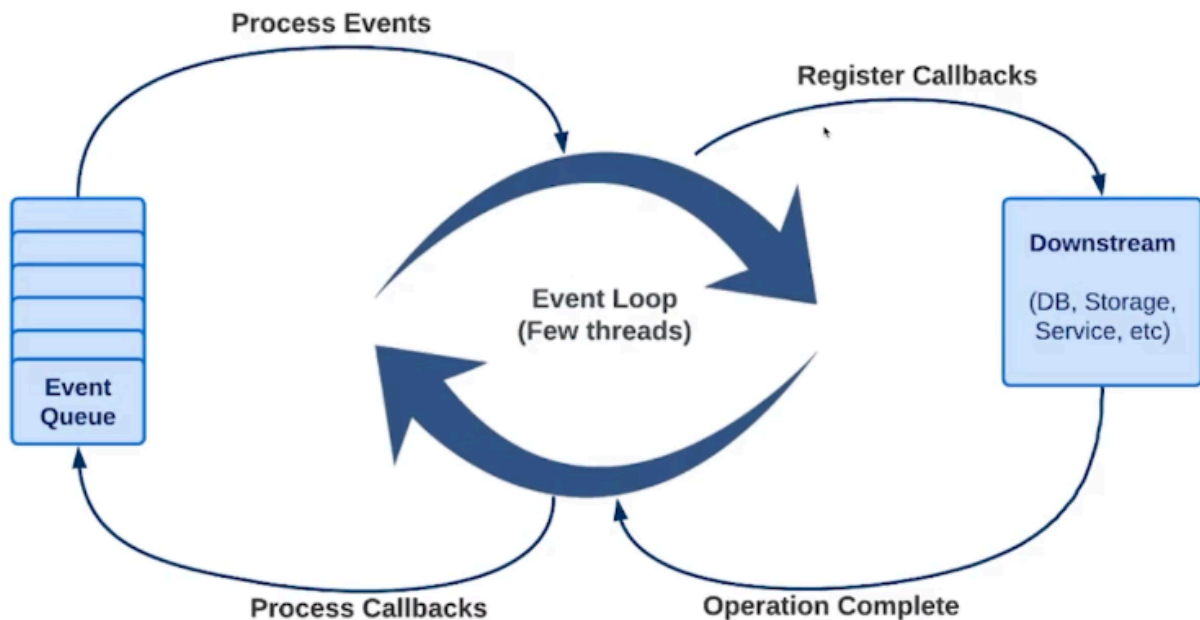
Rate limiter — ограничитель входящего потока: запросы сверх лимита отклоняются сразу и не попадают в систему.

Клиенты → [Rate Limiter: max 50 RPS] → Backend (очередь не растёт)

- Встроенный в сервис (например, Resilience4j `RateLimiter`).
- Внешний, перед сервисом: nginx, Traefik, API-шлюз.
- На стороне клиента: ненадёжен, клиент может слать запросы в обход.

Event Loop (Spring WebFlux)

Event loop — неблокирующая модель: небольшое фиксированное число потоков (порядка числа ядер) обслуживает множество соединений. Поток никогда не ждёт I/O: он регистрирует обратный вызов и переходит к следующей задаче.



Стек

- Netty — неблокирующий сетевой фреймворк на Java.
- Reactor Netty — сервер поверх Netty, по умолчанию используется WebFlux.
- Project Reactor — библиотека реактивных потоков (Mono , Flux).
- Spring WebFlux — реактивный веб-стек Spring на Reactor, по умолчанию работает на Reactor Netty.

Число worker-потоков Reactor Netty по умолчанию: число доступных процессоров, но не меньше 4 (`reactor.netty.ioWorkerCount`).

Принцип работы

1. Воркер берёт задачу и исполняет её
2. Задача упирается в I/O (БД, сеть)
3. Воркер НЕ ждёт: регистрирует в ОС уведомление «когда данные придут – сообщи»
4. Воркер берёт следующую задачу из очереди
5. Данные пришли: продолжение задачи ставится в очередь, и свободный воркер её дорабатывает

Плюсы и минусы

Плюсы	Минусы
Потоки не блокируются, CPU занят полезной работой	Другая парадигма: цепочки <code>Mono / Flux</code> , обратные вызовы
Достаточно нескольких потоков	Высокий порог входа, сложнее отладка и стек-трейсы
Хорош для I/O-heavy нагрузки	Не подходит для CPU-heavy задач

Блокирующий вызов в event loop

Модель держится на инварианте «воркер никогда не ждёт». Блокирующий вызов (JDBC, `Thread.sleep`, синхронный HTTP-клиент) занимает воркера целиком:

4 воркера, один заблокирован на 1 с → пропускная способность упала на 25% на это время
4 воркера, все заблокированы → сервис не принимает и не обрабатывает ничего

Получается худшее из двух моделей: потоков мало (как в event loop), а они ещё и ждут (как в thread-per-request). Блокирующий код выносят на отдельный планировщик:

```
Mono.fromCallable(() -> jdbcRepository.findById(id)) // блокирующий  
вызов  
    .subscribeOn(Schedulers.boundedElastic()); // не на event  
loop
```

Виртуальные потоки (Java 21+, Spring Boot 3.2+)

Архитектура остаётся thread-per-request, но каждый запрос обрабатывает виртуальный поток. Блокирующий код пишется как обычно, а ожидание I/O не занимает носитель. Подробнее об устройстве: файлы про Virtual Threads.

Как включить

```
spring:
  threads:
    virtual:
      enabled: true      # нужна Java 21+
```

Что включает флаг (по документации Spring Boot 3.2):

- Tomcat и Jetty обрабатывают запросы виртуальными потоками;
- `applicationTaskExecutor` (для `@Async`) становится `SimpleAsyncTaskExecutor` на виртуальных потоках;
- `taskScheduler` использует виртуальные потоки;
- исполнители для слушателей RabbitMQ и Kafka.

В самом Tomcat параметр `useVirtualThreads` по умолчанию `false`; Spring Boot включает его этим свойством.

Что меняется

- Лимит в 200 потоков больше не ограничивает число одновременных запросов: виртуальных потоков может быть десятки и сотни тысяч.
- Узкое место смещается к ресурсам ниже по стеку: пул соединений с БД, лимиты внешних API (следующий файл). Если нужно ограничить нагрузку на такой ресурс, ставят `Semaphore` или `bulkhead`; пулить виртуальные потоки не нужно.
- Код остаётся синхронным и привычным: `Thread.sleep`, `JDBC`, `RestClient`.

Оговорки

Оговорка	Суть
Не серебряная пуля	узкое место во внешней системе (например, 20 RPS у чужого API) никакими потоками не лечится
Pinning	JDK 21–23: блокировка внутри <code>synchronized</code> и нативные вызовы закрепляют носитель; с JDK 24 (JEP 491) остаются нативные вызовы
<code>ThreadLocal</code>	миллионы потоков × тяжёлые значения = лишняя память
CPU-bound	виртуальные потоки не ускоряют вычисления

Сравнение моделей

Модель	Потоки	Код	Когда брать
Thread-per-Request (Tomcat)	~200 платформенных, блокируются	обычный синхронный	по умолчанию; нагрузка умеренная, ресурсов хватает
Event Loop (WebFlux)	порядка числа ядер (не меньше 4), не блокируются	реактивный (Mono / Flux)	экстремальный I/O-heavy RPS, команда готова к реактивной парадигме
Virtual Threads	десятки и сотни тысяч, дешёвые	обычный синхронный	I/O-heavy нагрузка, хочется оставить привычный код (Java 21+, Boot 3.2+)

Event loop и виртуальные потоки решают одну задачу: не держать дорогой ресурс (поток) на время ожидания I/O. Event loop требует другой парадигмы, виртуальные потоки требуют внимания к pinning и внешним узким местам.

Закон Амдала и узкие места в продакшене

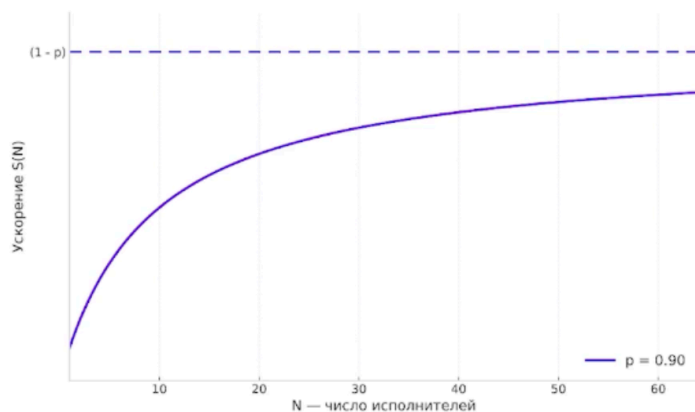
Рост числа потоков и ядер не даёт пропорционального роста пропускной способности: скорость ограничена последовательной частью программы и самым узким разделяемым ресурсом. Закон Амдала описывает первое, а практика (пул соединений, внешние API, блокировки) — второе.

Закон Амдала

Программа состоит из параллелизуемой части и последовательной (блокировки, ожидание ответов, общие ресурсы). Пусть P — доля параллелизуемой части, N — число процессоров. Ускорение:

$$S(N)=1(1-P)+PN$$

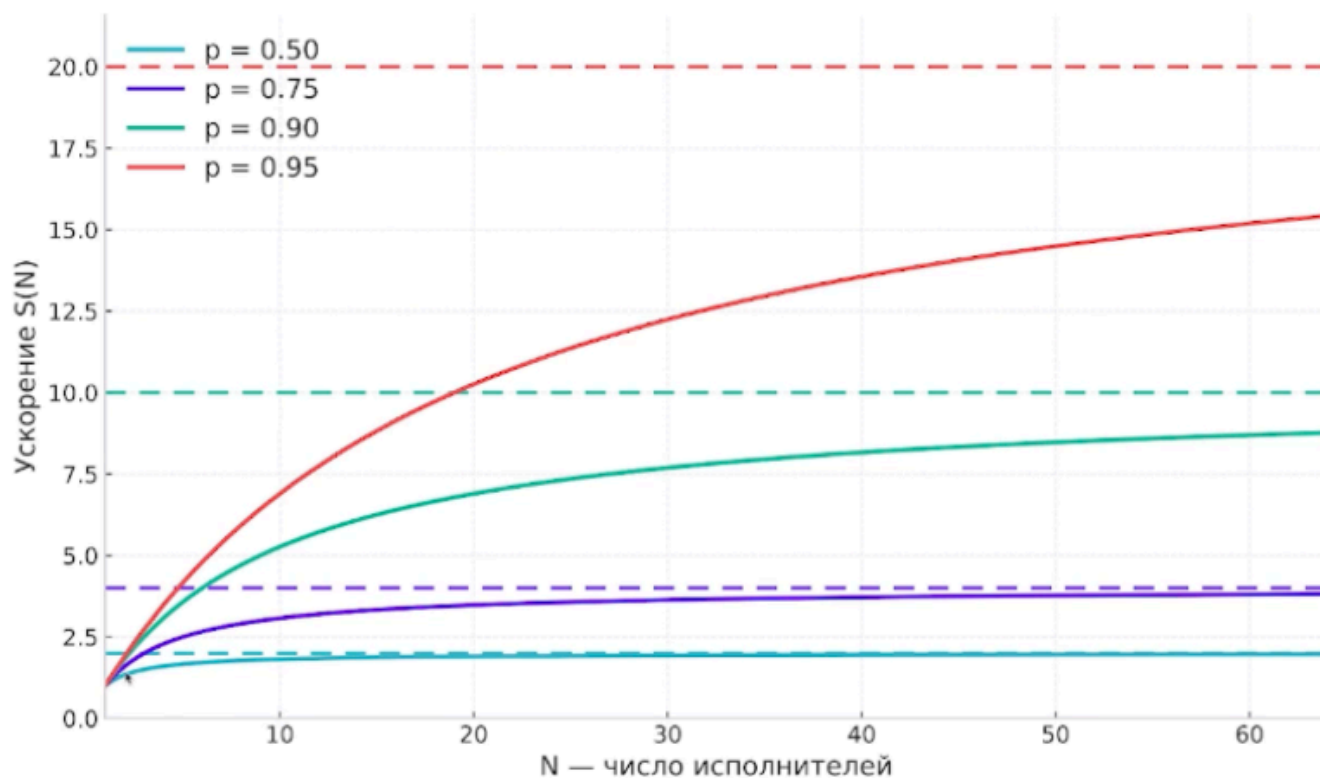
$$S = \frac{1}{(1 - P) + \frac{P}{N}} \quad \text{где} \quad \begin{cases} S & \text{— ускорение} \\ P & \text{— доля параллелизуемой части} \\ N & \text{— число процессоров} \end{cases}$$



Даже при ∞ количестве процессоров, но с $p = 90\%$ -> ускорение не больше чем в 10 раз

Предел ускорения при $N \rightarrow \infty$: $S = 1 / (1 - P)$.

Параллелизуемая доля P	Ускорение на 8 процессорах	Потолок при $N \rightarrow \infty$
0.50	1.8×	2×
0.90	4.7×	10×
0.95	5.9×	20×



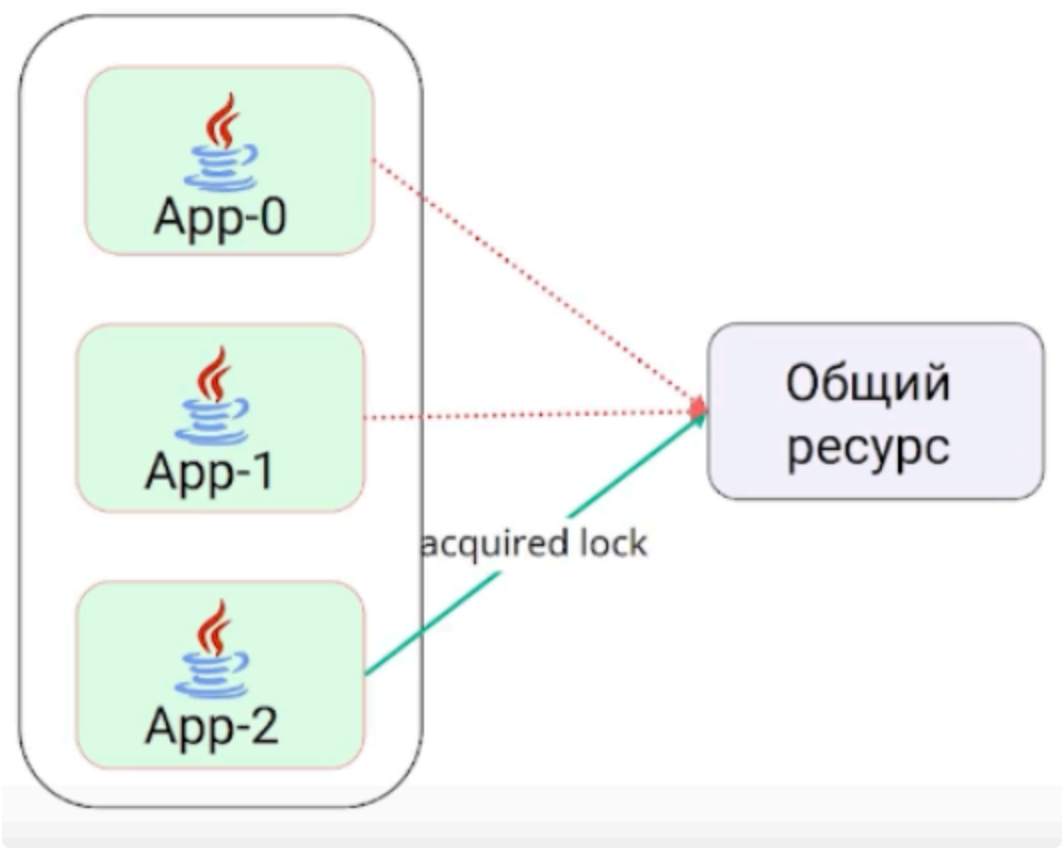
Почему P меньше 1 на практике: ожидание сети и БД, общие ресурсы (локи, пулы), последовательные участки кода.

Выводы

- Ядра, инстансы и реплики дают хороший рост, пока не упёрлись в узкое место.
- С какого-то момента дешевле убрать последовательную часть, чем добавлять ядра.
- Любая блокировка или критическая секция снижает потолок для всей системы: поэтому `synchronizedMap` на всю коллекцию убивает масштабируемость, `pinning` виртуальных потоков обнуляет выигрыш, а «просто добавить потоков» не работает.

Где чаще всего узкие места

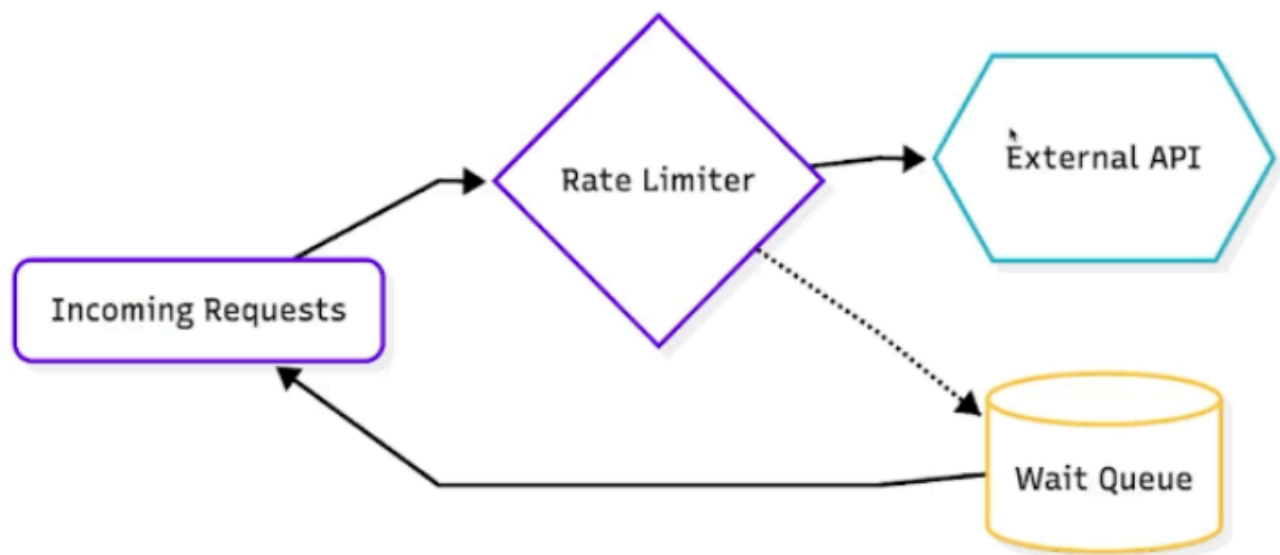
Любой разделяемый ресурс — потенциальное узкое место.



Узкое место	Что происходит
Пул соединений с БД	соединения дорого создавать, поэтому их переиспользуют; размер пула ограничивает число параллельных запросов к БД
Лимит внешнего API	если внешний сервис режет запросы, потоки и ядра не помогут; нужны очередь, ретрай, деградация, кеш и back-pressure

Узкое место	Что происходит
Блокировки в приложении	потoki держат лок на общем ресурсе, остальные ждут

Rate-Limiter как bottleneck



Ограничение сверху задаёт самое узкое звено:

$TPS \approx \min(\text{потoki приложения, пул соединений, пропускная способность БД, лимиты внешних API})$

Масштабировать нужно самое узкое звено, остальные при этом простаивают.

Пул соединений с БД как узкое место



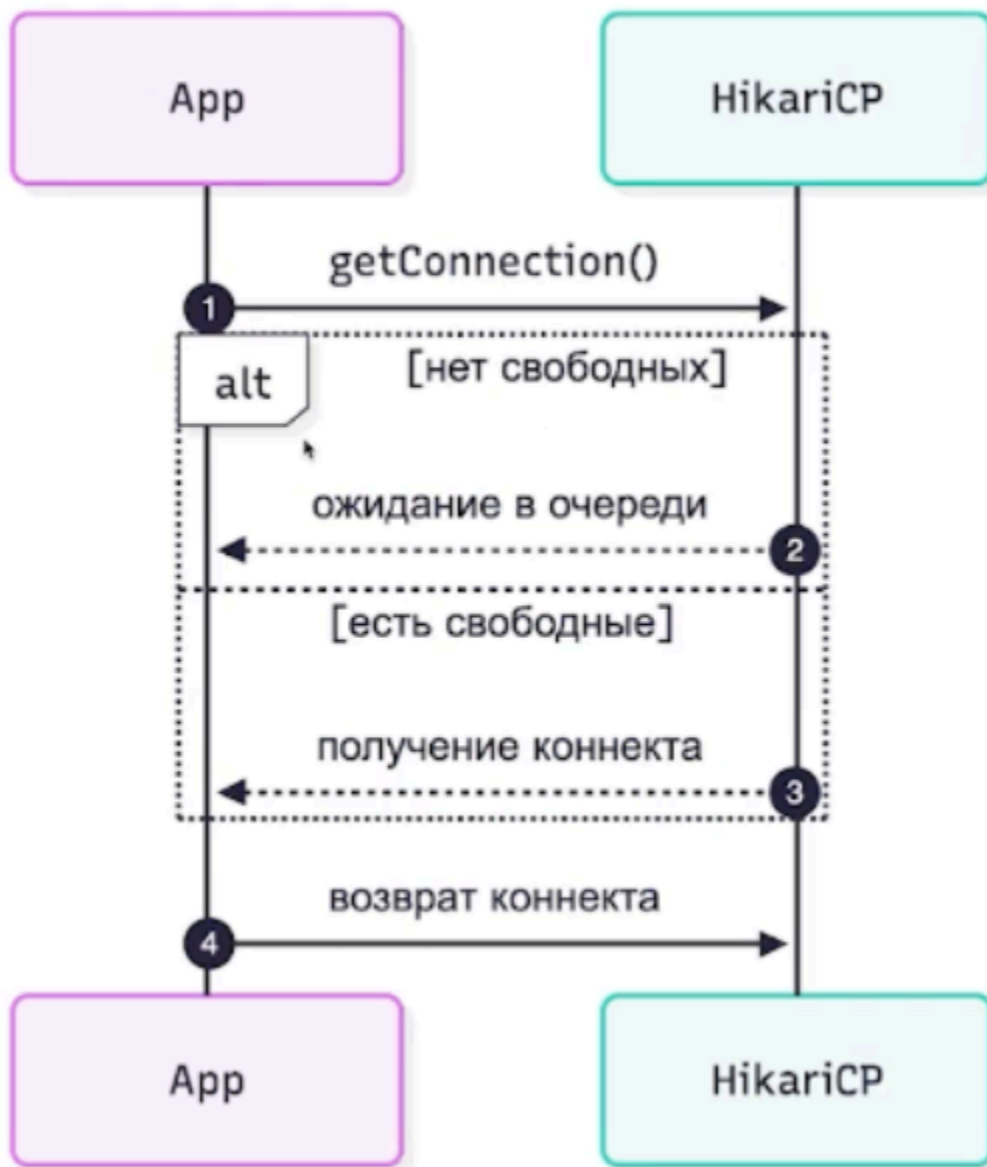
- Соединение с БД дорого создавать (у PostgreSQL на каждое соединение отдельный серверный процесс), поэтому соединения живут в пуле и переиспользуются.
- Размер пула — потолок параллельных запросов к БД: сколько бы потоков ни работало в приложении, одновременно в БД идёт не больше соединений, чем в пуле.
- Добавил 10 000 виртуальных потоков при пуле из 10 соединений: в БД по-прежнему 10 запросов, остальные ждут.

Потолок пропускной способности БД со стороны приложения по закону Литтла:

пул 10 соединений, запрос 20 мс → максимум $10 / 0.02 = 500$ запросов/с
пул 10 соединений, запрос 200 мс → максимум 50 запросов/с

HikariCP

HikariCP — пул соединений по умолчанию в Spring Boot.



Параметр	Значение по умолчанию	Что это
<code>maximumPoolSize</code> (<code>spring.datasource.hikari.maximum-pool-size</code>)	10	максимум соединений в пуле
<code>connectionTimeout</code> (<code>spring.datasource.hikari.connection-timeout</code>)	30 000 мс	сколько <code>getConnection()</code> ждёт свободное соединение, затем <code>SQLException</code>

- Если все соединения заняты, `getConnection()` блокирует поток до освобождения или до таймаута.

- Долгие или зависшие запросы удерживают соединения и могут положить весь сервис: пул общий, и очередь ожидающих растёт.
- Размер пула берут небольшим. Отправная точка из вики HikariCP (формула из PostgreSQL): `connections = (core_count × 2) + effective_spindle_count`, дальше подбирают нагрузочным тестом. Большой пул не ускоряет, а перегружает БД.

```
spring:
  datasource:
    hikari:
      maximum-pool-size: 10      # потолок параллельных запросов к БД
      connection-timeout: 3000  # 3 с вместо 30: лучше быстро упасть, чем
                                # держать потоки
```

Практические советы

- Сокращать блокирующие секции и синхронизацию: меньше последовательной части, выше потолок.
- Изолировать медленные ресурсы: вызовы внешних API в отдельные пулы или с ограничением параллелизма, чтобы они не забивали общие потоки.
- У каждого I/O должен быть таймаут и план Б: заглушка, кеш, устаревшие данные.
- Не ждать упавший сервис: circuit breaker переключает вызовы на быстрый отказ, когда ошибок много, вместо того чтобы держать потоки на таймаутах.

```
// ограничение параллелизма к внешнему API (bulkhead) + таймаут ожидания
Semaphore externalApi = new Semaphore(20);           // не больше 20
одновременных вызовов

if (!externalApi.tryAcquire(200, TimeUnit.MILLISECONDS)) { // не ждём
    бесконечно
    return fallback();                                     // быстрый отказ
}
try {
    return client.call();                                 // у клиента
    свой таймаут
} finally {
    externalApi.release();
}
```

Состояния circuit breaker:

Состояние	Что происходит
Closed	запросы идут, ошибки считаются
Open	доля ошибок превысила порог: запросы сразу отклоняются, ресурс не нагружается
Half-open	после паузы пропускается несколько пробных запросов: успех возвращает в Closed, ошибка в Open

Thread Dump: диагностика многопоточных проблем

Thread dump — снимок стеков всех потоков JVM в конкретный момент: у каждого потока видно состояние, текущий стек вызовов, какие мониторы он держит и какого ждёт. Это основной инструмент, когда сервис завис, нагрузка на CPU аномальна или упала пропускная способность.

Как снять дамп

```
jps                                # список Java-процессов: PID и имя
# класса
jstack <pid>                       # дамп в stdout
jstack -l <pid>                    # + Locked ownable synchronizers (локи
из java.util.concurrent)
jcmd <pid> Thread.print             # то же, что jstack, через jcmd
kill -3 <pid>                      # SIGQUIT: JVM печатает дамп в свой
stdout (процесс не завершается)
```

- `jstack` и `jcmd` подключаются к работающей JVM, перезапуск не нужен.
- Одного дампа мало: снимают 3–5 штук с интервалом 5–10 с и сравнивают. Поток, который во всех дампах стоит на одной и той же строке, и есть подозреваемый.
- Из кода: `Thread.getAllStackTraces()` и `ThreadMXBean` (`ManagementFactory.getThreadMXBean()`).

Виртуальные потоки

`jstack` и `jcmd Thread.print` виртуальные потоки не показывают (проверено на JDK 21: в дампе три созданных виртуальных потока отсутствуют). Для них есть отдельная команда:

```
jcmt <pid> Thread.dump_to_file /tmp/dump.txt # текстовый
формат
jcmt <pid> Thread.dump_to_file -format=json /tmp/dump.json # JSON,
потоки сгруппированы по контейнерам
```

Фрагмент вывода (проверено, JDK 21):

```
#22 "vt-0" virtual
    java.base/java.lang.VirtualThread.parkNanos(VirtualThread.java:635)
    java.base/java.lang.VirtualThread.sleepNanos(VirtualThread.java:807)
    java.base/java.lang.Thread.sleep(Thread.java:507)
    TD.sleep(TD.java:20)
```

В JSON потоки сгруппированы по контейнерам (корневой контейнер, пулы вроде ForkJoinPool-1, ThreadPoolExecutor), так видно, каким пулом и сервисом создан поток.

Как читать дамп

Запись одного потока (реальный вывод, JDK 21):

```
"worker-1" #18 [548] prio=5 os_prio=0 cpu=1.84ms elapsed=10.71s
tid=0x00007f96f8134310 nid=548 waiting for monitor entry
    java.lang.Thread.State: BLOCKED (on object monitor)
        at TD.lambda$main$0(TD.java:6)
        - waiting to lock <0x00000000a851a298> (a java.lang.Object)
        - locked <0x00000000a851a288> (a java.lang.Object)
        at java.lang.Thread.run(java.base@21.0.10/Thread.java:1583)
```

- "worker-1" — имя потока (поэтому потокам в пулах дают осмысленные имена через ThreadFactory);
- #18 — id потока в JVM, nid — id потока в ОС (в JDK 21 выводится десятичным, в старых версиях шестнадцатеричным);
- cpu, elapsed — CPU-время потока и время жизни;
- Thread.State — состояние потока;
- waiting to lock <адрес> — монитор, которого поток ждёт;
- locked <адрес> — монитор, который поток держит.

Состояния потоков

Состояние	Что значит в дампе	Типичная причина
RUNNABLE	выполняется или готов к выполнению (в том числе внутри нативного I/O)	работа или чтение из сокета
BLOCKED (on object monitor)	ждёт вход в synchronized	конкуренция за монитор
WAITING (parking)	LockSupport.park : ждёт без таймаута	ReentrantLock , Condition.await , LinkedBlockingQueue.take
WAITING (on object monitor)	Object.wait()	wait / notify
TIMED_WAITING (sleeping)	Thread.sleep	сон
TIMED_WAITING (parking)	park с таймаутом	poll(timeout) , await(timeout) , пул без задач

Запись ожидающего потока из очереди (реальный вывод) — WAITING (parking) на LinkedBlockingQueue.take :

```
"waiter" #20 [550] prio=5 ... waiting on condition
  java.lang.Thread.State: WAITING (parking)
    at jdk.internal.misc.Unsafe.park(Native Method)
      - parking to wait for  <0x00000000a85b8928> (a
java.util.concurrent.locks.AbstractQueuedSynchronizer$ConditionObject)
    at java.util.concurrent.locks.LockSupport.park(...)
    at
java.util.concurrent.LinkedBlockingQueue.take(LinkedBlockingQueue.java:435)
    at TD.lambda$main$2(TD.java:9)
```

Так в дампе выглядят простаивающие воркеры пула: у них take на очереди задач.

Deadlock в дампе

JVM сама ищет взаимные блокировки и выводит их отдельным разделом в конце дампа. Пример на двух потоках, берущих два монитора в разном порядке (реальный вывод):

```
Found one Java-level deadlock:
=====
"worker-1":
```

```
waiting to lock monitor 0x00007f9664058490 (object 0x00000000a851a298, a
java.lang.Object),
which is held by "worker-2"
```

```
"worker-2":
```

```
waiting to lock monitor 0x00007f96600015b0 (object 0x00000000a851a288, a
java.lang.Object),
which is held by "worker-1"
```

```
Found 1 deadlock.
```

- Каждый поток ждёт монитор, которым владеет другой: цикл worker-1 → worker-2 → worker-1.
- Ниже идёт раздел Java stack information for the threads listed above со стеками этих потоков и строками, на каком месте кода они застряли.
- Для ReentrantLock раздел тоже появляется (проверено на JDK 21), но вместо monitor в нём ownable synchronizer:

```
"rl-1":
```

```
waiting for ownable synchronizer 0x00000000a851a018, (a
java.util.concurrent.locks.ReentrantLock$NonfairSync),
which is held by "rl-2"
```

- Флаг -l добавляет в записи потоков раздел Locked ownable synchronizers: какие ReentrantLock и подобные локи держит каждый поток. Для диагностики зависаний на java.util.concurrent его стоит использовать.

Из кода

```
ThreadMXBean mx = ManagementFactory.getThreadMXBean();
long[] ids = mx.findDeadlockedThreads();           // мониторы и ownable
synchronizers (ReentrantLock)
if (ids != null) {
    for (ThreadInfo ti : mx.getThreadInfo(ids)) {
        System.out.println(ti.getThreadName() + " " + ti.getThreadState()
            + " lock=" + ti.getLockName() + " owner=" +
ti.getLockOwnerName());
    }
}
// worker-1 BLOCKED lock=java.lang.Object@... owner=worker-2
```

findMonitorDeadlockedThreads() ищет только взаимные блокировки на мониторах: для ReentrantLock он вернёт null (проверено), поэтому берут findDeadlockedThreads().

Что искать в дампе

Симптом	Что видно в дампе
Сервис завис, CPU низкий	много потоков <code>BLOCKED</code> на одном мониторе, или <code>deadlock</code> в разделе <code>Found</code> ; смотрят, кто держит лок (<code>locked <тот же адрес></code>)
Сервис завис, потоки ждут БД	десятки потоков в <code>HikariPool.getConnection</code> (<code>WAITING / TIMED_WAITING</code> на пуле соединений): пул исчерпан, см. файл про узкие места
CPU 100%	сопоставить <code>nid</code> с потоком из <code>top -H -p <pid></code> , найти его стек: цикл, регулярка, тяжёлое вычисление
Пул «не работает»	все потоки пула <code>WAITING</code> на <code>LinkedBlockingQueue.take</code> : задач нет, дело в источнике; или все <code>RUNNABLE / BLOCKED</code> на медленном ресурсе: пул исчерпан
Утечка потоков	растёт число потоков с одним шаблоном имени (<code>pool-N-thread-M</code>): пул создаётся на каждый запрос
Подозрение на pinning	стек виртуального потока под <code>synchronized</code> ; ищут через <code>jcmd Thread.dump_to_file</code> и JFR-событие <code>jdk.VirtualThreadPinned</code>

Практические правила:

- Давать потокам и пулам осмысленные имена (`ThreadFactory`), иначе дамп читается как `pool-3-thread-12`.
- Снимать несколько дампов подряд и сравнивать.
- Разбирать от конца: сначала раздел `Found ... deadlock`, затем потоки в `BLOCKED`, затем потоки, у которых стек совпадает во всех дампах.

Вопросы к собеседованию

Вопросы по многопоточности из подборки для собеседований. Под ответом, если тема уже разобрана в блоках выше, стоит ссылка на раздел конспекта.

Теория и термины

Расскажите о модели памяти Java?

Модель памяти Java (JMM) — часть спецификации языка: правила, по которым потоки взаимодействуют через общую память. Она определяет, когда запись одного потока гарантированно видна другому и какие перестановки инструкций допустимы.

Основа — отношение happens-before: если действие A happens-before B, то результат A виден в B и порядок A перед B соблюден. Оно транзитивно и связывает потоки только через один и тот же монитор или одну и ту же `volatile`-переменную. Основные рёбра: `unlock` монитора → последующий `lock` того же монитора; запись `volatile` → чтение той же переменной; `Thread.start()` → начало потока; завершение потока → возврат из `join()`; порядок операций внутри одного потока.

Отдельно: `final`-поля корректно построенного объекта видны другим потокам без синхронизации (safe publication).

Что JMM решает на практике: видимость (поток может не увидеть чужую запись из-за регистров и кэшей), reordering (компилятор, JIT и процессор переставляют инструкции) и атомарность (`count++` не атомарна). Без happens-before никаких гарантий нет.

Подробнее: [Java Memory Model: happens-before](#), [Основные рёбра happens-before](#).

Что такое « потокобезопасность »?

Потокобезопасность — свойство класса (кода), при котором он ведёт себя корректно при обращении из нескольких потоков независимо от того, как планировщик их чередует, и без дополнительной синхронизации со стороны вызывающего кода. Инварианты сохраняются, записи видны другим потокам, операции атомарны. Например, потокобезопасный счётчик не потеряет ни одного приращения, даже если одним экземпляром пользуются несколько потоков.

В чём разница между «конкуренцией» и «параллелизмом»?

- Конкурентность (concurrency) — свойство структуры программы: несколько задач одновременно находятся в процессе выполнения и чередуются. Для этого не нужно несколько ядер или потоков: чередовать задачи можно и на одном потоке (event loop, корутины в Kotlin, `async`-код).
- Параллелизм (parallelism) — одновременное выполнение задач на нескольких ядрах или процессорах.

Конкурентность — про организацию работы с несколькими задачами, параллелизм — про их одновременное исполнение. Программа может быть конкурентной без параллелизма (один поток чередует задачи) и параллельной без сложной конкурентной структуры (например, умножение элементов массива на число по частям на разных ядрах: результат не зависит от порядка).

Подробнее: [Ядра CPU: Конкурентность vs Параллелизм](#).

Что такое «кооперативная многозадачность»? Какой тип многозадачности использует Java? Чем обусловлен этот выбор?

Кооперативная многозадачность — способ деления процессорного времени между потоками, при котором каждый поток обязан отдавать управление следующему добровольно.

- Преимущества: простота реализации, меньшие накладные расходы на переключение контекста.
- Недостатки: если один поток завис или ведёт себя некорректно, зависает целиком вся система, и другие потоки никогда не получают управление.

Java использует **вытесняющую многозадачность**, при которой решение о переключении между потоками процесса принимает операционная система.

Что даёт вытесняющая модель:

- В отличие от кооперативной, управление передаётся операционной системе вне зависимости от состояния работающих приложений, поэтому отдельные зависшие потоки, как правило, не «подвешивают» всю систему целиком.
- За счёт регулярного переключения между задачами улучшается отзывчивость приложения и повышается оперативность освобождения ресурсов, которые больше не используются.
- В реализации вытесняющая многозадачность отличается от кооперативной тем, что требует обработки системного прерывания от аппаратного таймера.

Виртуальные потоки (JDK 21+) планирует уже JVM, и по времени они не вытесняются: виртуальный поток освобождает носитель только на блокирующей операции.

Бесконечный цикл без блокировок занимает носитель навсегда (проверено на JDK 21 с одним носителем: второй виртуальный поток не запустился за 2 с).

Что такое *ordering*, *as-if-serial semantics*, *sequential consistency*, *visibility*, *atomicity*, *happens-before*, *mutual exclusion*, *safe publication*?

- **ordering** — порядок операций, который наблюдают разные потоки. Для производительности компилятор и процессор могут переупорядочивать инструкции и выполнять их в произвольном порядке, пока внутри самого потока это незаметно. Эта гарантия называется **as-if-serial semantics**.
- **as-if-serial semantics** — в пределах одного потока результат выполнения такой, как если бы все операции шли строго последовательно, в порядке программы. На другие

потоки гарантия не распространяется: они могут наблюдать другой порядок.

- **sequential consistency** — результат выполнения такой, как будто все операции всех потоков выполнялись в одном общем порядке, согласованном с порядком операций внутри каждого потока. JMM гарантирует её только для программ без гонок данных (все обращения к общим данным упорядочены через happens-before). Это не то же самое, что as-if-serial: тот работает в рамках одного потока.
- **visibility** — определяет, когда действия в одном потоке становятся видны из другого потока.
- **happens-before** — логическое ограничение на порядок выполнения инструкций программы. Если запись в переменную и последующее её чтение связаны через эту зависимость, то как бы при выполнении ни переупорядочивались инструкции, в момент чтения все связанные с записью результаты уже зафиксированы и видны.
- **atomicity** — атомарность операций. Атомарная операция выглядит для других потоков единой и неделимой: она либо уже выполнена целиком, либо ещё не выполнена. Это не обязательно одна команда процессора (`synchronized` -блок атомарен, хотя команд в нём много). `count++` атомарной не является.
- **mutual exclusion** (взаимное исключение) — механизм, гарантирующий потоку исключительный доступ к ресурсу и не допускающий одновременного доступа к общему ресурсу. В каждый момент времени таким ресурсом может владеть только один поток. Простейший пример: `synchronized (obj) { ... }`.
- **safe publication** — показ объектов другим потокам из текущего без нарушения ограничений visibility. Способы такой публикации в Java:
 - `static {}` инициализатор;
 - `volatile` переменные;
 - `atomic` переменные;
 - сохранение в разделяемой переменной, корректно защищённой с помощью `synchronized`, синхронизаторов или других конструкций, создающих read/write memory barrier;
 - `final` переменные в разделяемом объекте, который был корректно проинициализирован.

Подробнее: [Основные рёбра happens-before](#), [«Без happens-before: reordering и broken publication»](#), [final-поля — safe publication без volatile](#).

Чем отличается процесс от потока?

- **Процесс** — экземпляр программы во время выполнения, независимый объект, которому ОС выделила ресурсы (процессорное время, память). Каждый процесс работает в своём адресном пространстве: он не может обратиться к переменным и

структурам данных другого процесса. Для обмена данными нужно межпроцессное взаимодействие (каналы, файлы, сокеты).

- **Поток (thread)** — единица исполнения внутри процесса: последовательность выполнения кода. Потоки процесса делят его память и открытые ресурсы (файлы, сокеты), но у каждого потока свой стек вызовов и счётчик команд. Создание и переключение потоков заметно дешевле, чем процессов, поэтому для дополнительной работы выгоднее создавать потоки. Платой за общую память становятся гонки данных, а сбой в одном потоке может завершить весь процесс.

Подробнее: [Процесс и поток](#).

Что такое «зелёные потоки» и есть ли они в Java?

Зелёные потоки (green threads) — потоки, которые эмулирует сама виртуальная машина: создание зелёного потока не создаёт поток ОС. JVM переключает их между собой, работая как один поток ОС.

- Плюсы: потоки ОС дороги, а зелёных можно создать намного больше, и переключение между ними быстрее (например, поток на каждое HTTP-соединение).
- Минус: все они выполняются на одном потоке ОС, поэтому не используют несколько ядер и два потока одновременно не работают.

В Java зелёные потоки были в ранних версиях, начиная с версии 1.2 по умолчанию используются native threads (потоки ОС).

Похожая идея вернулась в Java 21 в виде виртуальных потоков (Project Loom). Отличие: JVM планирует их на пуле потоков-носителей (`ForkJoinPool`), поэтому они используют несколько ядер.

Подробнее: [Виды потоков в Java](#).

Чем виртуальный поток отличается от платформенного и как его создать?

- **Платформенный поток**: отображается 1:1 на поток ОС, стек порядка 1 МБ в native-памяти, планирует ОС.
- **Виртуальный поток**: тот же `java.lang.Thread`, но его планирует JVM на небольшом наборе потоков-носителей (**carrier**, `ForkJoinPool` с параллелизмом по умолчанию, равным числу процессоров). Стек лежит в куче (continuation) и занимает килобайты.
- На блокирующей операции (IO, `sleep`, `ReentrantLock`, `park`) виртуальный поток снимается с носителя (**unmount**), и носитель берёт другой поток.
- Виртуальные потоки всегда daemon, а приоритет у них не меняется.

```
Thread.ofVirtual().name("vt-1").start(task);
Thread.startVirtualThread(task);

try (ExecutorService ex = Executors.newVirtualThreadPerTaskExecutor()) {
    // новый поток на каждую задачу
    ex.submit(task);
}
// close() ждёт завершения задач
```

Подробнее: [Virtual Threads \(Project Loom\): устройство и применение](#).

Что такое pinning виртуального потока?

Pinning — состояние, когда виртуальный поток при блокировке не снимается с носителя и держит его занятым.

- **JDK 21–23:** блокировка внутри `synchronized` (включая `wait()`) и нативные вызовы (JNI).
- **JDK 24+:** `synchronized` больше не закрепляет (JEP 491), остаются нативные вызовы.
- Если все носители закреплены, остальные виртуальные потоки ждут: масштабируемость пропадает.
- **Что делать:** на JDK 21–23 вокруг блокирующих операций брать `ReentrantLock` вместо `synchronized`. Искать pinning через JFR-событие `jdk.VirtualThreadPinned`.

Подробнее: [Проблемы Virtual Threads: pinning, нативные вызовы, ThreadLocal](#).

Когда виртуальные потоки не помогают? Почему их не пулят?

- **Не помогают:**
 - CPU-bound задачи: они дают масштаб (throughput), а не скорость.
 - Приложение, где не бывает порядка 10 000 одновременных задач.
 - Узкое место ниже по стеку (пул соединений БД, лимиты внешнего API).
- **Не пулят:** создание виртуального потока дёшево, поэтому правило такое: один поток на одну задачу. Для ограничения нагрузки на ресурс берут `Semaphore`, а не размер пула.
- **Память:** `ThreadLocal` с тяжёлыми значениями на сотнях тысяч потоков съедает память.

Подробнее: [Главные правила](#).

Каким образом можно создать поток?

- Создать потомка класса `Thread` и переопределить его метод `run()`.

- Передать в конструктор `Thread` экземпляр `Runnable` (в том числе лямбду): `new Thread(() -> ...)`. Поток завершится, когда завершится `run()`.
- Вызвать `submit()` у `ExecutorService`, передав `Runnable` или `Callable` (метод `call()` возвращает результат).
- Создать виртуальный поток (Java 21+): `Thread.ofVirtual().start(task)` или `Executors.newVirtualThreadPerTaskExecutor()`.

Подробнее: [Создание и запуск потоков в коде](#).

Чем различаются `Thread` и `Runnable` ?

- `Thread` — класс, объект которого представляет поток исполнения (для платформенного потока это обёртка над потоком ОС).
- `Runnable` — интерфейс, абстракция над выполняемой задачей с единственным методом `run()`.

Почему на практике берут `Runnable` :

- Класс может наследоваться только от одного класса, поэтому если он уже наследует другой, он не может наследовать `Thread`, а `Runnable` реализовать может.
- Логика задачи отделена от управления потоком: одну и ту же задачу можно запустить в новом потоке, в пуле потоков, в `CompletableFuture.runAsync` или в виртуальном потоке.

Подробнее: [Thread vs Runnable — в чём разница?](#).

В чём заключается разница между методами `start()` и `run()` ?

- `start()` создаёт новый поток и вызывает `run()` уже в нём.
- Прямой вызов `run()` — обычный вызов метода: он выполняется в текущем потоке, новый поток не запускается.
- Повторный `start()` на том же объекте `Thread` бросает `IllegalThreadStateException`, а `run()` можно вызывать сколько угодно раз.

Подробнее: [start\(\) vs run\(\) — в чём разница?](#).

Как принудительно запустить поток?

Никак. В Java нет способа принудительного запуска потока: этим управляет JVM, и API для этого не предоставляется. `start()` лишь переводит поток в состояние `RUNNABLE`, а когда он реально получит процессор, решает планировщик.

Что такое «монитор» в Java?

Монитор (mutex + очередь ожидания) — средство контроля доступа к ресурсу: у него не больше одного владельца в каждый момент времени. Если один поток захватил монитор, остальные, желающие использовать тот же ресурс, ждут его освобождения.

- В Java монитор есть у каждого объекта, им управляет виртуальная машина.
- Нестатический `synchronized`-метод при вызове захватывает монитор объекта `this`, `synchronized (obj)` — монитор `obj`, статический метод — монитор объекта `Class`. Если монитор занят, поток блокируется (`BLOCKED`) и ждёт.
- Монитор реентерабельный: владелец может захватить его повторно.
- Помимо взаимного исключения, у монитора есть очередь ожидания для `wait()` / `notify()`.
- Состояние монитора хранится в заголовке объекта (Mark Word).

Подробнее: [Монитор объекта, `synchronized` — взаимное исключение и устройство монитора](#).

Дайте определение понятию «синхронизация».

Синхронизация — координация потоков при доступе к общим данным. Она даёт три вещи: взаимное исключение (в критической секции один поток), видимость изменений (то, что записал один поток, увидит другой) и упорядоченность операций (`happens-before`).

В Java основной механизм — `synchronized` на мониторе объекта: поток, владеющий монитором, не пускает в защищённый код другие потоки; они переходят в состояние `BLOCKED`, пока монитор не освободится. Монитор освобождается при выходе из `synchronized`-блока или метода.

Подробнее: [`synchronized` — взаимное исключение и устройство монитора](#).

Какие существуют способы синхронизации в Java?

- `synchronized` — взаимное исключение на мониторе объекта (блок или метод).
- `volatile` — видимость и порядок для одной переменной; составные операции (`count++`) не атомарны.
- **Явные блокировки** (`ReentrantLock`, `ReadWriteLock`, `StampedLock`) — то же взаимное исключение с `tryLock`, таймаутами и несколькими условиями (`Condition`).
- **Атомики и CAS** (`AtomicInteger`, `AtomicReference`, `LongAdder`) — lock-free атомарные операции.
- **Синхронизаторы** (`CountDownLatch`, `CyclicBarrier`, `Semaphore`) — координация потоков по счётчику.
- `wait()` / `notify()` — ожидание условия внутри `synchronized` (у `Lock` то же делает `Condition`).

- `join()` — дождаться завершения другого потока.
- **Конкурентные коллекции** (`ConcurrentHashMap` , `BlockingQueue`) — безопасный обмен данными между потоками.
- **Без общего состояния** — неизменяемые объекты, `ThreadLocal` , локальные переменные.

Подробнее: [`synchronized` — взаимное исключение и устройство монитора](#), [`volatile` — барьер памяти, не атомарность](#), [`ReentrantLock` vs `synchronized`](#), [`ReadWriteLock`](#), [`StampedLock`](#), [CAS и CAS-loop — как работает lock-free, проблема ABA](#).

В каких состояниях может находиться поток?

Состояния описаны в `Thread.State` :

Состояние	Когда
NEW	объект <code>Thread</code> создан, <code>start()</code> не вызван (поток ещё не считается живым)
RUNNABLE	выполняется на ядре или ждёт своей очереди у планировщика
BLOCKED	ждёт монитор для входа в <code>synchronized</code>
WAITING	ждёт без таймаута: <code>wait()</code> , <code>join()</code> , <code>LockSupport.park()</code>
TIMED_WAITING	ждёт с таймаутом: <code>sleep(ms)</code> , <code>wait(ms)</code> , <code>join(ms)</code>
TERMINATED	<code>run()</code> завершился; из этого состояния назад пути нет, даже повторный <code>start()</code> не поможет

Подробнее: [Жизненный цикл — `Thread.State`](#), [Главное отличие: BLOCKED vs WAITING](#).

Можно ли создавать новые экземпляры класса, пока выполняется `static synchronized` метод?

Да, можно: `static synchronized` метод захватывает монитор объекта `Class` (`Counter.class`), а не блокирует создание экземпляров. Статические поля принадлежат классу, а не экземплярам, поэтому создание объектов этим методом не ограничивается. Другие `static synchronized` методы того же класса при этом ждут.

Подробнее: [Что даёт `synchronized`](#).

Зачем может быть нужен `private` мьютекс?

Объект для синхронизации делают `private` , чтобы сторонний код не мог на него синхронизироваться и случайно вызвать deadlock (взаимную блокировку).

```
class Counter {
    private final Object lock = new Object();    // снаружи не виден
    private int value;

    void inc() { synchronized (lock) { value++; } }
}
// synchronized (this) или synchronized-метод берут монитор this: он
// публичный,
// и внешний код может сам взять synchronized (counter) и держать его
// сколько хочет
```

Подробнее: [Что даёт `synchronized`](#).

Как работают методы `wait()` и `notify()` / `notifyAll()` ?

Методы определены в классе `Object` и служат для взаимодействия потоков при синхронизации.

- `wait()` — освобождает монитор и переводит поток в ожидание, пока другой поток не вызовет `notify()` / `notifyAll()`.
- `notify()` — будит один из ожидающих потоков (какой именно, заранее неизвестно). Проснувшись, поток сначала заново захватывает монитор, и только потом продолжает работу.
- `notifyAll()` — будит все ожидающие потоки, они по очереди борются за монитор.

Вызывать их можно только из `synchronized`-кода по монитору этого же объекта. Если ожидающих потоков нет, `notify()` и `notifyAll()` ничего не делают. Условие ожидания проверяют в цикле `while`, а не в `if`: из-за ложных пробуждений (spurious wakeups) и потому что к моменту пробуждения условие могло снова стать ложным.

```
synchronized (lock) {
    while (!condition) lock.wait();
}
```

Подробнее: [Взаимодействие потоков: `wait\(\)`, `notify\(\)`, `notifyAll\(\)`](#).

В чем разница между `notify()` и `notifyAll()` ?

Дело в том, что «висеть» на методе `wait()` одного монитора могут сразу несколько потоков. При вызове `notify()` только один из них выходит из `wait()` и пытается захватить монитор, а затем продолжает работу со следующего после `wait()` оператора. Какой из них выйдет - заранее неизвестно. А при вызове `notifyAll()`, все висящие на `wait()` потоки выходят из `wait()`, и все они пытаются захватить монитор. Понятно, что

в любой момент времени монитор может быть захвачен только одним потоком, а остальные ждут своей очереди. Порядок очереди определяется планировщиком потоков Java.

По умолчанию берут `notifyAll()`. Допустим, на одном мониторе ждут потоки по разным условиям (производитель ждёт места, потребитель ждёт данных). Проснувшийся поток проверяет своё условие в цикле `while`: если оно не выполнено, он снова засыпает.

- `notifyAll()` будит всех: каждый проверяет своё условие, нужный поток продолжает работу, остальные засыпают обратно. Всё работает.
- `notify()` будит одного, и это может быть не тот: он проверит условие, увидит, что оно не его, и снова заснёт, а сигнал уже потрачен. Поток, которому он предназначался, так и останется спать.

`notify()` безопасен, только когда все ожидающие равнозначны и достаточно разбудить любого.

Подробнее: [Взаимодействие потоков: `wait\(\)`, `notify\(\)`, `notifyAll\(\)`](#).

Почему методы `wait()` и `notify()` вызываются только в синхронизированном блоке?

`wait()` и `notify()` работают с условием, которое проверяет и меняет разделяемое состояние. Проверка условия и вызов `wait()` должны выполняться **атомарно**. Без монитора между ними может вклиниться другой поток: изменить условие и вызвать `notify()`, когда ожидающих ещё нет. Уведомление пропадёт, и поток, вызвавший затем `wait()`, заснёт навсегда (**lost wakeup**). Монитор делает пару «проверить условие и ждать» неделимой: `wait()` освобождает его уже внутри.

Это проверяется во время выполнения: если поток не владеет монитором объекта, `wait()`, `notify()` и `notifyAll()` бросают `IllegalMonitorStateException`.

```
synchronized (lock) {  
    while (!condition) lock.wait();    // проверка и ожидание под одним  
    монитором  
}
```

Подробнее: [Взаимодействие потоков: `wait\(\)`, `notify\(\)`, `notifyAll\(\)`](#).

Чем отличается работа метода `wait()` с параметром и без параметра?

- `wait()` освобождает монитор и ждёт, пока другой поток не вызовет `notify()` / `notifyAll()`.
- `wait(timeout)` тоже освобождает монитор, но ждёт не дольше `timeout` мс: просыпается по уведомлению или по истечении времени. `wait(0)` равносильно `wait()` (ждать бесконечно), отрицательное значение даёт `IllegalArgumentException`.

Из `wait(timeout)` нельзя узнать, по какой причине поток проснулся (метод возвращает `void`), поэтому условие проверяют заново, а остаток времени считают от дедлайна. После пробуждения поток должен ещё вернуть себе монитор, так что выход из `wait` может задержаться.

```
long deadline = System.nanoTime() +
TimeUnit.MILLISECONDS.toNanos(timeoutMs);
synchronized (lock) {
    while (!condition) {
        long left = TimeUnit.NANOSECONDS.toMillis(deadline -
System.nanoTime());
        if (left <= 0) break;        // таймаут: условие так и не выполнилось
        lock.wait(left);
    }
}
```

Подробнее: [Взаимодействие потоков: `wait\(\)`, `notify\(\)`, `notifyAll\(\)`](#).

Чем отличаются методы `Thread.sleep()` и `Thread.yield()` ?

- `Thread.sleep(ms)` приостанавливает текущий поток на заданное время: он переходит в `TIMED_WAITING`, процессор отдаётся другим потокам. Монитор при этом **не освобождается**: если `sleep()` вызван внутри `synchronized`, остальные потоки ждут вход. Бросает `InterruptedException`, если поток прервали.
- `Thread.yield()` — подсказка планировщику «я готов уступить процессор». Состояние потока не меняется (остаётся `RUNNABLE`), планировщик вправе её проигнорировать, а следующим может быть выбран тот же поток. Ничего не ждёт и ничего не освобождает.

Подробнее: [Цена многопоточности: Переключение контекста \(Context Switch\)](#).

Как работает метод `Thread.join()` ?

`t.join()` блокирует вызывающий поток, пока поток `t` не завершится (вызывающий в `WAITING`, у `join(ms)` — `TIMED_WAITING`). Прерываем: бросает `InterruptedException`.

После возврата всё, что сделал `t`, гарантированно видно вызывающему (**happens-before**).

```
void join()                // ждать завершения
void join(long millis)     // не дольше millis; join(0) –
    бесконечно
void join(long millis, int nanos) // то же, nanos округляются вверх до
    миллисекунд
```

`join(millis)` возвращается и по таймауту, и по завершению потока: различить можно только проверкой `t.isAlive()`.

Подробнее: [`join\(\)` — ожидание завершения потока](#).

Что такое *deadlock*?

Взаимная блокировка (deadlock) — ситуация, когда два или больше потока бесконечно ждут друг друга: каждый держит ресурс, нужный другому. Остальные потоки программы при этом продолжают работать.

Возникает, только когда **одновременно** выполнены четыре условия (Коффман):

- **взаимное исключение**: ресурс в каждый момент может использовать только один поток;
- **удержание и ожидание**: поток держит один ресурс и запрашивает ещё один;
- **невывесняемость**: ресурс нельзя принудительно отобрать, его отпускает только владелец;
- **циклическое ожидание**: есть цикл потоков, каждый из которых ждёт ресурс, занятый следующим (в простейшем случае двое).

```
// поток 1                // поток 2
synchronized (a) {        synchronized (b) {
    synchronized (b) { ... }    synchronized (a) { ... }
}
```

Deadlock возникает не только на `synchronized`, а на любых ресурсах, которые захватываются и ждут друг друга: `ReentrantLock` (JVM находит и такие, см. `ThreadMXBean.findDeadlockedThreads()`), блокировки строк в БД, пул потоков, в котором задачи ждут результатов других задач этого же пула.

Как избежать: нарушить одно из условий. Чаще всего убирают циклическое ожидание: **захватывают ресурсы всегда в одном и том же порядке**. Другие способы:

`tryLock(timeout)` и откат, если лок взять не удалось; меньше вложенных локов, один общий лок вместо нескольких.

Подробнее: [Deadlock — взаимная блокировка](#), [Условия Коффмана](#), [Как найти deadlock в проде](#).

Что такое *livelock*?

Livelock — потоки **не заблокированы** (`Runnable`) и постоянно что-то делают, реагируя друг на друга, но из-за симметричной реакции не продвигаются к цели: состояние меняется, прогресса нет, CPU занят впустую. От `deadlock` отличается тем, что потоки не ждут на мониторе, а активно крутятся.

Часто возникает как побочный эффект борьбы с `deadlock`: например, оба потока при неудачном `tryLock()` отпускают свои локи и сразу повторяют попытку в том же ритме, снова сталкиваясь.

Лечение — нарушить симметрию: случайная пауза (**backoff**) перед повтором.

Реальный пример `livelock`, — когда два человека встречаются в узком коридоре и каждый, пытаясь быть вежливым, отходит в сторону, и так они бесконечно двигаются из стороны в сторону, абсолютно не продвигаясь в нужном им направлении.

Подробнее: [Livelock — «вежливый» deadlock](#).

Как проверить, удерживает ли поток монитор определённого ресурса?

`Thread.holdsLock(obj)` — статический метод: возвращает `true`, если **текущий** поток владеет монитором объекта `obj` (то есть находится внутри `synchronized (obj)`). Для `null` бросает `NullPointerException`.

Типичное применение — проверка инварианта: `assert Thread.holdsLock(this);` в методе, который обязан вызываться под локом.

- Спросить про **другой** поток так нельзя: мониторы чужого потока видны только в `thread dump` или через `ThreadMXBean.dumpAllThreads(true, false)` и `ThreadInfo.getLockedMonitors()`.
- Метод работает только с мониторами `synchronized`. Для `ReentrantLock` он вернёт `false`; там есть `lock.isHeldByCurrentThread()`.

Подробнее: [Монитор объекта](#).

На каком объекте происходит синхронизация при вызове `static synchronized` метода?

У синхронизированного статического метода нет доступа к `this`, но есть доступ к объекту класса `Class`, он присутствует в единственном экземпляре и именно он выступает в качестве монитора для синхронизации статических методов. Таким образом, следующая конструкция:

```
public class SomeClass {  
  
    public static synchronized void someMethod() {  
        //code  
    }  
}
```

эквивалентна такой:

```
public class SomeClass {  
  
    public static void someMethod(){  
        synchronized(SomeClass.class){  
            //code  
        }  
    }  
}
```

- Монитор — `Class`-объект того класса, в котором метод **объявлен**: для `static synchronized` метода `Parent`, вызванного как `Child.m()`, это `Parent.class`.
- `static synchronized` и обычный `synchronized` метод друг друга не блокируют: у них разные мониторы (`Class` и `this`).

Подробнее: [Что даёт `synchronized`](#).

Для чего используется ключевое слово `volatile`, `synchronized`, `transient`, `native`?

- `volatile` — модификатор поля: запись в него happens-before каждого последующего чтения, поэтому читающий поток видит и само значение, и всё, что писавший поток записал до этого. Операции не переносятся через обращение к такому полю «в опасную сторону». На практике это значит: JIT не может держать значение в регистре и вынести чтение за цикл, а запись сопровождается барьером памяти, поэтому она не остаётся в буфере ядра и сразу становится видна другим. Чтение и запись самой

переменной атомарны (включая `long / double`), но **составные операции нет**: `count++` на `volatile int` остаётся гонкой. Подходит для флагов и безопасной публикации. Для ссылки на объект гарантии действуют на саму ссылку и на всё, записанное до её публикации; поля объекта, меняющиеся позже, не защищены.

- **synchronized** — модификатор метода или блок: даёт взаимное исключение (монитор) и видимость (освобождение монитора happens-before следующего захвата того же), а значит атомарность всего блока. К полям и конструкторам не применяется.
- **transient** — поле не сериализуется. **native** — метод реализован в нативном коде (JNI). К многопоточности оба слова отношения не имеют.

Подробнее: [`volatile` — барьер памяти, не атомарность](#).

В чём различия между `volatile` и `Atomic` переменными?

`volatile` даёт только **видимость и порядок**: запись видна другим потокам, но составная операция остаётся несколькими шагами. `count++` — это чтение, прибавление и запись, и параллельные обновления теряются.

`AtomicInteger` (а также `AtomicLong`, `AtomicReference` и др.) хранит значение в `volatile`-поле, то есть даёт ту же видимость, и **плюс** атомарные составные операции: `incrementAndGet()`, `compareAndSet()`, `updateAndGet()`. Они построены на атомарных инструкциях процессора (**CAS**) без блокировок (lock-free).

```
volatile int v;      v++;                                // read-modify-write: обновления
                    // теряются
AtomicInteger a;     a.incrementAndGet();                // один атомарный шаг
```

Атомарность действует только на **одну** переменную: инвариант из нескольких полей по-прежнему защищают `synchronized` или лок. Для сильно конкурируемых счётчиков есть `LongAdder`.

Подробнее: [Черо `volatile` не делает: составные операции](#), [Atomic-типы: `AtomicInteger`, `AtomicReference`, `LongAdder`](#).

В чём заключаются различия между

`java.util.concurrent.Atomic*.compareAndSwap()` и
`java.util.concurrent.Atomic*.weakCompareAndSwap()`.

В API методы называются `compareAndSet()` и `weakCompareAndSet()` (`AtomicInteger`, `AtomicReference` и др.).

- `compareAndSet(expected, new)` — атомарно ставит значение, если текущее равно `expected`, и возвращает `true`; `false` только когда значение действительно другое.

Память — как у `volatile`: даёт `happens-before`.

- `weakCompareAndSet...` — «слабая» версия: **может вернуть `false` без видимой причины**, даже если значение совпало (в документации: например, при конкуренции за память). Поэтому её вызывают только в цикле повторов. Выигрыш возможен там, где сильный CAS сам реализован циклом (архитектуры с LL/SC, например ARM).
- Гарантии памяти у слабых методов выбираются именем: `weakCompareAndSetPlain` — без барьеров, `weakCompareAndSetVolatile` — как `volatile`, есть также `Acquire / Release`. Исходный `weakCompareAndSet()` имеет `plain`-семантику при `volatile`-названии, поэтому с Java 9 он `@Deprecated`.

Подробнее: [CAS и CAS-loop — как работает lock-free, проблема ABA](#).

Что значит «*приоритет потока*»?

Приоритет потока (`1 – 10`) — подсказка планировщику: потоки с большим приоритетом теоретически получают больше процессорного времени. Это не гарантия: JVM отображает приоритеты на приоритеты потоков ОС, и результат зависит от ОС и JVM (на Linux HotSpot по умолчанию их практически не учитывает). Корректность программы не должна зависеть от приоритетов.

- `Thread.MIN_PRIORITY = 1`, `Thread.NORM_PRIORITY = 5` (по умолчанию), `Thread.MAX_PRIORITY = 10`.
- `setPriority(int)` — задать; значение вне диапазона `1–10` даёт `IllegalArgumentException`. `getPriority()` — получить.
- Новый поток **наследует** приоритет потока, который его создал.
- У виртуальных потоков приоритет всегда `NORM_PRIORITY`, `setPriority` игнорируется.

Подробнее: [Цена многопоточности: Переключение контекста \(Context Switch\)](#).

Что такое «*потоки-демоны*»?

Поток-демон — фоновый поток, который не удерживает JVM: JVM завершается, когда не осталось ни одного не-демон потока, а демоны при этом обрываются на месте. Блоки `finally` в них не выполняются, поэтому работа, которой нужно завершиться корректно (запись в файл, коммит), демонам не поручается.

- Статус задают через `setDaemon(true)` **до** `start()`, иначе `IllegalThreadStateException`; `isDaemon()` — проверить.
- Новый поток наследует статус потока, который его создал. `main` — не-демон.
- Примеры демонов: сборщик мусора и другие служебные потоки JVM, потоки `common pool`, все виртуальные потоки.

Можно ли сделать основной поток программы демоном?

Нет. `setDaemon()` допустим только до `start()`, а поток `main` JVM запускает сама до выполнения пользовательского кода, то есть он уже живой. Вызов

`Thread.currentThread().setDaemon(true)` бросает `IllegalThreadStateException`.

Статус демона определяет, будет ли поток удерживать JVM от завершения, поэтому менять его у работающего потока нельзя.

Что значит «усыпить» поток?

Усыпить поток — приостановить **текущий** поток на заданное время: `Thread.sleep(ms)` (с Java 19 также `Thread.sleep(Duration)`, или `TimeUnit.SECONDS.sleep(n)`). Метод статический: усыпить другой поток нельзя, `t.sleep()` усыпит вызывающий поток, а не `t`.

- Поток переходит в `TIMED_WAITING`, процессор отдаётся другим; монитор при этом **не освобождается**.
- Точность ограничена таймерами и планировщиком ОС: фактическая пауза может отличаться от заданной.
- Сон прерывается через `interrupt()`: поток получает `InterruptedException`, флаг прерывания при этом сбрасывается.
- Отрицательное значение даёт `IllegalArgumentException`.

Чем отличаются два интерфейса `Runnable` и `Callable`?

- **`Runnable`** — `void run()`: не возвращает результат и не может бросить проверяемое исключение. Подходит и для `new Thread(runnable)`, и для `executor.execute() / submit()`.
- **`Callable<V>`** — `V call() throws Exception`: возвращает результат типа `V` и может бросать проверяемые исключения. В `Thread` напрямую не передаётся: его отправляют в `ExecutorService.submit()` или оборачивают в `FutureTask`.
- Результат `call()` получают не напрямую: `submit(callable)` возвращает `Future<V>`, а `future.get()` отдаёт значение или `ExecutionException` с исключением, брошенным из `call()`.
- Оба интерфейса функциональные (подходят лямбды). `Runnable` с Java 1.0, `Callable` с Java 5 (`java.util.concurrent`).

Подробнее: [`Callable` и `Future` — задача с результатом](#).

Что такое `Future`?

`Future<V>` — интерфейс (`java.util.concurrent`, Java 5): «ручка» на результат асинхронной задачи, который ещё может быть не готов. Его возвращает

`ExecutorService.submit()` : задача выполняется в другом потоке, а через `Future` результат получают, задачу отменяют и проверяют её статус. Проще говоря, это то, что получаешь сразу вместо результата, а сам результат забираешь позже через `get()` .

```
Future<Integer> f = pool.submit(() -> 42);    // вернулся сразу
// ... своя работа ...
Integer r = f.get();                          // блокируется, пока результат
не готов
```

- `get()` / `get(timeout, unit)` — ждёт результат; исключение задачи приходит как `ExecutionException` , на таймауте бросается `TimeoutException` .
- `isDone()` (любое завершение: результат, исключение, отмена), `cancel(mayInterruptIfRunning)` , `isCancelled()` .
- Реализаций несколько: `FutureTask` , `CompletableFuture` , `ForkJoinTask` .
- Ограничение: результат можно получить только блокирующим `get()` , нет колбэков и цепочек. Это решает `CompletableFuture` .

Подробнее: [`Callable` и `Future` — задача с результатом](#).

Что такое `FutureTask` ?

`FutureTask<V>` реализует `RunnableFuture<V>` , то есть одновременно `Runnable` и `Future<V>` : он оборачивает `Callable<V>` (или `Runnable` с готовым результатом) в задачу, которую можно запустить через `Thread` или `Executor` , а результат взять через `Future` . Именно так `submit()` оборачивает задачи.

Проще говоря, `FutureTask` — это задача вместе с местом, куда будет положен её результат: запустить её можно где угодно, а забрать результат потом через `get()` .

```
FutureTask<Integer> ft = new FutureTask<>(() -> 42);
new Thread(ft).start();    // FutureTask — это Runnable
Integer r = ft.get();      // блокируется до результата
```

- Выполняется **один раз**: повторный `run()` ничего не делает, результат сохраняется.
- Исключение из `call()` сохраняется и бросается из `get()` как `ExecutionException` ; после `cancel()` `get()` бросает `CancellationException` .
- `cancel(true)` прерывает поток, выполняющий задачу (`interrupt()`), в отличие от `CompletableFuture.cancel()` .

Подробнее: [`Callable` и `Future` — задача с результатом](#).

В чем заключаются различия между `CyclicBarrier` и `CountDownLatch` ?

Главное отличие: `CountDownLatch` **одноразовый**, `CyclicBarrier` **многократный**.

Проще говоря: защёлка — «подождать, пока другие закончат», барьер — «все ждут друг друга, чтобы пойти дальше вместе».

- `CountDownLatch(count)` — защёлка-счётчик. Одни потоки вызывают `countDown()` (не блокируется), другие — `await()` и ждут, пока счётчик дойдёт до нуля. Сбросить счётчик нельзя: после нуля `await()` возвращается сразу.

Примером `CountDownLatch` из жизни может служить сбор экскурсионной группы: пока не наберется определенное количество человек, экскурсия не начнется.

- `CyclicBarrier(parties[, action])` — барьер, точка синхронизации. Каждый из `parties` потоков вызывает `await()` и блокируется, пока не придут все. Последний пришедший поток выполняет `action` (если он задан), затем всех отпускают, и барьер готов к следующему циклу.

`CyclicBarrier` является альтернативой метода `join()`, который «собирает» потоки только после того, как они выполнились.

- Если один из участников барьера вышел по таймауту или был прерван, барьер переходит в состояние *broken*: остальные получают `BrokenBarrierException`. Вернуть барьер в исходное состояние можно через `reset()`.

Подробнее: [`CountDownLatch` — защёлка-счётчик](#), [`CyclicBarrier` — барьер для группы потоков](#).

Что такое *data race*?

Data race — формальное понятие из JLS (17.4.5): два обращения к одной переменной из разных потоков, хотя бы одно из них запись, и между ними нет отношения `happens-before`. JMM в таком случае не гарантирует, какое значение увидит чтение: оно может быть устаревшим или результатом переупорядочивания.

```
int x; boolean ready;           // не volatile: между потоками нет happens-
before

// поток 1:  x = 1; ready = true;
// поток 2:  if (ready) print(x);  // JMM допускает ready == true и x == 0
```


Подробнее: [Java Memory Model: happens-before](#), [Демонстрация гонки данных \(Race Condition\)](#).

Что такое *race condition*?

Состояние гонки (race condition) — ошибка, при которой корректность результата зависит от того, как чередуются операции потоков, а этот порядок никто не контролирует. Результат меняется от запуска к запуску, поэтому баг трудно воспроизвести.

Два классических вида:

- **read-modify-write**: `count++` — чтение, прибавление и запись разделимы, и параллельные обновления теряются;
- **check-then-act**: проверили условие, затем действуем, а между ними условие успело измениться (`if (x == null) x = new X();`, `if (!map.containsKey(k)) map.put(k, v)`).

Отличие от **data race** разобрано в вопросе ниже.

Подробнее: [Демонстрация гонки данных \(Race Condition\)](#).

Существует ли способ решения проблемы *race condition*?

Гонку устраняют, убрав общее изменяемое состояние или сделав составную операцию атомарной. Способы, от предпочтительных:

- **Не делить состояние**: неизменяемые объекты, потокоограничение (`thread confinement`: локальные переменные, `ThreadLocal`, один поток-владелец).
- **Атомарные операции**: `AtomicInteger` / `AtomicReference` (`incrementAndGet()`, `compareAndSet()`), атомарные методы коллекций (`putIfAbsent()`, `computeIfAbsent()`, `merge()`).
- **Блокировки**: `synchronized` или `Lock` вокруг всей составной операции целиком (проверка и действие, чтение и запись).
- `volatile` подходит только для флагов и публикации значения: составные операции он не защищает.

Надёжного способа найти все гонки нет: помогают стресс-тесты, `jckit` и ревью, поэтому главное — правильный дизайн.

Подробнее: [Реальные причины гонок данных в Java](#).

Чем *data race* отличается от *race condition*?

- **data race** — про память: обращения к переменной из разных потоков без happens-before, JMM не обещает, что увидит чтение. Лечится появлением happens-before (volatile, synchronized, atomic).
- **race condition** — про логику: результат зависит от того, как чередуются потоки. Лечится атомарностью составной операции.

Они независимы. Пример: volatile убирает data race, но не race condition.

```
volatile int count;
count++;    // data race: НЕТ (volatile даёт happens-before)
           // race condition: ДА (чтение, +1 и запись разделимы, инкременты
           // теряются)
```

Подробнее: [Java Memory Model: happens-before](#).

Как остановить поток?

В Java нет безопасного способа остановить чужой поток принудительно: stop(), suspend() и resume() устарели и в современных JDK бросают UnsupportedOperationException. Остановка **кооперативная**: потоку посылают просьбу, и он сам решает, когда и как завершиться.

- **interrupt()** выставляет флаг прерывания. Поток проверяет его через isInterrupted() (или Thread.interrupted(), который ещё и сбрасывает флаг), освобождает ресурсы и завершается.
- Если поток заблокирован в sleep(), wait(), join() и других прерываемых ожиданиях, interrupt() выводит его оттуда через InterruptedException; флаг при этом **сбрасывается**, поэтому в catch его восстанавливают: Thread.currentThread().interrupt().
- **Свой флаг** (volatile boolean running) тоже работает, но не выводит поток из ожидания, и без volatile другой поток может не увидеть изменение.
- **Блокирующее чтение** из сокета на interrupt() не реагирует: закрывают сокет из другого потока или ставят таймаут. Каналы NIO (InterruptibleChannel) на interrupt() закрываются.
- **Задачи в пуле**: Future.cancel(true), shutdownNow(). Оба работают через тот же interrupt(): cancel(true) прерывает поток, выполняющий эту задачу, а shutdownNow() прерывает все рабочие потоки пула и возвращает список задач, которые не успели стартовать. Поэтому задача остановится, только если сама реагирует на прерывание.

```

while (!Thread.currentThread().isInterrupted()) {
    try {
        doChunkOfWork();
        Thread.sleep(50);
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();    // вернуть флаг, который
        стёрла JVM
        break;
    }
}

```

Подробнее: [`interrupt\(\)` — просьба, а не команда](#), [Классика: флаг остановки](#).

Почему не рекомендуется использовать метод `Thread.stop()` ?

`stop()` асинхронно бросал в целевом потоке `ThreadDeath` в произвольной точке выполнения. Стек раскручивался, и поток отпускал **все** захваченные мониторы сразу. Если в этот момент он обновлял разделяемый объект (списал деньги с одного счёта и не зачислил на другой), объект остаётся в несогласованном состоянии, а другие потоки читают его уже без защиты монитора.

- Остановка не гарантирована: `ThreadDeath` можно перехватить и проигнорировать, он мог прилететь и внутрь `finally`.
- Поэтому `stop()` устарел ещё в Java 1.2, помечен на удаление с Java 18, а с Java 20 просто бросает `UnsupportedOperationException`.

Вместо него — кооперативная остановка через `interrupt()`.

Подробнее: [`interrupt\(\)` — просьба, а не команда](#).

Что происходит, когда в потоке выбрасывается исключение?

Если исключение не перехвачено, **этот поток** завершается (`TERMINATED`); остальные потоки и JVM продолжают работать (JVM выходит, только когда не остаётся не-daemon потоков).

- Исключение получает обработчик: свой (`thread.setUncaughtExceptionHandler`), иначе общий (`Thread.setDefaultUncaughtExceptionHandler`), иначе JVM печатает стек в `System.err`.
- Мониторы `synchronized` освобождаются автоматически, а `ReentrantLock` без `finally` остаётся захваченным навсегда.
- В пулах: при `execute()` рабочий поток умирает, и пул заводит новый; при `submit()` исключение сохраняется в `Future` и бросается из `get()` как `ExecutionException`.

Если `get()` не вызвать, ошибка пропадёт молча.

В чем разница между `interrupted()` и `isInterrupted()` ?

Оба возвращают состояние флага прерывания, но по-разному:

- `Thread.interrupted()` — **статический**, работает только с **текущим** потоком и сбрасывает флаг в `false`.
- `thread.isInterrupted()` — метод экземпляра, можно вызвать для любого потока (в том числе текущего), флаг **не меняет**.

```
Thread.currentThread().interrupt();
Thread.currentThread().isInterrupted(); // true
Thread.currentThread().isInterrupted(); // true: флаг не тронут
Thread.interrupted();                  // true, и флаг сброшен
Thread.interrupted();                  // false
```

В цикле обычно проверяют `isInterrupted()`, чтобы не терять факт прерывания; `interrupted()` вызывают, когда прерывание обработано и флаг нужно снять.

Подробнее: [Флаг прерывания](#).

Что такое «пул потоков»?

Пул потоков — набор заранее созданных рабочих потоков и очередь задач: задачу отправляют в пул, свободный поток её берёт, выполняет и берёт следующую. Это экономит затраты на создание потока (стек, обращение к ОС), ограничивает число потоков и упрощает управление ими.

Основные типы (`java.util.concurrent`):

- `Executor` — интерфейс с одним методом `execute(Runnable)`;
- `ExecutorService` — добавляет `submit()` (результат через `Future`) и завершение работы (`shutdown()`, `shutdownNow()`);
- `ThreadPoolExecutor` — основная реализация с настройкой (`core/max` размеры, очередь, политика отказа);
- `ScheduledThreadPoolExecutor` — отложенные и периодические задачи;
- `ForkJoinPool` — задачи, делящиеся на подзадачи (`work stealing`);
- `Executors` — фабричные методы-обёртки над ними.

Фабричные методы `Executors`:

- `newFixedThreadPool(n)` — n потоков, **неограниченная очередь**; упавший поток заменяется новым;
- `newCachedThreadPool()` — потоки создаются по необходимости, простаивающие минуту завершаются; число потоков **не ограничено**;
- `newSingleThreadExecutor()` — один поток, задачи строго по порядку;
- `newScheduledThreadPool(n)`, `newSingleThreadScheduledExecutor()` — отложенные и периодические задачи;
- `newVirtualThreadPerTaskExecutor()` (Java 21) — виртуальный поток на каждую задачу, без пула.

В продакшене неограниченная очередь или число потоков при перегрузе ведут к исчерпанию памяти, поэтому часто создают `ThreadPoolExecutor` напрямую с ограниченной очередью.

Подробнее: [Преимущества использования пула](#).

Какого размера должен быть пул потоков?

Зависит от типа задач. Отправная точка — число процессоров N (`Runtime.getRuntime().availableProcessors()`):

- **CPU-bound** (вычисления) — порядка N (иногда $N+1$): больше потоков не ускоряют, а только добавляют переключения контекста.
- **I/O-bound** (ожидание сети, БД, диска) — больше N , потому что потоки значительную часть времени ждут: $N * (1 + WT/ST)$, где WT — время ожидания, ST — время обработки на задачу (оценивают профилированием).

Пример: 8 ядер, 10 мс вычислений и 90 мс ожидания БД $\rightarrow 8 * (1 + 90/10) = 80$ потоков.

Размер ограничивают и другие ресурсы: пул соединений с БД, сокет, память под стеки. Формула — отправная точка, окончательный размер подбирают нагрузочным тестом. Для I/O-bound на Java 21+ вместо подбора размера можно взять виртуальные потоки.

Подробнее: [Расчёт размера пула \(CPU-bound vs IO-bound\)](#).

Что будет, если очередь пула потоков уже заполнена, но подаётся новая задача?

Сначала пул пытается создать ещё потоки: когда очередь полна, а потоков меньше `maximumPoolSize`, заводится новый поток. Только если потоков уже `maximumPoolSize` и очередь полна, срабатывает `RejectedExecutionHandler`. Отказ даёт и `execute()`, и `submit()`.

Стандартные обработчики:

- `AbortPolicy` (по умолчанию) — бросает `RejectedExecutionException`;
- `CallerRunsPolicy` — задача выполняется в вызывающем потоке: отправитель замедляется, это естественный back-pressure;
- `DiscardPolicy` — задача молча отбрасывается;
- `DiscardOldestPolicy` — отбрасывается самая старая задача в очереди, новая ставится на её место.

Если очередь неограниченная (`newFixedThreadPool`), она не заполняется и отказа не будет: задачи копятся в памяти.

Подробнее: [Алгоритм обработки входящей задачи](#), [Стратегии отказа](#) (`RejectedExecutionHandler`).

В чём заключается различие между методами `submit()` и `execute()` у пула потоков?

- `execute(Runnable)` — из `Executor`, возвращает `void`. Исключение из задачи доходит до `UncaughtExceptionHandler`: рабочий поток умирает, пул заменяет его новым.
- `submit(...)` — из `ExecutorService`, принимает `Runnable` или `Callable<V>` и возвращает `Future`: через него получают результат (для `Runnable` `get()` вернёт `null`), отменяют задачу и проверяют её состояние. Исключение из задачи **сохраняется в `Future`** и бросается из `get()` как `ExecutionException`; если `get()` не вызвать, ошибка пропадёт молча.

```
pool.execute(() -> { throw new RuntimeException("x"); }); // стек в  
stderr, воркер заменён  
pool.submit(() -> { throw new RuntimeException("x"); }); // тишина, пока  
не вызван get()
```

Подробнее: [Особенности эксплуатации пула](#).

В чём заключаются различия между стеком (stack) и кучей (heap) с точки зрения многопоточности?

- **Стек** — свой у каждого потока: кадры вызовов, локальные переменные и параметры методов (примитивы целиком, для объектов — ссылки). Другие потоки его не видят, поэтому локальные переменные потокобезопасны без синхронизации.
- **Куча** — общая для всех потоков: в ней лежат объекты и их поля. Поэтому именно общие объекты в куче — источник гонок, и их защищают (`synchronized`, `volatile`,

atomic).

Локальная ссылка живёт в стеке, а сам объект — в куче. Пока ссылка не «убежала» из потока (не передана другому потоку, не записана в общее поле), объект принадлежит одному потоку и безопасен; после этого он общий.

```
void f() {  
    int n = 0; // стек: приватно  
    List<String> l = new ArrayList<>(); // ссылка в стеке, объект в куче  
    shared.add(l); // ссылка «убежала»: объект  
    теперь общий  
}
```

Подробнее: [Стек и куча — память потоков: общее и приватное](#).

Как поделиться данными между двумя потоками?

Данные делят через общее состояние (с синхронизацией) или передают через потокобезопасные «каналы»:

- **общий объект** — с защитой (`synchronized` , `volatile` , `atomic`-классы вроде `AtomicReference`); без неё читающий поток может не увидеть запись;
- **BlockingQueue** — классический producer-consumer: один поток кладёт, другой берёт, очередь сама блокирует и синхронизирует;
- **Future / CompletableFuture** — передать результат задачи вызвавшему потоку;
- **Exchanger<V>** — симметричный обмен между ровно двумя потоками: `exchange(x)` блокирует поток, пока второй не вызовет `exchange(y)` , затем каждый получает данные другого.

```
Exchanger<String> ex = new Exchanger<>();  
// поток 1: String got = ex.exchange("A"); // получит "B"  
// поток 2: String got = ex.exchange("B"); // получит "A"
```

Подробнее: [Блокирующие очереди \(`BlockingQueue`\)](#).

Какой параметр запуска JVM используется для контроля размера стека потока?

`-Xss<размер>` , например `-Xss512k` или `-Xss2m` (то же, что `-XX:ThreadStackSize=<КБ>`).
Применяется к новым потокам; по умолчанию 1 МБ на 64-битных платформах.

- Меньше размер — раньше `StackOverflowError` при глубокой рекурсии, зато можно создать больше потоков.

- Больше размер — глубже рекурсия, но каждый поток дороже по памяти (резервируется виртуальное адресное пространство, физическая память выделяется по мере использования).
- Для конкретного потока размер можно подсказать конструктором `Thread(ThreadGroup, Runnable, String, long stackSize)` (JVM вправе его игнорировать). На стек виртуальных потоков флаг не влияет: он хранится в куче и растёт по мере надобности.

Подробнее: [Проблема: OS-потоки дорогие](#).

Как получить дамп потока?

Thread dump — снимок стеков всех потоков JVM: состояние каждого потока, стек вызовов, захваченные и ожидаемые мониторы.

```
jstack <pid>                                # дамп в stdout (jstack -l — ещё и локи
java.util.concurrent)
jcmd <pid> Thread.print                       # то же через jcmd
kill -3 <pid>                                # SIGQUIT: JVM печатает дамп в свой stdout,
процесс не завершается
```

- Из кода: `Thread.getAllStackTraces()`, `ThreadMXBean.dumpAllThreads(...)`.
- Виртуальные потоки `jstack` не показывает: для них `jcmd <pid> Thread.dump_to_file [-format=json] файл`.
- Снимают серию из 3–5 дампов с интервалом в несколько секунд и сравнивают: подозрителен поток, стоящий на одном месте.

Не путать с **heap dump** (дамп кучи, HPROF): он снимается через `jmap` или автоматически при `OutOfMemoryError` флагом `-XX:+HeapDumpOnOutOfMemoryError` и нужен для диагностики памяти, а не потоков.

Подробнее: [Как снять дамп](#).

Что такое *ThreadLocal*-переменная?

`ThreadLocal<T>` — переменная, у которой каждый поток видит своё значение: `get()` и `set()` работают со значением текущего потока. Это изоляция вместо синхронизации: данные не разделяются, значит, блокировки не нужны.

```
static final ThreadLocal<SimpleDateFormat> FMT =
    ThreadLocal.withInitial(() -> new SimpleDateFormat("yyyy-MM-dd"));
```



```
// значение создаётся лениво в каждом потоке
String s = FMT.get().format(date);
```

- Значения хранятся в таблице `ThreadLocalMap` внутри объекта `Thread` (ключ — сам `ThreadLocal`, значение — объект).
- Изолируется **ссылка**, а не объект: если потоки положили ссылку на один и тот же объект, он остаётся общим.
- Значение, записанное через `set()` в одном потоке, другому не видно: из `main` в рабочий поток его не передать (там будет `null` или значение по умолчанию). Для передачи в дочерний поток есть `InheritableThreadLocal` (копирует значение при создании потока).
- В пулах потоки переиспользуются: значение переживает задачу и может «протечь» в следующую или коптить память, поэтому после использования вызывают `remove()` (в `finally`).
- Типичное применение: контекст запроса (пользователь, trace-id, транзакция в Spring), не потокобезопасные объекты вроде `SimpleDateFormat`.

Подробнее: [`ThreadLocal` — изоляция вместо синхронизации](#), [Главная проблема — Thread Pool](#).

Назовите различия между `synchronized` и `ReentrantLock` ?

Оба реентерабельны (повторный захват своего лока увеличивает счётчик, лок отпускается при снятии последнего захвата), оба дают взаимное исключение и `happens-before`. Различия — в возможностях:

Критерий	<code>synchronized</code>	<code>ReentrantLock</code>
Освобождение	автоматически при выходе из блока (в том числе по исключению)	вручную: <code>unlock()</code> в <code>finally</code>
Попытка без ожидания или с таймаутом	нет	<code>tryLock()</code> , <code>tryLock(time, unit)</code>
Прерываемое ожидание	нет (поток в <code>BLOCKED</code> не прервать)	<code>lockInterruptibly()</code>
Справедливость	нет	<code>new ReentrantLock(true)</code> (FIFO, медленнее)
Условия ожидания	один <code>wait/notify</code> на монитор	несколько <code>Condition</code> на один лок

Критерий	<code>synchronized</code>	<code>ReentrantLock</code>
Границы захвата	только блок или метод	захватить в одном методе, снять в другом
Диагностика	нет	<code>isLocked()</code> , <code>getHoldCount()</code> , <code>isHeldByCurrentThread()</code>

```
lock.lock();
try { /* критическая секция */ }
finally { lock.unlock(); }
```

- Производительность в современных JVM в большинстве случаев сопоставима, поэтому выбирают по возможностям: `synchronized` — по умолчанию, `ReentrantLock` — когда нужны таймаут, прерываемость, справедливость или несколько условий.
- На виртуальных потоках (JDK 21–23) `synchronized` закрепляет носитель (pinning), а `ReentrantLock` нет; с JDK 24 это исправлено.

Подробнее: [`ReentrantLock` vs `synchronized`](#).

Что такое `ReadWriteLock` ?

`ReadWriteLock` — интерфейс с двумя методами: `readLock()` и `writeLock()` , каждый возвращает `Lock` (сам он `Lock` **не расширяет**). Два режима одного лока:

- **read lock** — разделяемый: читателей одновременно сколько угодно;
- **write lock** — эксклюзивный: один писатель, читателей в этот момент нет.

Реализация — `ReentrantReadWriteLock` . Имеет смысл, когда читателей заметно больше, чем писателей, а операция под локом не мгновенная; иначе обычный `ReentrantLock` или `synchronized` проще и не медленнее.

```
rw.readLock().lock();
try { return cache.get(k); }
finally { rw.readLock().unlock(); } // запись — то же с rw.writeLock()
```

- **upgrade невозможен**: держа read lock, взять write lock нельзя — поток будет вечно ждать сам себя;
- **downgrade можно**: write lock, затем read lock, затем снять write lock;
- у read lock нет `Condition` ; в нечестном режиме (по умолчанию) возможно голодание читателей или писателей.

Подробнее: [`ReadWriteLock` — читатели и писатель](#).

Что такое «блокирующий метод»?

Блокирующий метод — метод, который не возвращает управление, пока не наступит нужное условие (придут данные, освободится ресурс, истечёт время), и вызывающий поток всё это время простаивает.

- Примеры: `Thread.sleep()`, `join()`, `Object.wait()`, `BlockingQueue.take()/put()`, `Future.get()`, `Lock.lock()`, `Semaphore.acquire()`, `ServerSocket.accept()`, чтение из сокета.
- Большинство прерываемы: объявляют `InterruptedException` и выходят по `interrupt()`; у многих есть версия с таймаутом (`poll(timeout)`, `tryLock(timeout)`). Исключения: чтение из сокета и вход в `synchronized` на `interrupt()` не реагируют.
- Противоположность — **неблокирующие (асинхронные)** методы: возвращаются сразу, а результат приходит позже (`Future`, `CompletableFuture`, колбэк).
- Блокирующий вызов занимает поток: в пуле это съедает рабочий поток, в event loop (Netty, WebFlux) ломает модель; виртуальные потоки делают такое ожидание дешёвым.

Что такое «фреймворк Fork/Join»?

Fork/Join (`java.util.concurrent`, Java 7) — фреймворк для задач «разделяй и властвуй»: крупную задачу рекурсивно делят на подзадачи, подзадачи выполняют параллельно, результаты объединяют.

- **ForkJoinPool** — пул для таких задач. В нём работает **work-stealing**: у каждого воркера своя двусторонняя очередь, он берёт свои задачи с одного конца, а простаивающий воркер «крадёт» задачу с другого конца чужой очереди, поэтому ядра не простаивают.
- **RecursiveTask<V>** (с результатом) и **RecursiveAction** (без) — сама задача: в `compute()` задачу либо считают напрямую (если она мала, порог), либо делят: `fork()` отправляет подзадачу в пул, `join()` ждёт её результат.

```
protected Long compute() {
    if (to - from <= THRESHOLD) return sumDirectly();
    int mid = from + (to - from) / 2;
    SumTask left = new SumTask(arr, from, mid), right = new SumTask(arr,
mid, to);
    left.fork();                                // левую — в пул
    return right.compute() + left.join();       // правую — сами, затем ждём
левую
}
```

- Ждущий `join()` воркер не блокируется, а берёт другую задачу, поэтому хватает потоков по числу ядер.
- Через общий `ForkJoinPool.commonPool()` по умолчанию работают `parallel streams` и асинх-методы `CompletableFuture`.
- Задачи должны быть CPU-bound и не блокироваться (I/O, `sleep`, локи).

Подробнее: [`ForkJoinPool` — пул для «разделяй и властвуй»](#).

Что такое Semaphore ?

Semaphore — синхронизатор со счётчиком **разрешений**: `acquire()` забирает разрешение (блокируется, если их нет), `release()` возвращает. Им ограничивают число потоков, одновременно работающих с ресурсом: пул соединений, вызовы внешнего API (bulkhead).

```
Semaphore sem = new Semaphore(3);           // не больше 3 одновременно
sem.acquire();                               // вне try: если acquire не удался,
release не нужен
try { callExternalApi(); }
finally { sem.release(); }
```

- `tryAcquire()` и `tryAcquire(timeout, unit)` — без вечного ожидания; `new Semaphore(n, true)` — справедливый режим (FIFO).
- Разрешения **не привязаны к потоку**: `release()` может вызвать другой поток, в том числе без предварительного `acquire()` (счётчик просто вырастет). Поэтому `Semaphore(1)` не то же, что мьютекс: у него нет владельца и реентерабельности.

Подробнее: [`Semaphore` — счётчик разрешений](#).

📌 Новые вопросы на проверку

Ниже идут вопросы, добавленные позже (коллекции, `CompletableFuture`, атомики, пулы и др.). Их можно разнести по теме и убрать этот блок.

Чем ConcurrentHashMap отличается от Hashtable и Collections.synchronizedMap() ?

- **Hashtable** и **synchronizedMap**: один монитор на всю карту, любая операция (даже `get`) блокирует остальные. Итерация требует ручного `synchronized (map)`, иначе `ConcurrentModificationException`.

- **ConcurrentHashMap (JDK 8+)**: вставка в пустой бакет через CAS, при коллизии `synchronized` только на первом узле бакета. Чтение без блокировок (`volatile`), поэтому потоки на разных бакетах не мешают друг другу.
- Итераторы **weakly consistent**: не бросают `ConcurrentModificationException`, могут как увидеть, так и не увидеть изменения после создания итератора.
- `null` нельзя ни в ключе, ни в значении (`NullPointerException`): `get() == null` однозначно означает «ключа нет».
- `compute`, `merge`, `putIfAbsent`, `computeIfAbsent` атомарны для одного ключа. Составная проверка `containsKey` + `put` остаётся *race condition* и здесь, вместо неё нужен `putIfAbsent` / `computeIfAbsent`.
- `size()` при параллельных изменениях приблизительный.

```
Map<String, Integer> counts = new ConcurrentHashMap<>();
counts.merge(word, 1, Integer::sum);           // атомарный счётчик по
ключу
```

Подробнее: [ConcurrentHashMap](#).

Когда использовать CopyOnWriteArrayList ?

- Каждая запись (`add`, `set`, `remove`) копирует весь внутренний массив под лок и подменяет ссылку. Чтение идёт без блокировок по текущему массиву.
- Итератор работает по **снимку** массива на момент создания: не бросает `ConcurrentModificationException`, не видит последующих изменений, `Iterator.remove()` не поддерживается (`UnsupportedOperationException`).
- Подходит, когда читают много, а пишут редко и список небольшой: слушатели событий, конфигурация, список подписчиков.
- Не подходит при частых записях: каждая стоит $O(n)$ по времени и памяти.

Подробнее: [CopyOnWriteArrayList](#).

Какие бывают BlockingQueue и как на них строится producer-consumer?

`BlockingQueue` — потокобезопасная очередь, у которой `put()` ждёт места, а `take()` ждёт элемента.

Реализация	Ёмкость	Особенность
<code>ArrayBlockingQueue</code>	ограниченная, массив	один лок на <code>put</code> и <code>take</code> , опциональная справедливость

Реализация	Ёмкость	Особенность
<code>LinkedBlockingQueue</code>	по умолчанию <code>Integer.MAX_VALUE</code> , можно ограничить	два лока (на <code>put</code> и на <code>take</code>), выше пропускная способность
<code>SynchronousQueue</code>	0	прямая передача от <code>producer</code> к <code>consumer</code> , нет хранения; используется в <code>newCachedThreadPool()</code>
<code>PriorityBlockingQueue</code>	неограниченная	порядок по приоритету (<code>Comparable</code> / <code>Comparator</code>)
<code>DelayQueue</code>	неограниченная	элемент доступен только после истечения задержки

- Четыре формы методов: бросают исключение (`add` / `remove`), возвращают `false` / `null` (`offer` / `poll`), блокируют (`put` / `take`), ждут с таймаутом (`offer(e, t, u)` / `poll(t, u)`).
- **Back-pressure**: ограниченная очередь тормозит `producer`'а, когда `consumer` не успевает, и память не растёт бесконечно.

```

BlockingQueue<Task> queue = new ArrayBlockingQueue<>(100);

// producer
queue.put(task);                                // ждёт, если очередь полна

// consumer
Task t = queue.take();                          // ждёт, если очередь пуста

```

Подробнее: [Блокирующие очереди \('BlockingQueue'\)](#). Ручную реализацию см. в вопросах про ограниченный буфер.

Чем `CompletableFuture` лучше `Future` ?

- `Future.get()` блокирует поток, у `Future` нет колбэков, комбинирования и обработки ошибок в цепочке, и его нельзя завершить вручную.
- `CompletableFuture` реализует `Future` и `CompletionStage`: этапы связываются в цепочку без блокировки, результаты нескольких задач комбинируются, ошибки обрабатываются в той же цепочке.
- Завершить вручную можно через `complete()` и `completeExceptionally()`, дождаться набора задач через `allOf()` / `anyOf()`.

```
CompletableFuture.supplyAsync(() -> loadUser(id))
    .thenApply(User::name)
    .thenAccept(System.out::println);
```

Подробнее: [`Callable` / `Future` → `CompletableFuture`: асинхронные результаты.](#)

Чем различаются `thenApply`, `thenCompose` и `thenCombine`? Как обрабатывать ошибки? В каком пуле выполняются этапы?

Метод	Что делает	Аналог
<code>thenApply(f)</code>	преобразует результат функцией <code>f</code>	<code>map</code>
<code>thenCompose(f)</code>	<code>f</code> сама возвращает <code>CompletableFuture</code> , результат «разворачивается»	<code>flatMap</code>
<code>thenCombine(other, f)</code>	объединяет результаты двух независимых этапов	<code>zip</code>

- Если в `thenApply` передать функцию, возвращающую `CompletableFuture`, получится `CompletableFuture<CompletableFuture<R>>`. Для зависимых асинхронных шагов нужен `thenCompose`.
- Ошибки:** `exceptionally(ex -> fallback)` подставляет значение, `handle((res, ex) -> ...)` вызывается всегда, `whenComplete` только наблюдает и результат не меняет.
- Исключение этапа приходит в обработчик обёрнутым в `CompletionException` (причина в `getCause()`). `get()` бросает `ExecutionException`, `join()` бросает `CompletionException`.
- Пул:** по умолчанию `ForkJoinPool.commonPool()`, его потоки `daemon`, поэтому JVM не ждёт их при завершении. Если `parallelism` общего пула равен 1 (по умолчанию число процессоров – 1, то есть на 1–2 процессорах), на каждую задачу создаётся новый поток.
- Для блокирующего IO общий пул не подходит: передают свой `Executor` (`supplyAsync(task, executor)`). Варианты `thenApplyAsync` и подобные запускают этап в пуле, а не в потоке, завершившем предыдущий.

```
CompletableFuture<Integer> total = supplyAsync(() -> userId, io)
    .thenCompose(id -> supplyAsync(() -> loadOrders(id), io)) //
зависимый запрос
    .thenCombine(supplyAsync(() -> loadDiscount(), io), //
независимый запрос
                (orders, discount) -> calc(orders, discount))
    .exceptionally(ex -> 0);
```

Подробнее: [`Callable` / `Future` → `CompletableFuture`: асинхронные результаты](#).

Что такое CAS и проблема ABA?

CAS (compare-and-set) — атомарная операция: «если значение равно ожидаемому, записать новое, иначе ничего». Реализована инструкцией процессора (cmpxchg на x86), в Java доступна как `compareAndSet` у `Atomic*` и `VarHandle`. На ней строятся lock-free структуры: чтение, вычисление, CAS, при неудаче повтор (**CAS-loop**).

ABA: между чтением и CAS значение изменилось $A \rightarrow B \rightarrow A$. CAS видит `A` и успешно проходит, хотя состояние структуры уже менялось. Например, в стеке узел был снят и возвращён обратно с другим `next`.

- Решение: добавить версию. `AtomicStampedReference` сравнивает и ссылку, и номер версии, `AtomicMarkableReference` сравнивает ссылку и флаг.
- В Java ABA с узлами встречается реже, чем в C/C++: пока на узел есть ссылка, GC его не освободит и не переиспользует. Проблема остаётся при пулах объектов и для значений вроде чисел.

```
AtomicStampedReference<String> ref = new AtomicStampedReference<>("A", 0);
int[] holder = new int[1];
String v = ref.get(holder); // значение и
                             версия
ref.compareAndSet(v, "C", holder[0], holder[0] + 1); // успех, только
если и значение, и версия прежние
```

Подробнее: [CAS и CAS-loop — как работает lock-free, проблема ABA](#).

Когда LongAdder лучше AtomicLong ?

- В `AtomicLong` все потоки делают CAS над одной переменной: при высокой конкуренции многие попытки неудачны и повторяются.
- `LongAdder` хранит `base` и массив ячеек (`Cell`): при конкуренции потоки пишут в разные ячейки. `sum()` складывает `base` и все ячейки.
- Результат: при частых записях `LongAdder` быстрее (JDK 21, 2 потока по 10 млн инкрементов: `AtomicLong` около 260 мс, `LongAdder` около 120 мс), но занимает больше памяти.
- `sum()` не атомарный снимок: при параллельных `add()` он может не учесть часть обновлений. У `LongAdder` нет `compareAndSet`.
- Выбор**: счётчики и метрики с частыми записями и редким чтением, то есть `LongAdder`. Когда по значению принимают решение (генератор id, лимит, CAS-логика), нужен `AtomicLong`.

Подробнее: [Atomic-типы: `AtomicInteger`, `AtomicReference`, `LongAdder`](#).

Как правильно остановить пул потоков?

Метод	Что делает
<code>shutdown()</code>	перестаёт принимать задачи (новые вызовут <code>RejectedExecutionException</code>), уже поданные, в том числе из очереди, выполняются
<code>shutdownNow()</code>	прерывает (<code>interrupt</code>) выполняющиеся потоки, снимает не начатые задачи из очереди и возвращает их списком <code>List<Runnable></code>
<code>awaitTermination(t, u)</code>	блокирует до завершения всех задач или таймаута, возвращает <code>true</code> , если пул остановился
<code>close()</code> (JDK 19+)	<code>shutdown()</code> и ожидание завершения; поэтому пул можно создавать в <code>try-with-resources</code>

```
pool.shutdown(); // мягкая
остановка
try {
    if (!pool.awaitTermination(30, TimeUnit.SECONDS)) {
        List<Runnable> notStarted = pool.shutdownNow(); // не успели:
        прерываем и забираем очередь
        pool.awaitTermination(30, TimeUnit.SECONDS);
    }
} catch (InterruptedException e) {
    pool.shutdownNow();
    Thread.currentThread().interrupt();
}
```

- `shutdownNow()` только ставит флаг прерывания: задача остановится, если реагирует на `interrupt`.
- Если пул не остановить, его потоки (они не `daemon`) не дадут JVM завершиться.

Подробнее: [Пулы потоков и `ThreadPoolExecutor` в Java](#).

Почему `Executors.newFixedThreadPool()` и `newCachedThreadPool()` не рекомендуют в продакшене?

В обоих нет ограничения на нагрузку, она скрыта в параметрах по умолчанию (проверено на JDK 21):

Фабрика	core / max	Очередь	Чем грозит
<code>newFixedThreadPool(n)</code> и <code>newSingleThreadExecutor()</code>	n / n	<code>LinkedBlockingQueue</code> без ограничения	очередь <code>runOutOfMemoryError</code> если задача приходит быстрее чем выполняется
<code>newCachedThreadPool()</code>	0 / <code>Integer.MAX_VALUE</code>	<code>SynchronousQueue</code>	число потоков неограничено, в момент всплеска нагрузки <code>runOutOfMemoryError</code> unable to allocate native thread
<code>newScheduledThreadPool(n)</code>	n / <code>Integer.MAX_VALUE</code>	<code>DelayedWorkQueue</code> без ограничения	очередь отложенных задач, размер растёт без ограничений

Лучше создавать `ThreadPoolExecutor` явно: с ограниченной очередью, именованными потоками и осознанной политикой отказа.

```
new ThreadPoolExecutor(
    4, 8, 60, TimeUnit.SECONDS,
    new ArrayBlockingQueue<>(100), // очередь
ограничена
    Thread.ofPlatform().name("worker-", 0).factory(), // понятные
имена в thread dump
    new ThreadPoolExecutor.CallerRunsPolicy()); // при
переполнении задачу выполняет вызывающий поток
```

Подробнее: [Пулы потоков и `ThreadPoolExecutor` в Java](#).

Зачем нужен `Condition` и чем он лучше `wait()` / `notify()` ?

- `Condition` — аналог `wait/notify` для `Lock`: `await()`, `signal()`, `signalAll()`, создаётся через `lock.newCondition()`.
- На одном локе можно создать **несколько** `Condition`, то есть несколько отдельных очередей ожидания (например, `notFull` и `notEmpty`). У `synchronized` монитор один, и `notify()` будит любой ожидающий поток, в том числе «не того».
- `await()` вызывается при удержании лока (иначе `IllegalMonitorStateException`): он отпускает лок на время ожидания и берёт его снова после пробуждения. Условие проверяют в цикле `while`.

- Дополнительно есть `await(timeout, unit)`, `awaitNanos()`, `awaitUninterruptibly()`.

Подробнее: [`Condition` — `wait`/`notify` для `Lock`](#), см. также вопрос про ограниченный буфер с `ReentrantLock`.

Что такое закон Амдала и что ограничивает масштабирование на практике?

Ускорение на N процессорах, если доля P программы параллелизуется:

$$S(N) = 1 / ((1 - P) + P / N) \quad \text{предел при } N \rightarrow \infty: 1 / (1 - P)$$

- При $P = 0.9$ ускорение никогда не превысит 10×, сколько бы ядер ни было (на 8 процессорах только около 4,7×).
- Последовательную часть образуют блокировки, критические секции и общие ресурсы. Поэтому `synchronizedMap` на всю коллекцию убивает масштабируемость, а «просто добавить потоков» не работает.
- На практике потолок задаёт самый узкий разделяемый ресурс: пул соединений с БД (в HikariCP по умолчанию 10), лимиты внешних API, общий лок. Пропускная способность не превышает `min(...)` по звеньям, и ускорить можно только самое узкое.

Подробнее: [Закон Амдала и узкие места в продакшене](#).

Чем отличаются thread-per-request, event loop и виртуальные потоки?

Модель	Потоки	Код	Когда брать
Thread-per-request (Tomcat)	порядка 200 платформенных, блокируются на IO	обычный синхронный	умеренная нагрузка, простота
Event loop (WebFlux, Netty)	порядка числа ядер, не блокируются	реактивный (Mono / Flux)	очень высокий IO-heavy RPS, команда готова к реактивной парадигме
Виртуальные потоки (JDK 21+)	десятки и сотни тысяч, дешёвые	обычный синхронный	IO-heavy нагрузка при желании сохранить привычный код

- Подробнее: [Модели обработки запросов в бэкенде: Thread-per-Request, Event Loop, Virtual Threads.](#)

`ScheduledThreadPoolExecutor` (`Executors.newScheduledThreadPool(n)`) запускает задачи по расписанию:

- **Исключение молча отменяет все дальнейшие запуски** периодической задачи (проверено: после исключения на третьем запуске задача больше не выполнялась, `ScheduledFuture.get()` бросает `ExecutionException`). Тело задачи оборачивают в `try / catch`.
- У старого `Timer` один поток: необработанное исключение останавливает его целиком, а не одну задачу. `ScheduledThreadPoolExecutor` с несколькими потоками надёжнее.

[illegible]

```
    }  
    }, 0, 10, TimeUnit.SECONDS);
```

Подробнее: [Пулы потоков и `ThreadPoolExecutor` в Java](#).

Что такое `StampedLock` и чем он лучше `ReadWriteLock` ?

`StampedLock` — лок с тремя режимами: `write` (эксклюзивный), `read` (общий) и **optimistic read** (без блокировки: читаем, затем проверяем `validate(stamp)`, что запись не происходила).

```
private final StampedLock sl = new StampedLock();  
private double x, y;  
  
double distanceFromOrigin() {  
    long stamp = sl.tryOptimisticRead();  
    double cx = x, cy = y;                                     // читаем без блокировки  
    if (!sl.validate(stamp)) {                                  // кто-то писал: откат на  
        настоящий read lock  
        stamp = sl.readLock();  
        try { cx = x; cy = y; } finally { sl.unlockRead(stamp); }  
    }  
    return Math.hypot(cx, cy);  
}  
  
void move(double dx, double dy) {  
    long stamp = sl.writeLock();  
    try { x += dx; y += dy; } finally { sl.unlockWrite(stamp); }  
}
```

- Оптимистичное чтение не блокирует писателя и не создаёт конкуренцию между читателями: выгодно при очень большом числе коротких чтений.
- **Ограничения:** не реентерабельный (повторный `writeLock()` в том же потоке заблокируется навсегда, а `tryWriteLock()` вернёт `0`), не привязан к потоку, `Condition` не поддерживается (`UnsupportedOperationException`), не реализует `Lock` (есть только обёртки `asReadLock()` / `asWriteLock()`).
- Данные, прочитанные оптимистично, могут оказаться несогласованными: пользоваться ими можно только после успешного `validate()`.

Подробнее: [`StampedLock` — оптимистичное чтение](#).

Что такое *false sharing*?

Два потока пишут в **разные** переменные, но те лежат в одной строке кэша (cache line, обычно 64 байта). Протокол когерентности (MESI) инвалидирует строку в кэше другого ядра при каждой записи, строка постоянно «перелетает» между ядрами, и код тормозит, хотя логически общих данных нет.

- Проверено на JDK 21 (2 потока, `volatile long x` и `volatile long y` в одном объекте, по 20 млн инкрементов): около 700 мс, а с неиспользуемыми полями-«набивкой» между ними около 170 мс.
- **Решение:** padding (неиспользуемые поля между горячими полями) или аннотация `@jdk.internal.vm.annotation.Contended`. Это внутренняя аннотация JDK, для пользовательского кода она включается флагом `-XX:-RestrictContended`.
- По этой причине ячейки `LongAdder` (`Cell`) помечены `@Contended`: соседние ячейки не должны делить строку кэша.

Подробнее: [Кэши CPU: Анатомия задержек](#).

Паттерны и принципы

Что такое *double checked locking Singleton*?

Double-checked locking — ленивая инициализация синглтона с блокировкой только при создании: после создания `getInstance()` идёт без `synchronized`, только с чтением `volatile`-поля.

```
class DoubleCheckedLockingSingleton {
    private static volatile DoubleCheckedLockingSingleton instance;

    static DoubleCheckedLockingSingleton getInstance() {
        DoubleCheckedLockingSingleton current = instance;           // одно
        чтение volatile
        if (current == null) {                                       // 1-я
            проверка: без лока
                synchronized (DoubleCheckedLockingSingleton.class) {
                    current = instance;
                    if (current == null) {                             // 2-я
                        проверка: под локом
                            instance = current = new
                                DoubleCheckedLockingSingleton();
                    }
                }
            }
        }
        return current;
    }
}
```

```
}  
}
```

- **Вторая проверка** нужна, потому что два потока могли пройти первую одновременно: без неё создались бы два экземпляра.
- **volatile обязателен.** `new X()` — это выделение памяти, конструктор и запись ссылки в поле. Без `volatile` запись ссылки может переупорядочиться до конца конструктора, и другой поток увидит ненулевую ссылку на недостроенный объект; `volatile`-запись этого не допускает. До Java 5 идиома не работала вовсе, с Java 5 (JSR-133) работает только с `volatile`.
- Проще и надёжнее — `holder`-идиома или `enum`-синглтон (следующий вопрос).

Подробнее: [Когда `volatile` подходит](#).

Как создать потокобезопасный Singleton?

Способ	Ленивый	Чем обеспечена потокобезопасность	Нюанс
Static field	нет	инициализация класса: JVM делает её один раз под своим локом	проще всего
Enum	нет	то же	защита от рефлексии и сериализации, рекомендация Effective Java
Synchronized accessor	да	<code>synchronized</code> на каждом вызове	лок при каждом чтении: медленно
Double-checked locking + <code>volatile</code>	да	<code>volatile</code> и лок только при создании	сложнее всего, легко ошибиться; зато подходит для поля экземпляра и инициализации с параметрами, а после ошибки при создании следующий вызов повторяет попытку
Holder (On Demand)	да	инициализация вложенного класса при первом <code>getInstance()</code>	лучший ленивый вариант без синхронизации в коде, но только для статического поля без параметров, а ошибка в конструкторе необратима (<code>ExceptionInInitializerError</code> , затем <code>NoClassDefFoundError</code>)

Во всех вариантах, кроме enum, конструктор должен быть `private`.

- **Static field**

```
public class Singleton {  
    public static final Singleton INSTANCE = new Singleton();  
}
```

- **Enum**

```
public enum Singleton {  
    INSTANCE;  
}
```

- **Synchronized Accessor**

```
public class Singleton {  
    private static Singleton instance;  
  
    public static synchronized Singleton getInstance() {  
        if (instance == null) {  
            instance = new Singleton();  
        }  
        return instance;  
    }  
}
```

- **Double Checked Locking & volatile**

```
public class Singleton {  
    private static volatile Singleton instance;  
  
    public static Singleton getInstance() {  
        Singleton localInstance = instance;  
        if (localInstance == null) {  
            synchronized (Singleton.class) {  
                localInstance = instance;  
                if (localInstance == null) {  
                    instance = localInstance = new Singleton();  
                }  
            }  
        }  
        return localInstance;  
    }  
}
```



```
}  
}
```

- **On Demand Holder Idiom**

```
public class Singleton {  
  
    public static class SingletonHolder {  
        public static final Singleton HOLDER_INSTANCE = new Singleton();  
    }  
  
    public static Singleton getInstance() {  
        return SingletonHolder.HOLDER_INSTANCE;  
    }  
}
```

Чем полезны неизменяемые объекты?

Неизменяемый объект (immutable) после создания не меняет состояние, поэтому он **потокобезопасен без синхронизации**: гонок нет, потому что писать нечего. Его можно свободно передавать между потоками, кэшировать, использовать как ключ в `HashMap`.

Видимость обеспечивают `final`-поля: если ссылка на объект не «утекла» до конца конструктора, любой поток увидит корректные значения полей, даже если объект опубликован без синхронизации.

Как сделать класс неизменяемым:

- сам класс `final` (или приватные конструкторы), чтобы наследник не добавил изменяемость;
- все поля `private final`, значения только из конструктора, без сеттеров;
- изменяемые компоненты (массивы, коллекции, `Date`) копируют при приёме в конструктор и при выдаче (`List.copyOf`, `array.clone()`);
- не отдавать `this` из конструктора.

Неизменяемые классы JDK: `String`, обёртки (`Integer`), `BigInteger`, `LocalDate`: «изменение» (`toUpperCase()`, `plusDays()`) возвращает новый объект, а старый остаётся прежним. `record` неизменяем только поверхностно: его поля `final`, но если поле — ссылка на изменяемый объект (например, `List`), содержимое этого объекта изменить можно, поэтому в конструктор кладут копию (`List.copyOf`).

Цена — новый объект при каждом изменении. Общее изменяемое состояние обычно хранят как ссылку на неизменяемый снимок (`AtomicReference<Config>` или `volatile -`

ссылка): изменение подменяет ссылку.

Подробнее: [final-поля — safe publication без volatile](#).

Что такое *busy spin*?

Busy spin (активное ожидание) — поток ждёт условия, крутясь в цикле и проверяя его, вместо того чтобы заснуть (`wait()`, `park()`, `sleep()`). Он не отдаёт ядро, поэтому реагирует на событие мгновенно: нет затрат на пробуждение, переключение контекста и потерю кэша. Цена — ядро занято на 100% всё время ожидания.

```
while (!ready) {                // ready – volatile или atomic, иначе JIT
    // вынесет чтение из цикла
    Thread.onSpinWait();        // подсказка процессору (Java 9+): мы крутимся в
    // ожидании
}
```

- Оправдан для очень коротких ожиданий и низкой задержки: CAS-циклы, spinlock, `BusySpinWaitStrategy` в LMAX Disruptor.
- Вреден, если ждать долго или ядер мало: выжигает CPU и мешает остальным потокам; тогда используют блокирующее ожидание.

Подробнее: [CAS-loop — повторять, пока не получится](#).

Перечислите принципы, которым вы следуете в многопоточном программировании?

- **Давайте потокам осмысленные имена** (`OrderProcessor`, а не `Thread-1`): в thread dump и логах это спасает при отладке; для пулов имена задают через `ThreadFactory`.
- **Не делите состояние, если можно**: неизменяемые объекты, локальные переменные, потокоограничение; общее изменяемое состояние держите минимальным.
- **Сужайте синхронизацию**: критическая секция минимального размера (блок вместо метода), не вызывайте чужой код и блокирующие операции под локом, несколько локов захватывайте всегда в одном порядке.
- **Предпочитайте высокоуровневые средства**: `ExecutorService`, `BlockingQueue`, `CountDownLatch` / `Semaphore` / `CyclicBarrier` вместо ручных `Thread` и `wait()` / `notify()`; `ConcurrentHashMap` и другие concurrent-коллекции вместо `synchronized`-обёрток.
- **Правильно обрабатывайте прерывание**: не подавляйте `InterruptedException`, восстанавливайте флаг (`Thread.currentThread().interrupt()`), освобождайте ресурсы.

- **Не теряйте исключения:** задавайте `UnhandledExceptionHandler`; в пулах `submit()` прячет исключение в `Future`, поэтому вызывайте `get()` или оборачивайте задачу в `try/catch`.
- **Ограничивайте ресурсы:** ограниченные пулы и очереди, таймауты на блокирующих вызовах.

Какое из следующих утверждений о потоках неверно?

1. Если метод `start()` вызывается дважды для одного и того же объекта `Thread`, во время выполнения генерируется исключение.
2. Порядок, в котором запускались потоки, может не совпадать с порядком их фактического выполнения.
3. Если метод `run()` вызывается напрямую для объекта `Thread`, во время выполнения генерируется исключение.
4. Если метод `sleep()` вызывается для потока, во время выполнения синхронизированного кода, блокировка не снимается.

Правильный ответ: **3**.

1. Верно: повторный `start()` на том же объекте `Thread` бросает `IllegalThreadStateException` (в том числе для завершённого потока).
2. Верно: порядок выполнения определяет планировщик, порядок запуска его не гарантирует.
3. **Неверно:** прямой вызов `run()` — обычный вызов метода в текущем потоке: нового потока нет, исключения нет. Запустить поток нужно через `start()`.
4. Верно: `sleep()` не освобождает ни мониторы `synchronized`, ни другие локи (`ReentrantLock` и любые `Lock`), в отличие от `wait()` и `Condition.await()`.

Подробнее: [start\(\) vs run\(\) — в чём разница?](#).

Код с собеседований

Даны 3 потока T1, T2 и T3? Как реализовать выполнение в последовательности T1, T2, T3?

Каждый поток перед работой ждёт предыдущий через `join()`, а запускать потоки нужно в порядке T1, T2, T3:

```
Thread t1 = new Thread(() -> System.out.println("T1"));
Thread t2 = new Thread(() -> { await(t1); System.out.println("T2"); });
```

```
Thread t3 = new Thread(() -> { await(t2); System.out.println("T3"); });
t1.start(); t2.start(); t3.start();          // именно в этом порядке

static void await(Thread t) {
    try { t.join(); } catch (InterruptedException e) {
        Thread.currentThread().interrupt(); }
}
```

`join()` на ещё не запущенном потоке возвращается сразу, поэтому если запустить T3 раньше T2, T3 не станет ждать и порядок нарушится (в проверке при запуске в обратном порядке это случилось примерно в 6% прогонов).

Другие способы:

- `Executors.newSingleThreadExecutor()` — задачи выполняются строго в порядке отправки;
- `CompletableFuture.runAsync(t1).thenRun(t2).thenRun(t3)` ;
- `CountDownLatch` или `Semaphore` : каждый поток открывает следующему.

Подробнее: [`join\(\)` — ожидание завершения потока](#).

Напишите минимальный неблокирующий стек (всего два метода — `push()` и `pop()`).

Стек Трейбера (lock-free): вершина хранится в `AtomicReference` , `push()` и `pop()` повторяют попытку через CAS, пока не получится. Блокировок нет: если CAS не удался, значит другой поток успел изменить вершину, и мы читаем её заново.

```
class NonBlockingStack<T> {
    private final AtomicReference<Node<T>> head = new AtomicReference<>();

    void push(T value) {
        Node<T> recent, node;
        do {
            recent = head.get();
            node = new Node<>(value, recent);
        } while (!head.compareAndSet(recent, node));
    }

    T pop() {
        Node<T> top;
        do {
            top = head.get();
            if (top == null) return null;
        } while (true);
    }
}
```

```

    } while (!head.compareAndSet(top, top.previous));
    return top.value;
}

private static class Node<T> {
    final T value;
    final Node<T> previous;
    Node(T value, Node<T> previous) { this.value = value; this.previous
= previous; }
}
}

```

Вариант проще (тоже корректен, но хуже: узел нестатический, с изменяемыми полями):

```

// class NonBlockingStack<T> {
//     private final AtomicReference<Element> head = new AtomicReference<>
// (null);
//
//     NonBlockingStack<T> push(final T value) {
//         final Element current = new Element();
//         current.value = value;
//         Element recent;
//         do {
//             recent = head.get();
//             current.previous = recent;
//         } while (!head.compareAndSet(recent, current));
//         return this;
//     }
//
//     T pop() {
//         Element result;
//         Element previous;
//         do {
//             result = head.get();
//             if (result == null) {
//                 return null;
//             }
//             previous = result.previous;
//         } while (!head.compareAndSet(result, previous));
//         return result.value;
//     }
//
//     private class Element {        // нестатический: держит скрытую ссылку
на стек
//         private T value;
//         private Element previous;

```

```
// }  
// }
```

- Узел — `static` вложенный класс с `final`-полями: он не держит скрытую ссылку на стек, а `final` и `CAS` дают безопасную публикацию.
- АВА здесь безвредна: на каждый `push()` создаётся новый узел, а память освобождает GC, поэтому «тот же» узел с другим содержимым появиться не может. С пулом переиспользуемых узлов проблема появилась бы.
- `pop()` возвращает `null` на пустом стеке, поэтому `null` как значение класть нельзя.

Подробнее: [CAS-loop — повторять, пока не получится](#), [Проблема АВА](#).

Напишите минимальный неблокирующий стек (всего два метода — `push()` и `pop()`) с использованием `Semaphore`.

`Semaphore(1)` работает как мьютекс: `acquire` — войти в критическую секцию, `release` (в `finally`) — выйти. Стек потокобезопасен, но **не неблокирующий**: потоки блокируются на `acquire`, в отличие от стека Трайбера выше.

```
class SemaphoreStack<T> {  
    private final Semaphore semaphore = new Semaphore(1);  
    private Node<T> head = null;  
  
    SemaphoreStack<T> push(T value) {  
        semaphore.acquireUninterruptibly();  
        try {  
            head = new Node<>(value, head);  
        } finally {  
            semaphore.release();  
        }  
  
        return this;  
    }  
  
    T pop() {  
        semaphore.acquireUninterruptibly();  
        try {  
            Node<T> current = head;  
            if (current != null) {  
                head = current.next;  
                return current.value;  
            }  
            return null;  
        } finally {  

```

```

        semaphore.release();
    }
}

private static class Node<E> {
    private final E value;
    private final Node<E> next;

    private Node(E value, Node<E> next) {
        this.value = value;
        this.next = next;
    }
}
}

```

- Поле `head` не нужно делать `volatile`: `release()` happens-before следующего успешного `acquire()`.
- `acquireUninterruptibly()` не бросает `InterruptedException`, поэтому сигнатуры методов остаются простыми.
- `Semaphore(1)` не реентерабельный: повторный захват тем же потоком заблокирует его самого. Для такой задачи на практике проще `synchronized` или `ReentrantLock`.

Подробнее: [`Semaphore` — счётчик разрешений](#).

Напишите минимальный неблокирующий `ArrayList` (всего четыре метода — `add()`, `get()`, `remove()`, `size()`).

Копирование при записи + CAS (lock-free): массив лежит в `AtomicReference` и никогда не меняется на месте. Запись создаёт копию с изменением и подменяет ссылку через `compareAndSet`; если CAS не удался (другой поток успел), цикл повторяется на свежем массиве. Чтение просто берёт текущий массив.

```

class NonBlockingArrayList<T> {
    private final AtomicReference<Object[]> content = new AtomicReference<>
        (new Object[0]);

    void add(T item) {
        Object[] old, renewed;
        do {
            old = content.get();
            renewed = Arrays.copyOf(old, old.length + 1);
            renewed[old.length] = item;
        } while (!content.compareAndSet(old, renewed));
    }
}

```

```

@SuppressWarnings("unchecked")
T get(int index) {
    return (T) content.get()[index];
}

void remove(int index) {
    Object[] old, renewed;
    do {
        old = content.get();
        if (index < 0 || index >= old.length) throw new
IndexOutOfBoundsException();
        renewed = new Object[old.length - 1];
        System.arraycopy(old, 0, renewed, 0, index);
        System.arraycopy(old, index + 1, renewed, index, old.length -
index - 1);
    } while (!content.compareAndSet(old, renewed));
}

int size() {
    return content.get().length;
}
}

```

- Чтение не блокируется и всегда видит согласованный снимок.
- Каждая запись копирует весь массив за $O(n)$: подходит для редких записей и частых чтений; на практике для этого есть `CopyOnWriteArrayList` (он делает то же, но под локом).

Исходное решение из источника (неверное, оставлено для сравнения):

- `volatile` на массиве даёт только видимость ссылки, а `add()` и `remove()` читают, копируют и записывают без CAS: параллельные записи затирают друг друга (8 потоков по 5000 `add()` дали размер 7 513 вместо 40 000);
- `remove()` содержит баг: условие `index + 1 < size` пропускает копирование хвоста, и `remove(1)` на `[a, b, c]` даёт `[a, null]`;
- `content` читается по несколько раз за вызов, а не один раз в локальную переменную.

```

// class NonBlockingArrayList<T> {
//     private volatile Object[] content = new Object[0];
//
//     NonBlockingArrayList<T> add(T item) {
//         return add(content.length, item);
//     }
// }

```



```

//
// NonBlockingArrayList<T> add(int index, T item) {
//     if (index < 0) {
//         throw new IllegalArgumentException();
//     }
//     boolean needsModification = index > content.length - 1;
//     if (!needsModification) {
//         if (item == null) {
//             needsModification = content[index] != null;
//         } else {
//             needsModification = !item.equals(content[index]);
//         }
//     }
//     if (needsModification) {
//         final Object[] renewed = Arrays.copyOf(content,
Math.max(content.length, index + 1));
//         renewed[index] = item;
//         content = renewed;
//     }
//     return this;
// }
//
// NonBlockingArrayList<T> remove(int index) {
//     if (index < 0 || index >= content.length) {
//         throw new IllegalArgumentException();
//     }
//     int size = content.length - 1;
//     final Object[] renewed = new Object[size];
//     System.arraycopy(content, 0, renewed, 0, index);
//     if (index + 1 < size) {
//         System.arraycopy(content, index + 1, renewed, index, size -
index);
//     }
//     content = renewed;
//     return this;
// }
//
// T get(int index) {
//     return (T) content[index];
// }
//
// int size() {
//     return content.length;
// }
// }

```

Подробнее: [CopyOnWriteArrayList](#).

Напишите потокобезопасную реализацию класса с неблокирующим методом `BigInteger next()`, который возвращает элементы последовательности: `[1, 2, 4, 8, 16, ...]`.

`BigInteger` неизменяем, а каждый вызов создаёт новый объект, поэтому CAS по ссылке корректен: значения никогда не повторяются (ABA невозможна). Если CAS не удался, другой поток успел продвинуть последовательность, и мы повторяем от свежего значения. `BigInteger` нужен, потому что `AtomicLong` переполнился бы после 2^{62} .

```
class PowerOfTwo {
    private final AtomicReference<BigInteger> current = new
AtomicReference<>(null);

    BigInteger next() {
        BigInteger recent, next;
        do {
            recent = current.get();
            next = (recent == null) ? BigInteger.valueOf(1) :
recent.shiftLeft(1);
        } while (!current.compareAndSet(recent, next));
        return next;
    }
}
```

То же короче: `updateAndGet` крутит такой же цикл CAS; лямбда обязана быть без побочных эффектов (её могут вызвать повторно):

```
BigInteger next() {
    return current.updateAndGet(r -> r == null ? BigInteger.ONE :
r.shiftLeft(1));
}
```

Метод lock-free (хотя бы один поток всегда продвигается), но не wait-free.

Подробнее: [CAS-loop — повторять, пока не получится](#).

Напишите простейший многопоточный ограниченный буфер с использованием `synchronized`.

```

class SyncBuffer<T> {
    private final Object[] items;
    private int head, tail, count;

    SyncBuffer(int capacity) {
        items = new Object[capacity];
    }

    synchronized void put(T value) throws InterruptedException {
        while (count == items.length) {           // while, не if: условие
перепроверяется после пробуждения
            wait();
        }
        items[tail] = value;
        tail = (tail + 1) % items.length;
        count++;
        notifyAll();                             // на одном мониторе ждут и
producer'ы, и consumer'ы
    }

    @SuppressWarnings("unchecked")
    synchronized T get() throws InterruptedException {
        while (count == 0) {
            wait();
        }
        T value = (T) items[head];
        items[head] = null;                       // не держим ссылку на объект
        head = (head + 1) % items.length;
        count--;
        notifyAll();
        return value;
    }
}

```

- Один монитор (`this`) защищает всё состояние (`items` , `head` , `tail` , `count`), поэтому `volatile` не нужен.
- Проверка условия и изменение состояния выполняются под одним `synchronized` , а `wait()` освобождает монитор на время ожидания.
- `notifyAll()` обязателен: `notify()` может разбудить поток «не той» стороны (producer вместо consumer), и сигнал потеряется.

Исходный вариант (корректен только для одного producer и одного consumer), оставлен для сравнения:

```

// class QueueSynchronized<T> {
//     private volatile int size = 0;
//     private final Object[] content;
//     private final int capacity;
//
//     private int out;
//     private int in;
//
//     private final Object isEmpty = new Object();
//     private final Object isFull = new Object();
//
//     QueueSynchronized(final int capacity) {
//         this.capacity = capacity;
//         content = new Object[this.capacity];
//         out = 0;
//         in = 0;
//         size = 0;
//     }
//
//     private int cycleInc(int index) {
//         return (++index == capacity)
//             ? 0
//             : index;
//     }
//
//     @SuppressWarnings("unchecked")
//     T get() throws InterruptedException {
//         if (size == 0) {
//             synchronized (isEmpty) {
//                 while (size < 1) {
//                     isEmpty.wait();
//                 }
//             }
//         }
//         try {
//             synchronized (this) {
//                 final Object value = content[out];
//                 content[out] = null;
//                 if (size > 1) {
//                     out = cycleInc(out);
//                 }
//                 size--;
//                 return (T) value;
//             }
//         } finally {
//             synchronized (isFull) {

```

```

//          isFull.notify();
//      }
//  }
//  }
//
//  QueueSynchronized<T> put(T value) throws InterruptedException {
//      if (size == capacity) {
//          synchronized (isFull) {
//              while (size == capacity) {
//                  isFull.wait();
//              }
//          }
//      }
//      synchronized (this) {
//          if (size == 0) {
//              content[in] = value;
//          } else {
//              in = cycleInc(in);
//              content[in] = value;
//          }
//          size++;
//      }
//      synchronized (isEmpty) {
//          isEmpty.notify();
//      }
//      return this;
//  }
// }
//
// Почему сломан при нескольких потоках:
// – проверка размера и ожидание идут на мониторах isEmpty/isFull, а изъятие
// элемента на this: между ними
// состояние никто не держит (check-then-act не атомапен);
// – два consumer'а одновременно видят size == 1, первый забирает элемент,
// второй читает уже пустой слот:
// get() возвращает null, size уходит в минус (put аналогично
// перезаписывает элементы);
// – проверено на JDK 21 (4 producer + 4 consumer, ёмкость 2, 80 000
// элементов): 75 868 значений null.

```

Подробнее: [Взаимодействие потоков: `wait\(\)`, `notify\(\)`, `notifyAll\(\)`](#).

Напишите простейший многопоточный ограниченный буфер с использованием `ReentrantLock`.

```

class LockBuffer<T> {
    private final Object[] items;
    private int head, tail, count;

    private final ReentrantLock lock = new ReentrantLock();
    private final Condition notFull = lock.newCondition();
    private final Condition notEmpty = lock.newCondition();

    LockBuffer(int capacity) {
        items = new Object[capacity];
    }

    void put(T value) throws InterruptedException {
        lock.lockInterruptibly();           // до try: если лок не взят,
unlock() вызывать нельзя
        try {
            while (count == items.length) {
                notFull.await();           // отпускает lock на время
ожидания
            }
            items[tail] = value;
            tail = (tail + 1) % items.length;
            count++;
            notEmpty.signal();             // два Condition: будим только
consumer'ов, signal() достаточно
        } finally {
            lock.unlock();
        }
    }

    @SuppressWarnings("unchecked")
    T get() throws InterruptedException {
        lock.lockInterruptibly();
        try {
            while (count == 0) {
                notEmpty.await();
            }
            T value = (T) items[head];
            items[head] = null;
            head = (head + 1) % items.length;
            count--;
            notFull.signal();
            return value;
        } finally {
            lock.unlock();
        }
    }
}

```

```
}  
}
```

- Два `Condition` (`notFull`, `notEmpty`): `producer`'ы и `consumer`'ы ждут в разных очередях, поэтому хватает `signal()` без `notifyAll()`. Это главное преимущество перед `synchronized`-версией.
- Относительно исходной версии: `lockInterruptibly()` вынесен из `try` (иначе при прерывании `unlock()` бросит `IllegalMonitorStateException` и скроет `InterruptedException`), убраны лишний `if` перед `while`, `volatile` и лок в конструкторе.

Подробнее: [`Condition` — `wait`/`notify` для `Lock`](#), [Зачем несколько условий](#).